

Do We Really Need Imputation in AutoML Predictive Modeling?

GEORGE PATERAKIS, Computer Science Department, University of Crete, Heraklion, Greece

STEFANOS FAFALIOS, JADBIO Gnosis DA S.A., Heraklion, Greece

PAULOS CHARONYKTAKIS, JADBIO Gnosis DA S.A., Heraklion, Greece

VASSILIS CHRISTOPHIDES, ENSEA, ETIS, Cergy, France

IOANNIS TSAMARDINOS, Computer Science Department, University of Crete, Heraklion, Greece

Numerous real-world data contain missing values, while in contrast, most Machine Learning (ML) algorithms assume complete datasets. For this reason, several imputation algorithms have been proposed to predict and fill in the missing values. Given the advances in predictive modeling algorithms tuned in an Automated Machine Learning context (AutoML) setting, a question that naturally arises is to what extent sophisticated imputation algorithms (e.g., Neural Network based) are really needed, or we can obtain a descent performance using simple methods like Mean/Mode (MM). In this article, we experimentally compare six state-of-the-art representatives of different imputation algorithmic families from an AutoML predictive modeling perspective, including a feature selection step and combined algorithm and hyper-parameter selection. We used a commercial AutoML tool for our experiments, in which we included the selected imputation methods. Experiments ran on 25 binary classification real-world incomplete datasets with missing values and 10 binary classification complete datasets in which synthetic missing values are introduced according to different missingness mechanisms, at varying missing frequencies. The main conclusion drawn from our experiments is that the best method on average is the Denoise AutoEncoder on real-world datasets and the MissForest in simulated datasets, followed closely by MM. In addition, binary indicator variables encoding missingness patterns actually improve predictive performance, on average. Last, although there are cases where Neural-Network-based imputation significantly improves predictive performance, this comes at a great computational cost and requires measuring all feature values to impute new samples.

CCS Concepts: • **Computing methodologies** → **Machine learning algorithms**;

Additional Key Words and Phrases: Missing values, imputation, automl, machine learning, optimization

ACM Reference Format:

George Paterakis, Stefanos Fafalios, Paulos Charonyktakis, Vassilis Christophides, and Ioannis Tsamardinos. 2024. Do We Really Need Imputation in AutoML Predictive Modeling?. *ACM Trans. Knowl. Discov. Data.* 18, 6, Article 147 (April 2024), 64 pages. <https://doi.org/10.1145/3643643>

The research work was supported by the Hellenic Foundation for Research and Innovation (H.F.R.I.) under the “First Call for H.F.R.I. Research Projects to support Faculty members and Researchers and the procurement of high-cost research equipment grant” (Project Number: 1941).

Authors’ addresses: G. Paterakis and I. Tsamardinos, Computer Science Department, University of Crete, Heraklion, Crete, Greece; e-mails: gpat@csd.uoc.gr, tsamard.it@gmail.com; S. Fafalios, JADBIO Gnosis DA S.A., Heraklion, Greece; e-mail: stefanosfafalios@gmail.com; P. Charonyktakis, JADBIO Gnosis DA S.A., 100 Nikolaou Plastira str., Vasilika Vouton, Heraklion, Greece; e-mail: haronykt@jadbio.com; V. Christophides, ENSEA, ETIS, Cergy, France; e-mail: vassilis.christophides@ensea.fr.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2024 Copyright held by the owner/author(s).

ACM 1556-4681/2024/04-ART147

<https://doi.org/10.1145/3643643>

1 INTRODUCTION

Real-world data often contain missing values, stemming from faulty sensors, non-responders in questionnaires, incomplete data entry, or other reasons. For example, in the openml portal, as of March 2022, 364 of the 3,487 active datasets contain missing values. Unfortunately, most **Machine Learning (ML)** algorithms demand complete datasets on which to operate.¹ To address this problem, a plethora of imputation algorithms, ranging from simple to very advanced, have been developed to predict the missing values and allow the remaining algorithms in the analysis pipeline to complete.

The problem of imputation has been under study for decades [28, 47, 62]. Initially, it was studied in the context of estimating the coefficients of linear models, call it *estimation perspective*. In contrast, we study imputation from a *predictive modeling perspective* where the goal is to create an accurate model to predict a specific outcome of interest (target variable) in new samples. There are important differences in approaching the subject, under these two perspectives. Under the estimation perspective, (a) some methods would impute the missing values in the training data but would not create an *imputation model* that is able to impute test data [15, 77]. Hence, these methods cannot be applied to predictive modeling. In addition, (b) standard guidelines [67] suggest using the outcome in imputing feature values, e.g., to differentiate imputation values in cases vs. controls. This technique is not applicable in predictive modeling where the outcome is unknown in test samples. Finally, (c) a useful metric of imputation efficacy under the estimation perspective is the *imputation accuracy* [29, 34], i.e., the accuracy of predicting the missing values. Imputation accuracy is important for estimation purposes but may not be indicative of the impact of imputation on predictive performance.

Under the predictive modeling perspective, several interesting questions arise as follows:

- Are advanced predictive modeling algorithms in need of imputation beyond the simple **Mean/Mode (MM)** technique? A non-linear algorithm could potentially learn a rule of the sort “if a feature value equals its mean (i.e., it is missing), then do not use it but instead rely on other observed features values for prediction.” Hence, it is questionable whether imputation would provide an advantage to such an algorithm.
- Is the need for sophisticated imputation further reduced in **Automated Machine Learning context (AutoML)** whereby the most appropriate combination of algorithm and hyper-parameter values (**combined algorithm and hyper-parameter selection (CASH)** optimization) [68] is taking place?
- Do **Binary Indicator (BI)** variables (1 if the value of a feature is missing and 0 otherwise) encoding the missingness patterns provide additional information to a classifier to learn a predictive model?
- How does the feature selection step interact with imputation? Feature selection aims to reduce the number of features that enter the model without sacrificing predictive performance and leads to more interpretable models by providing insights regarding the underlying data generation. It remains open how the benefits of feature selection are impacted when we impute the missing values.
- What is the tradeoff between the computational overhead of imputation and the improvement in predictive performance? Imputation algorithms impute all the missing values, independently of whether they contribute to the predictions of the model. In other

¹Although some ML algorithms such as KNN and Naive Bayes are robust to missing values, their implementations in popular platforms like sklearn does not currently support the presence of missing values.

words, imputation is unsupervised and not guided by the outcome to predict. Hence, they potentially perform a significant amount of unnecessary computations.

- If imputation algorithms indeed improve performance, then are there any characteristics of the datasets (called meta-features) that allow us to predict the value of imputation prior to their analysis and decide whether imputation is worth the computational overhead?

To the best of our knowledge, this is the first empirical study that answers all the above research questions via an experimental evaluation over 25 binary classification real-world datasets, as well as 10 complete datasets in which synthetic missing values are introduced according to different missingness mechanisms, at varying missing frequencies. The MM imputation is used as a baseline and is compared against state-of-the-art representatives of different imputation algorithmic families, namely *Discriminative*, such as Miss-Forest [66], and *Generative*, such as SoftImpute [44] and **probabilistic principal component analysis (PPCA)** [70] exploiting matrix-factorization, or **Generative Adversarial Imputation Nets (GAIN)** [83], and **Denoise AutoEncoder (DAE)** [21] based on Neural Networks. The imputation algorithms are integrated into the **Just Add Data Bio (JADBIO)** AutoML platform [73], which performs CASH and it includes a feature selection step.

In summary, the results show that the single best-performing algorithm is DAE and MissForest for the real and the simulated datasets, respectively. For five of the six imputation algorithms studied, the inclusion of BI variables is beneficial, on average. MM, when BI variables are included and CASH is taking place, is a close competitor and places as the second-best algorithm. Advanced imputation methods do offer a significant advantage but only in a few datasets. In contrast, they require the measurements of all feature values to impute new samples, which in some way invalidates the feature selection step and leads to models of high dimensionality. In addition, they require orders of magnitude more computational time. Meta-level analysis has indicated that only one feature is correlated with the relative performance of the algorithms; unfortunately, the correlation is not statistically significant when corrected for multiple testing. More datasets and new meta-features are needed to extract patterns of when sophisticated imputation should be used over the simple MM.

Overall, in an AutoML setting where optimization is taking place and BI variables are included, MM is a reasonable option; other algorithms should be used only if feature selection is not required and computational time is of little importance relative to improving predictive performance.

The article is organized as follows. Section 2 introduces missing data mechanisms and a taxonomy of imputation families. In Section 3, we present the experimental environment, the selected datasets for evaluation, and the metrics and hyper-parameters tuned. Section 4 describes the missing data generation procedure. The experimental results for real-world data with missing data and simulated missing data are presented in Sections 5 and 6, respectively. In Section 7, we discuss the results of the meta-level analysis on real-world datasets. Related work is discussed in Section 8, followed by the contributions and lessons learned in Section 9. Finally, Section 10 presents the conclusions and limitations of the study. The detailed information about the datasets, missing value simulation setup, and experimental results are provided in Appendices A, B, and C, respectively.

2 BACKGROUND AND CONTEXT

2.1 Missingness Mechanisms

The concept of a *missing mechanism* [62] formalizes the generation process of missing data. In this respect, the BI are modeled as random variables and assigned a distribution. There are three types of underlying mechanisms that generate missing data, namely, **missing completely at random (MCAR)**, **missing at random (MAR)**, and **missing not at random (MNAR)**. For formal definitions of these mechanisms, readers are referred to Reference [40]. Intuitively, MCAR implies that

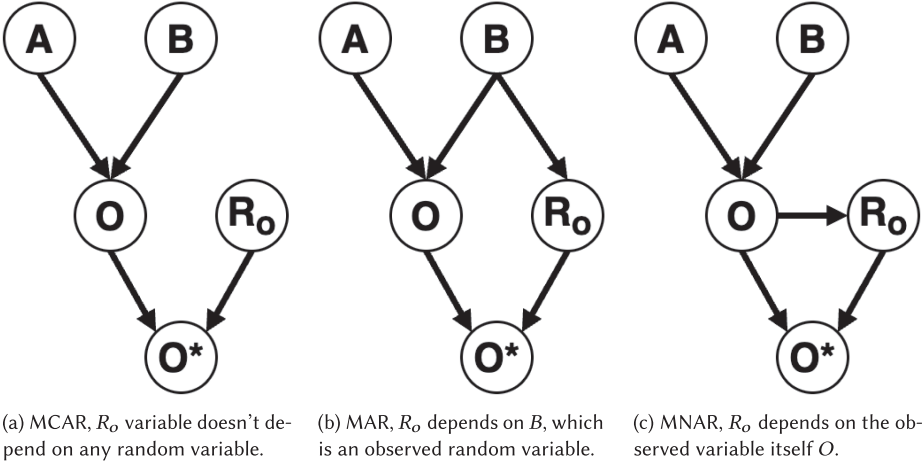


Fig. 1. Panel (a) illustrates the MCAR missingness mechanism. Panel (b) denotes the MAR missingness and panel (c) the MNAR missingness. A , B are random variables, while O is the observed variable of interest. R_o is variable that represents the missingness in O in the form of mask variable. O^* is the result of applying the R_o mask to the O variable.

the probability of a value missing is independent of the actual value, the other observed quantities, and any latent variables. MAR implies that the missingness only depends on the observed data (so it can be predicted). MNAR refers to the case that the missing values are related to both the observed and unobserved variables, including the missing value itself. When missingness is MNAR, it is in principle and in general not possible to impute the missing values in a way that follows the unknown underlying data distribution.²

An illustrative example is given in Figure 1, which is adapted from Reference [46]. The missingness mechanisms can be described using a causal graph. Let us assume A and B are observed random variables and O a latent variable. Each variable is depicted as a node of the graph. Assume that A and B have a direct connection to O , which is the variable (node) of interest. The R_o node is a mask variable that denotes the missingness inserted into O , which causes O^* . O^* is a surrogate of O but with missing values inserted in the positions specified by R_o . As seen in Figure 1(a), MCAR missing values do not depend on any of the variables A , B , O . In contrast, missingness depends on B for the MAR mechanism, and in O itself for the MNAR data, as seen in Figure 1(b) and (c).

2.2 An Imputation Family Taxonomy

There are numerous imputation algorithms and approaches in the literature, and we do not attempt a full review. Readers are encouraged to explore comprehensive surveys available in the field for a more in-depth understanding [2, 4, 12, 13, 24]. Imputation approaches can be partitioned into various distinct families/groups of methods. A taxonomy is attempted in Figure 2. First, imputed values can be decided based on only the feature with the missing value (*Univariate* imputation) or several features (*multivariate* imputation). The former methods include *Mean/Median/Max* imputation for continuous data and *Mode* imputation for categorical data. *Multivariate* methods can be partitioned into *Iterative* and *Distance-Based*, also known as Hot-Deck methods [6]. Distance-based methods employ a distance or a similarity metric for samples to find neighbors or cluster them. A

²Recent work [46] shows that when the causal graph of the distribution is known there are cases where MNAR data can be imputed.

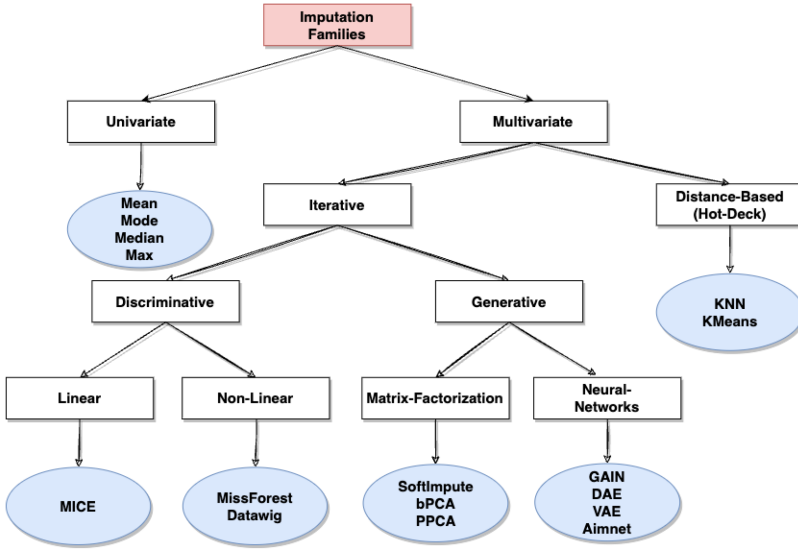


Fig. 2. Taxonomy of imputation families: rectangular nodes represent families, while oval nodes represent algorithms of that family.

commonly used algorithm in this category is the **K-nearest neighbors imputation (KNNi)** [71], which imputes values based on the neighbors of the sample with missing values. *K-means*-based methods cluster the samples before imputation [37].

Iterative methods, start with a simple initial guess (e.g., using MM imputation) and, in each iteration, try to improve the imputed values. We further split iterative methods into *Discriminative* and *Generative*. *Discriminative* methods, build a predictive model per feature with missing values, given the other features in the dataset. This model is used to predict the missing values of the corresponding feature, in each iteration. The Discriminative family can either utilize a (generalized) linear model or a non-linear model. Linear discriminative methods include **Multivariate Imputation by Chained Equations (MICE)** [76]. Non-linear discriminative methods include the *MissForest* algorithm [66] employing Random Forests, and *Datawig* [9] that can impute continuous, categorical, and text data by employing different loss functions according to the missing features' datatype.

Generative methods try to model the joint distribution of the data and use the generative model to impute values. They can be split into two categories, methods that employ matrix factorization and methods that use neural networks. The matrix-factorization family includes low-rank matrix decomposition methods: First, missing values are imputed with an initial guess, and the matrix is decomposed (factorized) and used to predict the missing values. Imputation is improved in each cycle via expectation-maximization steps. Examples of this family include the PPCA [70], SVDImpute [71], bPCA [10], and SoftImpute [44]. Such algorithms scale better w.r.t. to the number of features than MICE or MissForest that train a different model for each feature with missing values in each iteration. Recently, neural networks have also been tried as generative models. These algorithms are essentially non-linear alternatives to matrix factorization. These methods start with an initial guess and then train a neural network that learns the joint distribution. This family includes methods based on **AutoEncoders (AE)**, such as DAE [21, 35, 41] and **Variational Autoencoder (VAE)** [17, 45, 61]. Also, it includes generative adversarial networks (GAIN) [64, 83]. Finally, HoloClean, a data cleaning tool, implements an attention-based neural network for imputation, named

Table 1. Comparison of the Imputation Methods and Their Characteristics

Algorithm	Model Family	Base Model	Learning Procedure	Categorical Handling	Approx. Complexity
MM	Univariate	—	No Iterative	Native	$O(n \cdot m)$
SOFT	Generative	SVD	Iterative	One-hot-encoding	$O(k \cdot n \cdot m \cdot \#sing)$
PPCA	Generative	PCA	Iterative	One-hot-encoding	$O(k \cdot n \cdot m \cdot \#comp)$
MF	Discriminative	RF	Iterative	Native	$O(k \cdot m^2 \cdot n \cdot \log(n) \cdot \#trees)$
GAIN	Generative	GAN	Iterative	Ordinal-Encoding	—
DAE	Generative	AE	Iterative	One-hot-encoding	—

Abbreviations: n is the number of samples, m is the number of features, k is the number of iterations, $\#trees$ stands for the number of trees in the forest(hp), $\#comp$ stands for the number of principal components employed in the matrix factorization, and $\#sing$ for the number of singular values of the SVD.

Aimnet [82]. A detailed comparison of imputation methods is detailed in Section 8. In the next subsection, we will explain the rationale for our choice to include in our empirical study a subset of the aforementioned imputation methods.

2.3 Description of the Selected Imputation Methods

In this section, we present the main characteristics of the imputation methods given in Table 1 that we included in our testbed. In the analysis of their computational complexity, n denotes the number of samples, m the number of features, $\#comp$ the number of principal components, $\#sing$ the number of singular values, and $\#trees$ for the number of trees.

2.3.1 Mean/Mode. MM is the most common imputation method in AutoML tools and is included as the baseline methodology. It is an instance of the univariate imputation family. In the MM algorithm, missing values are imputed with the mean in the training data of the corresponding feature if it is continuous and the mode (most frequent value) if it is discrete. MM is the most computationally efficient method as it needs only $O(n \cdot m)$ to impute the whole dataset. A variation of MM imputation is mentioned in medical literature [51] where missing values of a sample are imputed based on the mean/mode of the class to which it belongs. However, in the case of predictive modeling, this approach becomes problematic as the class of a sample is unknown during inference, as discussed in Section 1.

2.3.2 MissForest. **MissForest (MF)** is a discriminative iterative method based on Random Forests [66]. First, the missing values are imputed by Mean/Mode. Subsequently, for each feature with missing values serving as the outcome, the algorithm trains a random forest on the rest of the features and uses it to predict the outcome's missing values. After imputing all missing values, the algorithm uses the (now) complete dataset to warm-start the new iteration until a stopping criterion is met. MF is one of the slowest methods as it requires building a forest per feature for a number of iterations. The approximate worst case is $O(k \cdot m^2 \cdot n \cdot \log(n) \cdot \#trees)$. MF encounters scalability issues in datasets with more than 50 features. In addition, MF needs to store a forest for each feature, which creates model-storing issues. To avoid such issues, in our experiments we limit the maximum allowed depth of the Random Forests.

2.3.3 Probabilistic PCA. PPCA is a statistical iterative method [39]. In each iteration, a **principal component analysis (PCA)** is performed, which is improved in the next step using maximum likelihood estimation [58] and assuming a multivariate Gaussian distribution of the data. To impute a new sample, the optimal set of principal components found from training is used to identify the missing values that maximize the joint probability of the sample. Categorical features are one-hot-encoded before applying PPCA and then are inverse transformed after PPCA returns the imputed data. PPCA is one of the fastest methods, scaling linearly to the number

of samples, features, and the number of principal components computed. The approximate complexity for PPCA is $O(k \cdot n \cdot m \cdot \text{\#comp})$.

2.3.4 SoftImpute. **SoftImpute (SOFT)** is a statistical iterative method [44]. It starts with the initialization of missing values with the mean. Then, it iteratively solves the optimization problem on the complete matrix using a soft-thresholded SVD and proceeds iteratively until a stopping criterion is met. Categorical features are one-hot-encoded before applying SOFT and then are inverse transformed after SOFT returns the imputed data. SOFT like PPCA is very very fast, utilizing an EM approach. The approximate time complexity is $O(k \cdot n \cdot m \cdot \text{\#sing})$.

2.3.5 Denoise Autoencoder. DAE is a deep learning algorithm based on autoencoders [21]. The Denoise Autoencoder is based on an overcomplete implementation and a dropout layer. DAE projects the input data to a higher-dimensional subspace where the missing data are recovered by the decoder. The categorical data are one-hot-encoded before DAE is applied. Then the one-hot-encoded data are transformed back to the original representation. The complexity of the DAE is mostly measured by the number of epochs needed for the algorithm to impute the dataset accurately and the hidden layers' size and depth. For further information see Section 3.8.

2.3.6 Generative Adversarial Imputation Nets. GAIN is an adaptation of the GAN framework [83]. A generator is used to impute missing data based on the observed data. The discriminator tries to determine which data are observed and which are imputed. The goal of the generator is to provide an accurate imputation whereas the goal of the discriminator is to distinguish between the observed and missing data. The two neural networks are trained in an adversarial process. Categorical data are turned into ordinal features and normalized between 0 and 1. After applying the GANs we revert the categorical data to the original representation by doing the inverse procedure. The complexity of GAIN is mostly bottlenecked by the number of iterations needed to train the GANs. See Section 3.8 for more details.

2.3.7 Binary Indicators. BI is not an imputation method but a feature construction method. Specifically, for each feature F_j with missing values, we construct a new feature, call it I_{jk} , which indicates whether the value at the k th sample of feature j is missing or not. The idea is to encode in I_j the missingness pattern. BIs may help the classifier and allow it to learn whether to trust value F_{jk} for prediction. BI can complement any imputation method. BI's complexity is $O(n \cdot m)$; however, we should note that it increases the complexity of subsequent stages of the ML pipeline by increasing the dataset's dimensionality by a maximum factor of 2. We note that imputation models extended with BI do not use BI to impute the missing data. BI are merged along with the imputed dataset. It is important to mention that BIs are not utilized during the imputation phase for BI-extended methods. Instead, they are added to the imputed dataset.

2.4 Rationale of the Selection of Algorithms

MM imputation is selected as a baseline and one of the most commonly used methods. *MissForest* is selected as a representative of a multivariate iterative imputation method over MICE, based on the results on imputation accuracy presented in Reference [79]. Encoding missingness information using *BI* is also experimentally evaluated, as it performed better than other methods in Reference [43]. Distance-based methods are excluded for various reasons. First, they need to memorize the full dataset to produce imputation as they do not learn a model. k -means and KNN imputation were not included in our testbed as according to other empirical studies are outperformed by *MissForest* [30, 42, 66].

As representatives of matrix completion-based methods, we chose *PPCA* that have shown the best performance according to previous empirical studies [25, 31, 58]. *SoftImpute* was also selected

based on the experimental results presented in Reference [85]. *GAIN* and *DAE* were also included as representatives of neural network–based methods as they excel in several studies [11, 54]. The former is based on using a generative adversarial network to learn the probability distribution to impute and the latter on autoencoder. *VAE* and *Aimnet* were not included based on the inferior or comparable results to *GAIN* and *MM*, respectively [11, 38]. Finally, *Datawig* [9] is not included as, according to the results reported in Reference [9], it is outperformed by *MissForest* for both continuous and categorical data while it comes with a high cost to fit one neural network per feature with missing values.

2.5 How Is Imputation Treated in AutoML Platforms

AutoML platforms employ imputation methods, as well as modeling algorithms that directly treat missing values as a separate category. The current versions of *JADBio* (version v1.4) and *AutoSklern* [16] employ *MM* by default, while *DataRobot*³ may also include *BI* variables. *AutoSklern* allows the user to specify additional imputation methods to optimize over as part of the pipeline. *TPOT* employs median imputation for all missing features [36]. *Auger.AI*⁴ does iterative regression or mean imputation for numerical features depending on the dataset size and creates a new category for the categorical features. *BigML*⁵ by default does not impute the missing value; the missing values are handled internally by their predictive models, which are based on trees only. *Autoprognosis* optimizes the ML pipeline over a variety of missing data imputation algorithms. Specifically, it employs *MICE*, *MissForest*, *Bootstrapped Expectation-Maximization* imputation, *Soft-Impute*, and *MM* [3]. *DriverlessAI* by *H2O* creates a new value to express missingness when the *XGBoost*, *LightGBM*, and *RuleFit* algorithms are used. For generalized linear models, it performs *MM* imputation, while for tensorflow models missing values are treated as outliers [22]. *GAMA* [20] does not impute missing values by default. *Autogluon* [14] uses median imputation for continuous features and introduces a new “Unknown” category for categorical features.

3 EXPERIMENTAL SETUP

We now present the design choices for the experimental setup and the comparative evaluation.

3.1 Datasets

Incomplete Real Datasets. There are currently 364 datasets with missing values in the OpenML repository [78], we restricted our selection to binary classification datasets. We selected 25 binary classification datasets in an effort to cover a range of various dataset characteristics. The datasets contain both continuous and discrete features. The number of features ranges from 7 to 69, the sample size ranges from 155 to 31,406, the prevalence of the minority class ranges from 0.06 to 0.48, the number of features with at least 1% missing values ranges from 1 to 32, and, finally, the percentage of missing values ranges from 1.11% to 71.64%. Table 6 presents the characteristics of the datasets, along with their OpenML id.

Complete Datasets: We selected 10 complete datasets from OpenML, where we introduce and simulate missingness. The number of features ranges from 9 to 135, the sample size ranges from 101 to 5,473, and the prevalence of the minority class ranges from 10% to 49%. Table 7 contains these values for each dataset, along with their OpenML id.

³<https://www.datarobot.com/>

⁴<https://auger.ai/>

⁵<https://bigml.com/>

3.2 Evaluation Task and Metric

We note that the evaluation concerns only *binary classification*. The main metric of predictive performance is the **Area Under the ROC curve (AUC)**. To save space and make interpretation easier, we report classification accuracy and F1-score results in the Appendices C. The datasets are split to 50% training and 50% hold-out test set used only for performance evaluation. Our experiments were conducted only once, due to the computational complexity of the experimental procedure (see Section 3.6). We applied statistical tests to compensate for the lack of repeated experiments. This allows reliable conclusions to be drawn from the experimental results.

3.3 AutoML Environment

To experiment with different imputation algorithms when CASH optimization is taking place, we employed the JADBIO AutoML platform [73]. JADBIO is a commercial product (a version of JADBIO with basic functionality is freely available) but was offered to us for research purposes. JADBIO includes feature selection as part of the ML pipeline and, thus, it can be used to study the effect of feature selection on imputation.

A quick description of JADBIO's architecture now follows. For each dataset to analyze, an internal knowledge base system, called **Algorithm and Hyper-Parameter Space selection (AHPS)** in Reference [73], selects the feature construction, preprocessing, feature selection, and modeling algorithms to try, along with a set of values for their hyper-parameters. The AHPS also selects the configuration evaluation protocol, e.g., 10-fold cross-validation, repeated cross-validation, or hold-out to estimate the performance of each configuration and select the winning one. The knowledge in AHPS is engineered by experienced analysts but also induced by meta-level learning algorithms.

The choices of the AHPS are based on the meta-features of the dataset (e.g., sample size, number of features), as well as the user preferences. For example, an algorithm that does not scale to the number of samples in the current dataset, will not be selected by AHPS. The choice of the evaluation protocol also depends on the meta-features: For a typical-sized dataset, JADBIO may run a 10-fold cross-validation, for a large balanced dataset a hold-out, while for a small sample or an imbalanced dataset, it may run a repeated cross-validation protocol.

Subsequently, JADBIO executes all configurations effectively performing a grid search for CASH optimization. However, JADBIO includes pruning heuristics that may drop a configuration in the early folds of cross-validation if it is not deemed promising, departing from a pure grid search strategy [74]. Once configurations execute, the final model is built on all available data using the winning configuration.

The final performance of the model producing with the winning configuration is the cross-validated AUC *adjusted for the bias incurred due to multiple tries* (called "winner's curse" in statistics). This adjustment is conceptually equivalent to adjusting p -values in multiple hypotheses testing. JADBIO uses the BBC-CV algorithm for the performance estimate adjustment [74]. In Reference [73], experiments on 360 omics datasets of small sample size show that this estimation protocol returns slightly conservative out-of-sample AUC performances of the returned model. Nevertheless, for the purposes of this article, JADBIO's performance estimation was not used; instead, the performances on the 50% held-out set are reported.

Regarding the settings of JADBIO employed in this set of experiments, we note the following. One of the user preferences indirectly controls the execution time and the number of configurations to try and has the settings *Preliminary*, *Typical*, *Extensive*, with *Extensive* trying more configurations and performing a more thorough optimization. *All subsequent experiments were run using the Preliminary setting* to make the computational requirements manageable. The number of configurations may vary between datasets depending on their meta-features, but in our experiments,

it ranges from 900 to over 1,000. The training protocol of JADBio depends on the sample size, the class imbalance, and other factors. For typical-size datasets, JADBio uses a repeated 10-fold cross-validation with #repeats from 1 to 20. A heuristic procedure stops repetitions of cross-validation if no progress is detected. Overall, JADBio uses estimation protocols that execute each configuration between 10 to 200 times per dataset to choose the winning configuration and produce a model.

JADBio optimizes over the following set of algorithms. For feature selection, JADBio uses the Lasso [69] and a variant of the SES algorithm [75] with an upper bound on the number of conditional independence tests to perform. For classification, it optimizes over Ridge Logistic Regression, Decision Tree, Random Forests, and Support Vector Machines with polynomial, linear, and radial basis kernels.

To evaluate imputation algorithms, we embedded them into the JADBio configurations as the second step, after the standardization of continuous features and before feature selection, using the API provided. It is important to note that *configurations are cross-validated as an atom*, and hence, *learning to impute is based only on the training data*. This is necessary to avoid overestimating the performances of configurations and correspondingly, the imputation methods. Each imputation method returns an imputation model that is used to impute the test data before modeling is applied. It is worth noting that even if the feature selection step selects a small subset S of features when some values of S are missing in the test set, the imputation model may impute them based on other features. Hence, even if the predictive model requires just the features in S , the predictive pipeline may require more features. Specifically, all multivariate algorithms selected in the article *require all features to impute*. Hence, the predictive pipeline always requires all features when these algorithms are employed, even with feature selection.

3.4 Imputation Algorithms Implementations

We used the JADBio version 1.4.0 for our experiments. MM and BI methods were already implemented by the developing team of the tool used. For PPCA and SoftImpute, we relied on third-party implementations in R from the PCA methods 1.64.0 [65] and 'softImpute' package version 1.4.1 respectively. We implemented MissForest in python 3.8.4 using the iterativeImputer and RandomForest models from sci-kit learn 1.0.1 [53]. Pytorch 1.7.1 version [52] was utilized for the implementation of GAIN and DAE. We adapted the DAE implementation found at <https://github.com/Harry24k/MIDA-pytorch> to closely follow the description of DAE by the original authors in Reference [21]. We employed the GAIN from <https://github.com/dhanajitb/GAIN-Pytorch>.

3.5 Machine Specifications

The predictive performance experiments of the article were conducted on a fedora-powered VM using 8-core AMD Threadripper 3970x at 3.7 GHz with 12 GB RAM. The neural networks were trained using CPUs. The execution time results reported were measured on an eight-core AMD Ryzen-3600x at 4.6 GHz with 16 GB RAM and Windows 11 OS.

3.6 Computational Resources Employed

During the experiments, more than 41 days of CPU time have been spent training more than 80,000 configurations to conduct the experiments mentioned in the article.

3.7 Availability of Code

The code is available on the Github repository: https://github.com/mensxmachina/Imputation_in_AutoML. The code in the repository consists of scripts for the plots, the datasets, the meta-level analysis as well as the basic implementation of each imputation algorithm.

3.8 Exploring the Hyper-parameter Space of Imputation Algorithms

In the experiments, 24 **hyper-parameter (hp)** value sets were tried for the imputation algorithms: MM (1 hp set), MissForest (2 hp sets), SoftImpute (3 hp sets), PPCA (3 hp sets), DAE (9 hp sets), and GAIN (6 hp sets). The values tried for each hyper-parameter are shown in Table 2. These choices were based on the algorithm's authors' defaults and suggestions. These 24 hp sets were coupled with all other choices of JADBIO multiplying by 24 the number of configurations normally tried. In subsequent experiments, each of these 24 hp sets is run on the original dataset, as well as the dataset with the inclusion of the BI features, leading to 48 different combinations. MM has no parameters and therefore does not need tuning. For MissForest, we train RF models with 250 trees, which offers higher imputation accuracy according to Reference [66]. However, we restrict the maximum depth of the tree and maximum leaf nodes, because the trained model had storing memory issues (see Section 2.3.2). SoftImpute and PPCA require selecting the number of principal components to use as a hyper-parameter. The majority of papers in the literature fails to report the tuning of the aforementioned methods that led us to develop the following heuristic: We select as many components required to explain $x\%$ of the data variance. The values of x are shown in Table 2 as the values of "variance-explained." The default hyper-parameters are used for DAE with the exception of the dropout layer and the hidden layers' dimensions. The range of the dropout layer is based on Reference [63], while the theta value is tuned within a neighborhood of the author's suggested default value. In the current implementation, we have three hidden layers for the encoder and the decoder. For each successive layer in the encoder, θ hidden layer nodes are added and hyperbolic tangent is used as the activation function, as it produces better results for small and medium-sized datasets [21]. The model is trained using Stochastic Gradient Descent with an adaptive learning rate with a time decay factor of 0.99 and Nesterov's accelerated gradient. GAIN architecture consists of three hidden layers for the discriminator and the generator while using Rectified Linear Unit as the activation function. For GAIN, we tune two hyper-parameters; alpha and hint rate. These hyper-parameters are considered the most important for GAIN. Alpha balances the loss between the discriminator and the generator, while the hint rate is responsible for the training of the discriminator. Both DAE and GAIN are trained at the specific epochs as suggested by authors and use the sigmoid activation function for the output layer.

4 SIMULATING MISSING DATA

To experiment with a ranging percentage of missing values, as well as different missing mechanisms, we simulated the presence of missing values in the complete datasets presented in Appendix A.2.

4.1 Simulating Missing Completely at Random Data

Under MCAR, missing values are missing with a given probability (percentage) independently of any other factors such as the value itself or the values of other features. To simulate missing values at a realistic missingness percentage we sampled 64 real-world datasets from the OpenML repository with varying characteristics (see Section B.1). We then computed the 25%, 50%, and 75% quantiles of missingness percentages. Features with less than 1% of missing values were excluded from the calculation, as they probably point to features that missing values from typos or non-systematic sources. The quantile values turn out to be about 10%, 25%, and 50% of missingness. The quantile values are then used to vary the missingness percentages in both MCAR and MAR simulation experiments. We then introduced missing values with the given percentages at the 10 complete datasets described in Section 3.1. Even though it is trivial to introduce MCAR missing values, for consistency reasons, we employed the code available in Reference [48], which is also

Table 2. The Set of Values Tried for Each Hyper-Parameter Tuned

Algorithm	Hyper-parameter	Value
Mean/Mode	—	—
MissForest	n-trees	250
	maxDepth	<u>20</u> , 30
	maxLeafNodes	30
SoftImpute	variance-explained	50%, 70%, <u>90%</u>
PPCA	variance-explained	50%, 70%, <u>90%</u>
DAE	dropout	0.25, 0.4, <u>0.5</u>
	batch-size	64
	θ	5, <u>7</u> , 10
	epochs	500
GAIN	alpha	0.1, <u>1</u> , 10
	hint-rate	0.5, <u>0.9</u>
	batch-size	64
	epochs	10.000

For each algorithm, all combinations of values for its hyper-parameters shown were tried and combined with all other choices for feature selection and modeling by JADBio. There are 48 combinations of imputation algorithms and hyper-parameter values. The default hyper-parameter values are underlined.

used for the MAR simulations below. To simulate the MCAR mechanism the software discards values uniformly at random from the dataset at the specified missingness percentage.

4.2 Simulating Missing at Random Data

Under MAR, missing values are missing with a probability (percentage) that depends (is conditional) on other observed features, i.e., $P(I_j = 1 | F_{k_1}, \dots, F_{k_m})$. To realistically simulate data under MAR, one needs to decide (a) the number of features upon which the probability depends, (b) the functional form of the conditional probability function, and (c) the set $\{F_{k_1}, \dots, F_{k_m}\}$. To answer (a) we needed a realistic estimate of the number m of features in the conditional probability. To that end, in the corpus of the 10 real-world binary datasets of Table 7, we randomly selected one feature with missing values as the target feature and then performed predictive modeling using JADBio including feature selection.⁶ These experimental results suggest that, on average, a feature with missing values is dependent on 12 features, so we set $m = 12$. Subsequently, for each F_j we randomly selected a set of m other features with uniform probability. Finally, for the functional form of P , we used a logistic regression model: $P(I_j = 1 | F_{k_1} = f_1, \dots, F_{k_m} = f_m) = \frac{1}{1 + e^{(-w \cdot f)}}$, where w is a set of randomly chosen coefficients from a normal Gaussian distribution, f is the vector of values of the features F_{k_l} , and $\langle \cdot, \cdot \rangle$ denotes the inner product. For the simulation, the software [48] was also used. The software allows the simulation of MAR missing data, as described above, with prespecified missingness percentages. The same percentages as in the MCAR case were used.

⁶Table 9 contains the dataset name, the missing feature, and the number of features selected for the missing feature as the outcome.

Table 3. p -values of the Matched t -test and the q -values after FDR Correction (Sorted)

Base Imp.Method	p -value	q -value
MF	0.036	0.182
PPCA	0.064	0.182
GAIN	0.091	0.182
MM	0.148	0.222
SOFT	0.31	0.375
DAE	0.552	0.552

Only MF has p -value < 0.05 . GAIN and PPCA have p -value < 0.1 . Setting the q -value threshold to 0.25 leads to accepting the hypothesis that BI's are beneficial for four algorithms (MF, GAIN, PPCA, MM), expecting a 25% (one of four) of these discoveries to be false on average.

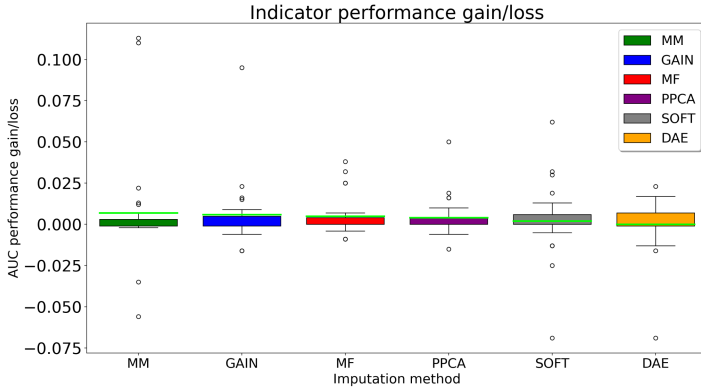
5 COMPARATIVE EVALUATION ON REAL-WORLD DATASETS WITH MISSING VALUES

The 25 real-world datasets with missing values were analyzed with JADBio, optimizing over configurations that include the imputation algorithms selected and their hyper-parameter values.

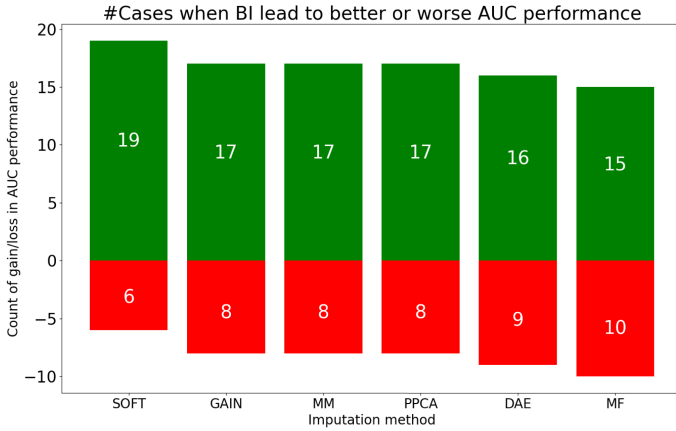
5.1 Binary Indicators Improve the Predictive Performance

First, we partition results achieved when optimizing over any single imputation algorithm. Specifically, for each imputation algorithm, on a given dataset, the best AUC was selected over all configurations that include the specific algorithm. *We will refer to this best AUC simply as the AUC of a given imputation algorithm, in all subsequently reported results.* Figure 3(a) shows the difference in AUC performance when Binary Indicators are used versus when excluded. As we can see, MM and GAIN have the largest average increase by 0.0074 AUC and 0.0056 AUC, respectively. MF when extended with BI shows an average AUC increase of 0.0046, while PPCA shows an increase of 0.0038 AUC. The lowest average improvement is achieved by SOFT, which improves by 0.00022 AUC. Contrary to the above observations, DAE is the only method that does not benefit from the addition of BIs with a negligible decrease of 0.0004 AUC when BI's are included. Figure 3(b) offers a complementary view. It illustrates the count of datasets per imputation method where the inclusion of BI is beneficial to the downstream performance. We observe that for every imputation method, including BI is beneficial in most instances. SOFT exhibits improvement across 19 of 25 datasets. GAIN, MM, and PPCA in 17 datasets. Finally, including BI, leads to enhancements in DAE and MF across 16 and 15 datasets, respectively.

To determine the statistical significance of the results, we performed a paired matched t -test for each algorithm with the null hypothesis H_0 being that the BI+base has worse performance than the base method. The resulting p -values were converted to q -values with the Benjamini/Hochberg [7] method, to control for multiple testing. Table 3 shows the results. Using a q -value threshold of 0.25 there are four statistically significant results, resulting in accepting the alternative hypotheses that MF, GAIN, PPCA, and MM improve their performance when BIs are present. At the level of $q = 0.25 = \frac{1}{4}$ this implies that, in the worse case, we expect one of these four discoveries to be false. While the inclusion of BIs may, in the worse case, double the dimensionality of the dataset, based on the above results, we would recommend their inclusion when the above imputation algorithms are employed.



(a) AUC performance gain for each imputation method when BIs are included. The horizontal line with lime color, denotes the mean AUC gain/loss of including binary indicators for the specified imputation method. BI indicators increase the predictive performance on average for all but one imputation method (DAE).



(b) For each method (x-axis), the green bars denote the count of datasets (y-axis) for which including BI in the base method improves performance. The red bars denote the count of datasets for which the performance deteriorates by including BI variables. In the majority of cases, adding BIs is beneficial to the downstream task performance.

Fig. 3. Panel (a) denotes the performance of each imputation method when including binary indicators minus the performance without binary indicators for each dataset. Panel (b) denotes the count of datasets per imputation method that indicators improve or deteriorate the performance.

5.2 BI+DAE Is the Best Imputation Method in Real-world Data

Figure 4 shows the average ranking achieved by each algorithm using the Autorank tool [26] (lower ranking is better). To avoid clutter, and based on the results of Section 5.1, we only show results when BIs are included. The horizontal black bars in the graph connect tools with non-statistically different ranks, according to a non-parametric Friedman test and post hoc Nemenyi test.

Results show that BI+DAE is the highest ranking algorithm with an average rank of 2.84, followed by BI+MM with a 2.94 average ranking, BI+MF with 3.5, and BI+GAIN with 3.56, although their

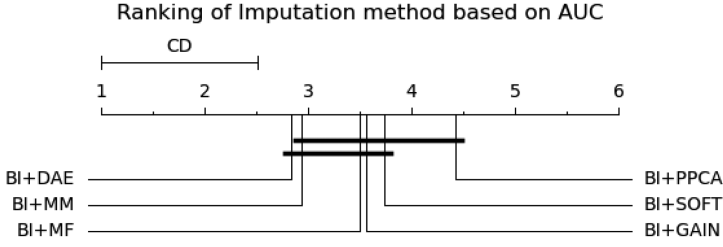


Fig. 4. The average rank of each imputation method when binary indicators are used. BI+DAE has the lowest average ranking. Rank differences are not statistically significant except for the average rank between BI+DAE and BI+PPCA.

rank difference is not statistically significant at the 0.05 level. The two lowest-ranked methods are BI+SOFT and BI+PPCA, with 3.74 and 4.42 average rankings, respectively. BI+DAE's rank is statistically significantly lower compared to BI+PPCA.

5.3 BI+MM Is the Best Method When Considering the Efficiency–Effectiveness Tradeoff

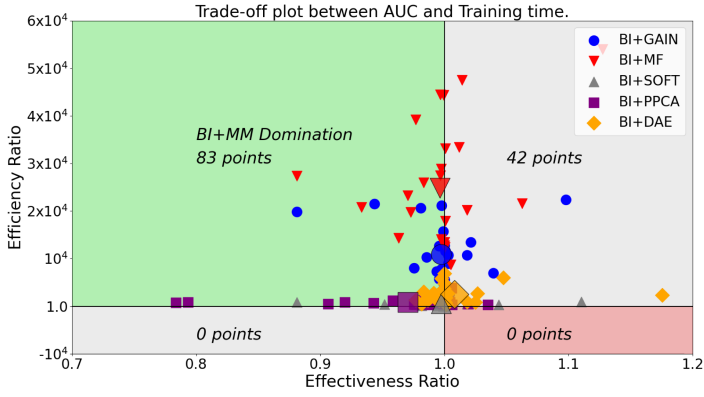
We now study the performance effectiveness vs. the computational efficiency tradeoff of the algorithms. In Figure 5(a) we use MM as the baseline. A point (execution run) corresponds to AutoML predictive modeling on a dataset with a given imputation algorithm. This results in $5 \times 25 = 125$ points. The x -axis shows the effectiveness ratio defined as the ratio of the AUC corresponding to the point divided by the corresponding performance of BI+MM. Similarly, the y -axis shows the efficiency ratio defined as the training time of the point divided by the corresponding time of BI+MM. Hence, points in the first/fourth quadrant (top-left/bottom-right) correspond to runs where BI+MM dominates/is-dominated by other algorithms on the same datasets in both time and AUC. Notice that the scale of the y -axis is logarithmic. Larger points correspond to the mean value of an imputation method over all datasets.

In total BI+MM is inferior in terms of predictive performance in 42 cases (16 of the 25 datasets) and, unsurprisingly, never gets dominated in terms of AUC performance and efficiency at the same time. The computational time of the other algorithms is orders of magnitude slower than BI+MM. However, in 83 of 125 points, BI+MM is both more efficient and effective than the compared method. Only BI+DAE scores on average higher than the AUC score. All the other imputation methods are on average slower to train and worse in terms of predictive performance.

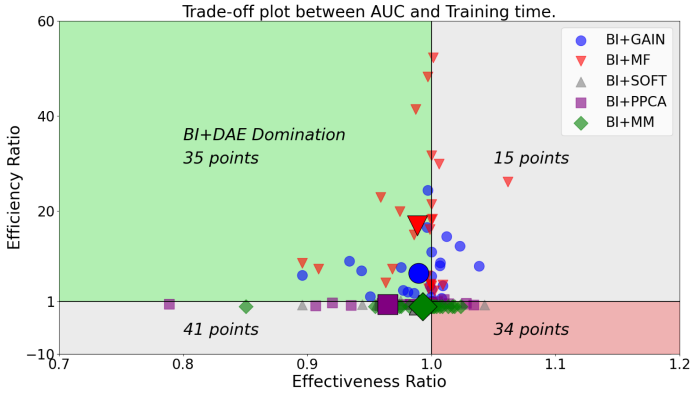
Figure 5(b) shows the same exact results with BI+DAE as the baseline. In contrast to BI+MM above, BI+DAE dominates the other imputation methods on predictive performance and training time, in only 35 of 125 combinations. In 41 of 125 cases, it provides better predictive performance but at a higher computation cost. In 15 points, BI+DAE is faster but has lower predictive performance than the compared imputation method. Finally, 34 times it is dominated in both metrics. In conclusion, when *if a single imputation algorithm is to be used, BI+MM arguably provides the best tradeoff between computational time and predictive performance.*

5.4 Best Imputation Subset for Maximizing AUC Performance Is {BI+MM, BI+DAE}

In this section, we examine the results from a different perspective, trying to answer the question: What is the minimal-size subset of algorithms to try to achieve close-to-maximum AUC performance? To answer this question, we have implemented a simple greedy algorithm, where we assume the analyst starts with the subset {BI+MM} as an efficient baseline and adds algorithms to



(a) The Figure illustrates the relative efficiency for each imputation method against BI+MM versus the relative effectiveness in terms of AUC. Larger points indicate the mean values for a given algorithm. 83 out of 125 points fall in the top-left quadrant where BI+MM dominates other algorithms in both efficiency and effectiveness. Only BI+DAE outperforms BI+MM in effectiveness by 0.8% on average, however, it is 2500 times slower than BI+MM.



(b) The same analysis as above, instead of BI+MM, BI+DAE is used as the reference algorithm. 34 out of 125 points fall in the bottom-right quadrant where BI+DAE is dominated in both metrics.

Fig. 5. Panel (a) denotes the efficiency-effectiveness tradeoff by using BI+MM as reference algorithm. Panel (b) illustrates the efficiency-effectiveness tradeoff when BI+DAE is used as reference. Each point in panels (a) and (b) represents a dataset. The x -axis shows the effectiveness, defined as the ratio of the AUC achieved by the marked imputation method divided by the corresponding performance of the reference method, for a dataset. Similarly, the y -axis shows the efficiency that is defined as the training time of the marked imputation method divided by the time of the reference method for a dataset.

consider. In each iteration, the algorithm that leads to the largest AUC improvement of the subset when added is selected for inclusion. The maximum AUC performance is the sum of the maximum AUC for each dataset when including all imputation methods in the optimization pipeline, averaged across all datasets.

The results are shown in Figure 6 and quantitatively in Table 11. The x -axis shows the imputation algorithms in order of addition to the subset. For each algorithm, several hyper-parameter

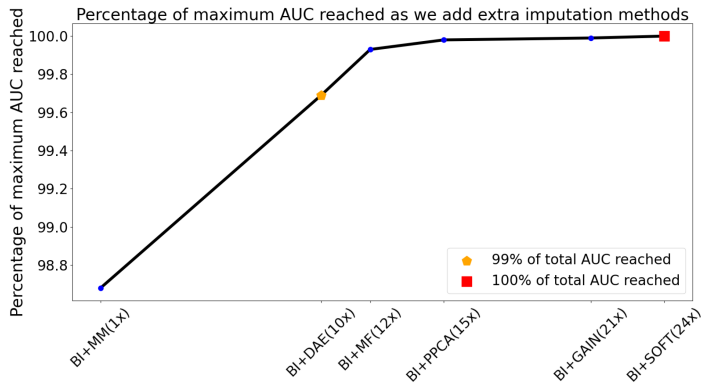


Fig. 6. The percentage of maximum AUC achieved by each subset of imputation methods. Each tick in the x -axis shows the algorithm to add to the previous subset. The multiplier next to the name of an algorithm shows the factor by which the total number of configurations tried is multiplied, due to the different combinations of hyper-parameter values of the imputation algorithms. BI+DAE and BI+MM allow us to recover 99.69% of the maximum AUC while increasing the configuration space by 10 times.

combinations are tried and combined with all other feature selection and modeling choices by AutoML. Hence the total number of configurations tried is multiplied by this factor. At each tick, the multiplication factor for the whole set is depicted in the parenthesis next to the name of the algorithm added to the set in that step. For example, BI+MM has no hyper-parameters (1 $\times$), while BI+DAE has 9, so the multiplicative factor of the set $\{BI + MM, BI + DAE\}$ is 10 $\times$. The y -axis is the average (over all datasets) relative AUC achieved when performance is optimized over all algorithms and their hyper-parameters in the corresponding subset.

BI+MM, by itself, accounts for 98.69% of the maximum AUC. When BI+DAE is added to the mix, relative performance reaches 99.69%. The next best algorithm to add is BI+MF; 100% of AUC is reached when invoking all imputation algorithms. *In summary, the addition of BI+MF, BI+PPCA, BI+GAIN, and BI+SOFT provide only marginal gains to the set $\{BI+MM, BI+DAE\}$.*

5.5 The Interplay between Feature Selection and Imputation

Feature selection algorithms try to reduce the number of features that enter the model without sacrificing predictive performance. Feature selection is often the *primary task* in analysis, while the predictive model may be just a side-benefit. For example, a medical doctor may be more interested in the quantities that determine the risk of disease and may reveal new medical knowledge, rather than the risk model itself. Feature selection leads to more interpretable models that provide intuition into the domain. In fact, the solution to the feature selection problem is directly linked to the causal model that underlies data generation [72]. In other circumstances, it is important to reduce the cost of measuring the features to provide predictions. The cost may be measured in monetary units, the computational cost to compute the features or risk to a patient from medical procedures that measure these features.

Figure 7 shows the impact of feature selection for each imputation algorithm on the real dataset. The drop in AUC performance when feature selection is enforced vs. not enforced (i.e., optimizing over all configurations) in the final configuration is shown. For each algorithm is about two to three AUC points. In other extensive experiments with hundreds of complete (no missing values), small-sample, high-dimensional omics datasets, JADBio has been shown to reduce the number

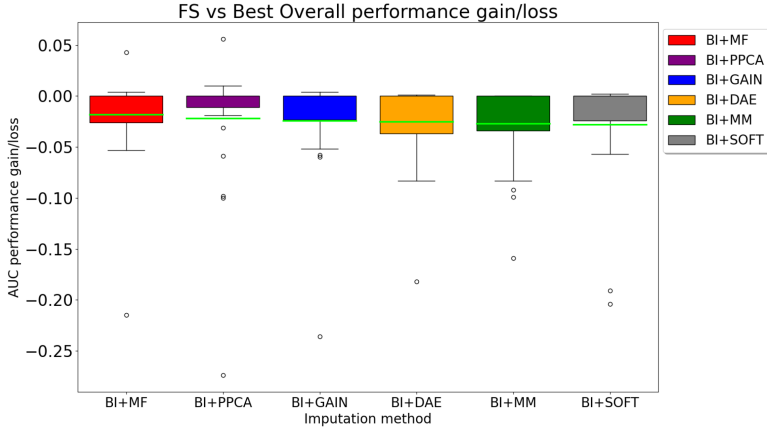


Fig. 7. The loss in predictive performance when enforcing feature selection in the pipeline for the real-world data. For all imputation methods, enforcing feature selection leads to a drop in predictive performance (red lines) by less than three AUC points (on average). While feature selection can reduce the required features to measure for an acceptable loss of performance in some applications, it is invalidated by the imputation models that need to measure all features.

of features by a factor of 4,000 without a noticeable drop in AUC performance [73]. The results provide evidence that feature selection may be more challenging in the presence of missing values.

In any case, the problem of including both imputation and a feature selection step in the ML pipeline is that *imputation invalidates feature selection*, in some sense. Let us explain this statement with an example. Let us assume the pipeline that produces the final model consists of MF imputation, Lasso feature selection, and RF predictive modeling. Let us assume that Lasso selects the features $\{A, B, C\}$. If any of these values (say the value of A) is missing on a new sample, then the MF imputation model will impute them using a Random Forest for A using some other subsets of features. If any of those are also missing, then MF will invoke its Random Forests for each value that is missing, and so on, recursively. Hence, *if there are missing values on the test samples, one may need to measure an arbitrarily large feature subset, not just the selected features*. The storage required to apply the ML pipeline includes both the RF as well as the MF model, which in turn includes a Random Forest for every feature that may need imputation.

6 COMPARATIVE EVALUATION ON DATASETS WITH SIMULATED MISSING VALUES

This section focuses on comparing imputation methods in datasets with generated missing values. To that end, we compare the predictive performance of each imputation method under various missingness mechanisms and percentages. Additionally, we study the effect of feature selection when the missingness increases. The figures in this section illustrate the more general MAR case. The results for MCAR results are included in Appendix C.2 and are qualitatively similar. Finally, results regarding imputation accuracy can be found in Appendix C.7.

6.1 BI+MF Is the Best Imputation Method in MCAR and MAR Simulated Missing Data

Figure 8 presents the AUC performance results (see Figure 16(b) for MCAR results). The AUC performance denoted is the absolute difference in performance at the specified missingness percentage minus the AUC performance of the complete dataset. First, we note that the figure illustrates that as the missingness percentage increases the average predictive performance for every imputation method used decreases, as expected. As we can see, increasing the missingness from 25% to

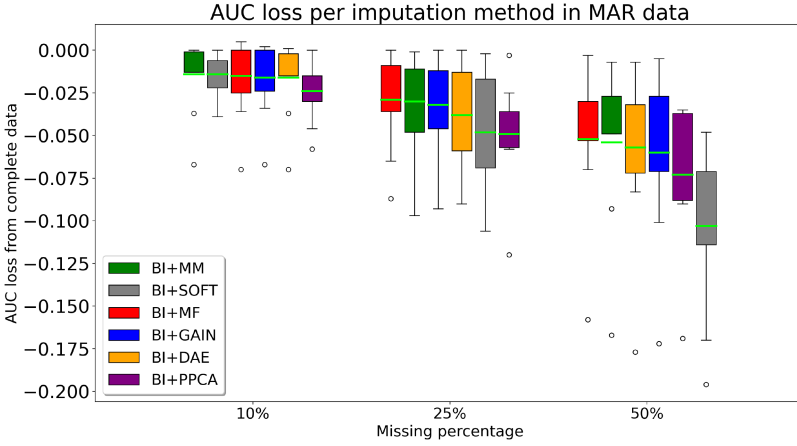


Fig. 8. MAR data: AUC difference of each imputation method from the complete dataset. BI+MM is the best at 10% missingness. BI+MF and BI+MM are the top methods, tied at 25% missingness. BI+MF has the lowest avg. loss at 50%.

50% leads to a sizable performance drop for all imputation methods. Specifically, methods based on linear dimensionality reduction, namely BI+PPCA and BI+SOFT are the most affected by this increase in missing values.

Figures 8 and 16(b) illustrate that in both MAR and MCAR data, respectively, *MissForest combined with Binary Indicators* is, on average, the best-performing method. Additionally, we note that PPCA and SOFT are the two worst imputation methods, especially as the missingness percentage increases. Table 12(a) in Appendix C.2 contains the quantitative results in detail and a detailed discussion on the ranking of the algorithms.

6.2 The Best Imputation Subset for Maximizing AUC Performance Is {BI+MM, BI+MF}

We now identify the minimal-size algorithm subset with close-to-optimal performance for simulated missing data. We use again the simple greedy algorithm introduced in Section 5.4 and apply it to the MCAR and MAR simulated data results. The results for MAR are in Figure 9, which is similar to Figure 6. The quantitative results are shown in Table 13(b). As shown in the figure, the {BI + MM, BI + MF} subset can score over 99% of the total max AUC for MAR data and would be the suggested set of algorithms to run in such problems. The results for MCAR are in Appendix C.2.6. They are qualitatively similar. The results for simulated missing values are somewhat different than the ones in the real datasets, namely BI+MF scores better than BI+DAE, which is now placed in third place. Possible reasons why are discussed in Section 9.

6.3 BI+MM Provides the Best Tradeoff between Effectiveness and Efficiency

Figures 10 and 16(a), show the effectiveness vs. efficiency tradeoff of the algorithms. The aforementioned figures are similar to Figure 5(a) above for the real datasets. We repeat the explanation of the figure: The reference (baseline) algorithm is BI+MM. The x -axis shows the effectiveness ratio defined as the ratio of the AUC corresponding to the point divided by the corresponding performance of BI+MM. Similarly, the y -axis shows the efficiency ratio defined as the training time of the point divided by the corresponding time of BI+MM. Hence, points in the first/fourth quadrant (top-left/bottom-right) correspond to runs where BI+MM dominates/is-dominated-by other algorithms on the same datasets in both time and AUC. Notice that the scale of the y -axis is

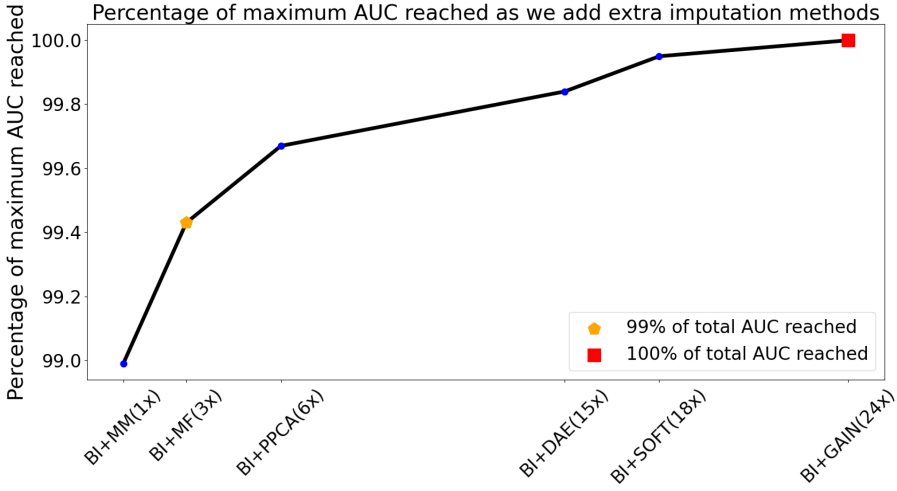


Fig. 9. The percentage of maximum AUC achieved by each subset of imputation methods. Each tick in the x-axis shows the algorithm to add to the previous subset. The multiplier next to the name of an algorithm shows the factor by which the total number of configurations tried is multiplied, due to the different combinations of hyper-parameter values of the imputation algorithms. The set containing BI+MF and BI+MM recovers 99.43% of the maximum AUC for MAR data. The complexity of the configuration space increases by 3 times when including both MM and MF.

logarithmic. Larger points correspond to the mean value of an imputation method over all datasets. There are five imputation methods to compare against MM for 10 datasets over 3 percentages of missing values. This will naturally result in 150 points. However, MissForest did not run in three of the datasets (image dataset variations) due to its dimensionality; see Section 2.3.2 for details. The resulting plot will consist of 147 points.

For MAR data, BI+MM is never dominated in both metrics, as it is by far the most efficient method. In 104 of 147 cases, it dominates the opposing imputation methods in terms of both effectiveness and efficiency. However, 43 times it is dominated in efficiency. Only BI+MF has on average better predictive performance than BI+MM. However, it is 23,000 times slower to train on average. All the other imputation methods are worse than BI+MM on average while also taking more time to train. The results for MCAR data are qualitatively similar (see Figure 16(a) and discussion in Appendix C.2).

In total, BI+MM is again found to provide the best, arguably, tradeoff between efficiency and effectiveness. The results in the simulated data are further validating the results in the real-world data, verifying that BI+MM is indeed a decent imputation method all around. *BI+MM, on average, is on par with more sophisticated methods such as BI+MF, BI+GAIN, and BI+DAE, while being thousands of times faster to train.*

7 META-LEVEL ANALYSIS OF REAL-WORLD RESULTS

In Machine Learning it is always invariably the case that there is no single better algorithm for all datasets, a one-size-fits-all type of algorithm. Hence, one needs to optimize over several choices for the dataset at hand. This school of thought is what gave rise to AutoML systems. The field of Meta-Level Learning [18] studies how to predict the most promising algorithm or algorithms to run on a given dataset based on its characteristics. These characteristics are called *meta-level*

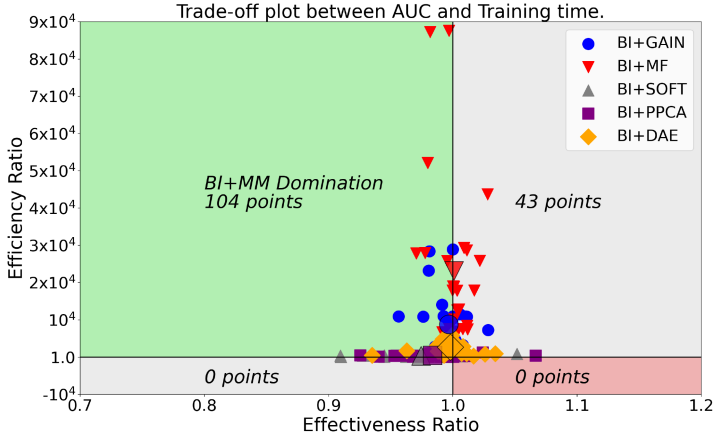


Fig. 10. The relative efficiency for each imputation method against BI+MM versus the relative effectiveness in terms of AUC. Larger points indicate the mean values for a given algorithm. One hundred four of 147 pairs are won by BI+MM in both efficiency and effectiveness. Only MF dominates MM on average in terms of effectiveness. However, MF is 23.000 $\times$ slower than MM.

features or meta-features of the dataset and include the sample size, the number of features, the type of features, the percentage of missing values, and others [60].

In this section, we try to identify meta-features that correlate with the performance of the imputation algorithms. Such correlations could help predict which algorithms to run on a given dataset. They could also shed light on the dataset properties that enable an algorithm to perform better and lead to the design of better algorithms. Hence, we defined and computed the meta-features in Table 4. The selected meta-features can be split into three categories: (1) General meta-features, which report general characteristics of the dataset such as the number of samples or features. (2) Missing value-related meta-features, which provide insight into the dataset's missing patterns, such as missing value percentage of features. (3) Cluster-based meta-features. One such type of metric is the silhouette coefficient, computed with the k -means algorithm with $k = 2, 3, 4$, as was proposed in Reference [1]. It shows the tendency of the data to cluster. Another type of such metric is the number of PCA components that explain $\%x$ of the data. It shows whether the data are limited to a lower-dimensional subspace and the extent of cross-correlations between features. General meta-features were extracted using the pymfe package [5]. We implemented the missing and clustering-based meta-feature extraction using sklearn [53] and numpy [23]. To apply clustering or PCA the data are first imputed with MM.

We then correlated (Spearman correlation) these meta-features with the AUC performance of an algorithm relative to the performance of BI+MM as the baseline. A positive (negative) correlation indicates that when the meta-feature increases, the performance of the algorithm increases (decreases), relative to BI+MM. There are five algorithms (except BI+MM, which is used as a baseline) and 16 meta-features, leading to 80 correlations over datasets. Only one correlation was found to be significant at the level significance 0.1 (p -value = 0.059). Specifically, *BI+PPCA relative AUC performance is positively correlated (correlation = 0.383) with the number of categorical variables in a dataset*. This means that as the number of categorical variables increases we expect BI+PPCA to perform better relative to BI+MM. However, when correcting the p -values for multiple testing using the FDR control technique of Benjamini-Hochberg [7], we see that the q -value is 0.991, which means that detecting one such correlation is expected even if all meta-features are uncorrelated

Table 4. Meta-features Used in the Meta-level Analysis

Name	Category	Description
inst_to_attr	General	Samples to features ratio
Minority Class %	General	% of minority class
nr_attr	General	Number of features
nr_inst	General	Number of samples
n_num	General	Number of numerical features
n_cat	General	Number of categorical features
% NA	Missing	% of missing values in data
% samples /w NA	Missing	% of samples with missing values
% features /w NA	Missing	% of features with missing values
% NA/Feat. /w (NA 1+%)	Missing	Mean % of missing values per feature with more than 1% missing
# components 50%	Clustering	Number of components that explain 50% of data variance
# components 70%	Clustering	Number of components that explain 70% of data variance
# components 90%	Clustering	Number of components that explain 90% of data variance
Slh(k=2)	Clustering	Mean Silhouette Coefficient of all samples when using 2 clusters
Slh(k=3)	Clustering	Mean Silhouette Coefficient of all samples when using 3 clusters
Slh(k=4)	Clustering	Mean Silhouette Coefficient of all samples when using 4 clusters

The first column contains the name of the meta-feature, the second column denotes the category of the meta-feature, and the third column provides a brief explanation of the meta-feature.

with the relative performance. BI+PPCA does not handle categorical features natively, which further makes us believe that the result is probably a false positive. Statistically significant correlations could not be found using meta-learning analysis. Further experiments containing more datasets and meta-features need to be conducted.

8 RELATED WORK

In this section, we discuss related work on missing values imputation and position our contributions. We focus on empirical studies that compare different imputation methods based on the performance of the predictive models build on imputed datasets rather than the original values of complete datasets [13, 29, 56, 80].

Current literature can be split into two categories: AutoML and Adhoc ML modeling. The first category extends a specific AutoML tool by adding imputation methods, while the latter creates a predictive modeling pipeline that may contain a subset of a modern AutoML tool's pipeline, such as hyper-parameter optimization, model selection, and pre-processing. AutoML in general is able to optimize the performance over various stages in a pipeline. As we optimize the whole pipeline, we expect the effect of each stage to become less significant, as other stages may compensate. AutoML tools allow us to get more insights on which features are more important for the task (feature selection), optimize the hyper-parameters for each stage of the pipeline (hyper-parameter tuning), and select the best predictive model for each imputation method (model selection). Consequently, imputation methods can be evaluated fairly under this optimization framework.

As shown in Table 5, the majority of related work either uses datasets with native or simulated missing values. The literature mainly focuses on the binary classification task (included in all previous works). Of the eight previous works, two papers include binary+regression [8, 55], and one work binary and multi-class data [19]. Reference [30] is the only study that includes all three types of outcomes. The most prominent missingness mechanism is MCAR found in all works that simulate missing values. Reference [30] is the only work that includes deep learning-based imputation methods. Binary Indicators are very prominent in AutoML tools; however, only Reference [55] has studied their effect when extending imputation methods. Finally, Reference [49] is the only work

Table 5. An Overview of Related Work on Predictive Modeling

Study	#Datasets	Mechanism	% Missing values	#Imp. methods	NNs	BI	System	FS	Tuning	#Models	Eval	Metric	Meta
[38]	6B	Nat	7–84%	3N, 2C, 2M.	No	No	Adhoc	No	Imp+Pred	7	R-TT (70-30)	ACC-F1	No
[81]	13B	Nat	0.6–33.6%	2N, 1C, 4M	No	No	Adhoc	No	No	5	TT (80-20)	AUC-F1	No
[57]	2B	MC, MR	0–40% **	7C	No	No	Adhoc	No	No	3	TT (66.6-33.3)	ACC	No
[8]	5B, 5R	MC-MN	10–50%	8M	No	No	Adhoc	No	Imp	4	R-TT (50-50)	ACC-R2	No
[30]	31B, 21R, 17M	MC, MR, MN	1–50%*	6M	Yes	No	Adhoc	No	Imp	2	CV (3-5)	RMSE-F1	No
[55]	10B, 3R	Nat	—	7M	No	Yes	Adhoc	No	Pred	3	NCV	ACC	No
[19]	23B, M	MC	7%	6M	No	No	AutoML	Yes	—	10	R-TT (75-25)	ACC	No
[49]	5B	Nat	—	4N, 2C, 1M	No	No	AutoML	—	Imp+Pred	Ensemble	CV (5)	B-ACC	No
Ours	35B	N, MC, MR	1–72%	6M	Yes	Yes	AutoML	Yes	Imp+Pred	4	TT (50-50)	ACC-F1-AUC	Yes

Most benchmarks either use datasets with simulated missing values or with native but not both. Abbreviations: # symbol means number, “—” denotes that the paper does not mention any details about the topic, on column data B, M, R denotes binary, multiclass, and regression datasets, respectively. On the column mechanism, values Nat, MC, MR, and MN denote Native, MCAR, MAR, and MNAR. On column #Imp. methods, N, C, and M denotes numerical, categorical, and mixed imputation method, respectively. The NNs column denotes that Neural Network imputation methods were included. BI means that methods were extended with BIs, FS means feature selection was included in the pipeline, and Tuning represents whether the study tuned the imputation methods (Imp), the predictive models (Pred), or both (Imp+Pred). Eval column presents the evaluation methodology, R denotes repeated, TT: train-test split and number in parenthesis the percentages of the train and test set, respectively, CV: Cross-validation and the number in parenthesis the number of folds, NCV denotes Nested Cross Validation. The metric column denotes the metric used for the evaluation methodology, ACC is classification accuracy, B-ACC is balanced accuracy, F1 is F1-score, RMSE is the root mean squared error, and AUC is the Area under the ROC curve. Meta presents whether a study has conducted a meta-level analysis.

*One of the features was made missing. *missing values were only generated on the train data.

that included ensemble models for the prediction phase while Reference [19] is the only work that includes feature selection as part of the pipeline. As shown in the Table 5, none of the related work has included every step mentioned in the table’s columns.

Summarizing the related work section, the majority of the literature uses datasets with native missing values or generates them through a simulation based on various missingness mechanisms and missingness proportions. However, none of the mentioned studies benchmarks imputation methods on both native and generated missing value datasets. The studies on real-world datasets in general conclude that simple imputation methods such as MM are on par with other more complex methods. Research on datasets with simulated missing values concludes that more complex methods can indeed improve predictive performance on average. However, there is no universal best method proposed by any of the aforementioned benchmarks. Literature mainly focuses on the binary classification task. Accuracy and F1-score are the more prominent metrics in the literature. In the majority of the studies, the hold-out split is used for the evaluation. Some studies, use repeated splits or cross-validation to handle randomness. Specific predictive models could benefit from native handling of missing values compared to simple imputation, for instance, Gradient Boosted Trees. However, not all classifiers support missing value handling, making imputation still an essential part of the pre-processing step of ML pipelines. In general, hyper-parameter tuning, model selection, and feature selection are given less importance in previous literature. Most works skip one or more of the previous steps or fail to mention information about the specific stage. For example, only one predictive model is tuned or imputation methods are used with default parameters specified by the authors of the methods or the package implementations.

The research closer to ours is Reference [49]. In the aforementioned paper, Autosklearn was extended to include the data cleaning process, the emerging tool named AutoClean. Part of the extension was imputation. The study compares mean, median, mode, KNN [71], and Iterative imputation [77] for continuous features. For categorical features, constant, KNNi, and mode imputation were selected. The study used five binary classification datasets with 891 to 10,500 observations and 9 to 39 features that include missing values at low percentages. AutoClean optimized the pipeline by Bayesian hyper-parameter optimization. In autosklearn the predictive model is an ensemble of

methods. They evaluated the performance by using a fivefold cross-validation and balanced accuracy metric. The study concluded that KNNi is a valuable addition to the simpler imputation methods. However, in most cases, simple imputation methods are selected more frequently than KNNi for both continuous and categorical data. Contrary to the aforementioned literature, we included feature selection in our experimental setup. Also, we conducted comparisons on both datasets with native and simulated missing values. In general, our evaluation was conducted on more datasets, with a higher range in terms of samples and features. Finally, we included neural network imputation methods and extended imputation with binary indicators.

In Reference [19], TPOT AutoML tool was extended with imputation methods, specifically mean, median, mode, max, MICE [77], and EM [27]. The median and mode were found to be the best imputation methods based on a restricted simulation study on 23 datasets at 7% MCAR missingness. The data were split multiple times (20) to account for randomness. At each split, 25% of the data were used as a hold-out set. Compared to the mentioned work, we simulated missing values with other mechanisms and missing proportions as well as used datasets with native missing values in our experiments. Also, we included recent state-of-the-art methods based on NNs such as DAE and GAIN. We also implemented and measured the effect of binary indicators when coupled with MM and complex methods.

Similarly, missing data imputation has been also researched as part of the data cleaning systems. Reference [38] compared deletion, mean, median, mode, new-category, and HoloClean [59] on six datasets with native missing values. For the predictive task 7 models were considered: Logistic Regression, KNN, Decision Tree, Random Forest, AdaBoost, Naive Bayes, and XGBoost. For the evaluation, the data were split 70% train–30% test set, repeated 20 times to account for randomness. They used accuracy and F1-score for evaluation according to the dataset's imbalance. They concluded that simple imputation methods yield competitive performance to more complex methods such as HoloClean. Contrary to the aforementioned work, we included neural networks, the binary indicator method, and feature selection in the experimental setup. We conducted experiments on more datasets that, generated missing values but also had real-world missing values.

The benchmark study [81] was conducted on 13 real-world datasets from OpenML and concluded that mean/mode is comparable to more complex imputation methods such as random, SOFT [44], MF [66], KNNi [71], Hot-Deck [33], and MICE [77]. In the study, 20% of the data was kept as a hold-out and reported measures were AUC and F1-score. Specifically, while measuring the F1-score, MM had the highest average ranking. In contrast, for the AUC score, KNNi is found to be the best-performing method. However, both KNNi and MM are among the three best methods in both metrics. Hyper-parameter tuning was not considered in this article, the imputation and predictive methods used default parameters. In our work, we tune all the steps of the pipeline for a fair comparison. We also included deep learning methods that are the current state-of-the-art for imputation as well as binary indicators.

The largest benchmark study was conducted in Reference [30] on 69 real-world datasets with simulated missing values. The missing mechanisms were MCAR, MAR, and MNAR. The generation of missing values was set at 1%, 10%, 30%, and 50% missingness. They compared MM, KNNi [71], MF [66], custom DL-based imputation inspired by Reference [9], GAIN [83] and variational autoencoders [32] for the imputation problem. Cross-validation scores are reported, data split into 5 folds for all but deep learning methods. For deep learning methods, three folds were used for the split due to training costs. For regression data, RMSE is the reported metric. For classification data, the F1-score was reported. They concluded that MissForest is the best imputation method. However, they used a single classifier for the prediction phase. We argue that different imputation methods work better with different classifiers, which should be tuned as well. For example, on the cylinder-band dataset GAIN imputation method performs best with the Ridge Logistic Regression,

whereas the DAE imputation method performs best with the RandomForest classifier on the same dataset. Additionally, missing values for the downstream task were generated for a randomly sampled feature in the dataset. We also uniformly simulated missing values, which is a harder problem to solve for the imputation methods as less observed data exist. In the mentioned work, GAIN had a convergence problem in 33% of the cases resulting in the worst ranking among the mentioned methods. In our work, GAIN does indeed converge due to different hyperparameter tuning. Finally, we include Binary Indicators as well as the DAE imputation method, which is the best method on average in real-world data with missing values. Simulated missingness results in our work are on par with the results of the aforementioned work as MissForest is the best method in both works. However, deep learning methods in our work are among the best methods and not the worst as in the literature mentioned.

Another study [57], compared the predictive performance on two datasets with imputed and incomplete data. Missing values were simulated on categorical features on the train data. They generated MCAR and MNAR missing values from 10% to 40% missingness in categorical features. One-third of the data were kept as hold-out test set. The accuracy score on the test set is reported. For the imputation of categorical features, they used six imputation models: mode, random, k-NN [71], iterative imputation based on logistic regression, random forest [66], and SVM. For the classification, they used three predictive models, ANNs, decision trees, and random forests. The authors optimized the hyper-parameters for the ANNs only. The imputation models and the other classifiers were not tuned. They did not conclude that an imputation method or a classifier is better than others and heavily depends on the nature and proportion of the missing data. However, results indicated that imputation is better than simply creating a new category in the data. In our work, for fair evaluation, we tune both imputation and predictive models. We include binary indicator methods and neural network imputation. Finally, we simulated missingness on both numeric and categorical features.

In Reference [55], the authors compared mean, median, KNNi [71], Iterative Imputer, Iterative Imputer /w Bagging (Multiple Imputation) [77], MIA (the native handling of missing values by Gradient Boosted Trees), and MIA /w bag. All of the previous methods, except MIA, were also extended with binary indicators. The study was conducted on 13 real-world datasets from four databases with native missing values. Nested cross-validation with five outer folds is used for estimating the accuracy score of the downstream task. The predictive models were set to Gradient-boosted trees and linear models. They concluded that MIA is a better alternative to imputation. Also, the indicator method helps improve the performance of the predictive task, which is on par with the results of our work. They conclude that simple imputation using mean or median is on par with KNNi and iterative imputation with linear models. In our work, we included deep learning imputation in our set of imputation methods. Also, we tuned the imputation methods to fairly evaluate the performance of each method, as tuning is important in the performance of some imputation methods.

Last, Reference [8] introduced a group of three methods named OptImpute, focusing on optimizing KNNi and iterative imputation based on SVMs and decision trees. They compared the group against five other imputation methods: mean/mode, K-nearest neighbors [71], iterative known [84], Bayesian PCA [50], and predictive-mean matching [77]. They compared the introduced method across 84 datasets with simulated missing values measuring imputation accuracy. They additionally measured the group's performance on learning algorithm performance on 10 datasets. The missing values are generated by the MCAR mechanism with a range from 10% to 50%. The classifiers used for regression tasks are LASSO and SVR while for the classification tasks SVM and Optimal Trees. These datasets range in size, having 150 to 5,875 observations and 4 to 16 features. Data were split 50%–50% into train and test sets. The splits were repeated 100 times to account for

randomness. Their group of methods improved the predictive performance of the models. Their method scored 86.1% average accuracy and average **R-Squared (R2)** of 0.339 compared to 84.4% and 0.315 R2 for the classification and regression data, respectively. However, no neural networks were used and the methods introduced have not been compared individually. Additionally, tuning was applied only to the group of proposed methods and not to the other imputation methods that were used and the predictive models. We tune all of the imputation methods and models. We also extend methods with binary indicators and include NNs imputation methods in our test bed, as well as MissForest. We also report multiple metrics (Accuracy, F1, and AUC) for the binary classification task. Finally, we have a wider range of datasets, both with native and simulated missing values.

8.1 Synopsis of Contributions Relative to the Related Work

Compared to the related work, we contribute in various ways. Our work can be directly compared to two other works that are conducted in an AutoML tool [19, 49]. Compared to the mentioned works, we include more datasets, more missingness mechanisms, neural network methods, and binary indicators in the experimental setup. For the first time, deep learning methods are compared to simple imputation methods in an AutoML predictive setting. One of the deep learning methods (BI+DAE) has the best average performance on real-world data with native missing values. Additionally, for the first time, the effect of the imputation methods on predictive performance is measured on datasets with generated missing values and native missing values. Until now, comparisons were conducted on only one of the two settings, specifically half of the papers use real-world datasets with missing values in them, while the other half use complete datasets with generated missing values. Contrary to the majority of the literature, we tune both imputation and predictive methods to fairly evaluate them. Only two of the eight related mention tuning both imputation and predictive modeling methods [38, 49]. We also conducted experiments on more datasets compared to the majority of the literature, while unlike [30], our simulation setting is applied to all features in the datasets and not only one. Also, only one of the eight aforementioned works includes feature selection as part of the ML pipeline. Finally, meta-learning, for the first time, is used to identify useful data characteristics that could give insights into the choice of a simple vs. a sophisticated imputation method. In general, as shown in Table 5, our testbed is the most complete overall in terms of dataset selection, missingness selection, imputation method selection, and pipeline steps. This allows us to fairly evaluate imputation methods in a state-of-the-art AutoML environment.

9 LESSONS LEARNED AND CONTRIBUTIONS

The main insights that are drawn from our experimental results are the following:

- *Including BI in the dataset improves the predictive performance of the machine learning pipeline for most algorithms* (see Section 5.1). The inclusion of BIs does increase the dimensionality—and difficulty—of the machine learning task. However, it does encode the information about which missing values are missing; this allows a classifier to learn which values to trust or not. Results indicate that encoding this information turns out to be more beneficial than harmful.
- *BI+DAE is found to be the single best imputation method in real-world data with native missing values followed by BI+MM, which is the standard in AutoML tools.* As seen in Section 5.2, both methods have the same number of wins (when comparing only BI extended methods) across datasets with BI+DAE having higher mean AUC. The worst performance is exhibited by matrix-factorization (linear dimensionality reduction) methods such as PPCA. These methods do scale with the number of features and may be more suitable for high-dimensional, low-sample datasets.

- *BI+MM exhibits the best tradeoff between efficiency and effectiveness.* As expected (see Sections 5.3 and 6.3 and Appendix C.2.4), BI+MM is the fastest method to train and also is more effective in the majority of the comparisons. MF, due to its iterative nature, is the slowest among all closely followed by GAIN. GAIN's main bottleneck is the number of epochs required to train the network. The authors' suggestion was 10,000 epochs, which is 20 times more than the 500 epochs suggested by the authors of the DAE method.
- Based on the results of Section 5.4, we would suggest practitioners to optimize their models over the BI+MM and BI+DAE algorithms. *BI+MM and BI+DAE score over 99% of the maximum AUC in real-world data as shown in Section 5.4.* Specifically, BI+MM scores 98.68% of the maximum AUC. Adding BI+DAE to the pipeline leads to 99.69% of the maximum AUC. However, this comes at the cost of increasing the configuration space by 10 $\times$, as DAE has nine tuning configurations compared to one of BI+MM. Also, to reach 100% of the optimal performance, we have to train 24-times more configurations than by simply using BI+MM.
- *BI+MF is the best method in datasets with simulated missing values.* As shown in Section 6.1, in both MCAR and MAR simulations, BI+MF is on average the best. In contrast, BI+MF is the third best with real-world data, falling behind BI+DAE. Despite our best efforts to realistically simulate missing values, there may still be differences between real-world missing-data generative mechanisms and our simulations. First, we simulated MCAR and MAR missing values. Real-world missing values may be NMAR. Second, the missingness probability for MAR data is determined by a generalized linear model (logistic regression model). Real-world missing values may follow non-linear models. The majority of the literature employs similar simulations for comparing imputation algorithms. However, as indicated by this study, results with simulated missingness may not generalize to real-world datasets. New simulation methodologies need to be proposed to this end.
- *Missingness increase leads to a deterioration in predictive performance.* As shown in Section 6.1, increasing missingness causes a drop in the AutoML tool's capability of predicting the outcome. Missingness at 10% leads to a 0.024 AUC drop compared to the complete dataset. Similarly, 25% missingness leads to 0.05 AUC drop, while at 50% we can inspect up to 0.144 drop average as seen in Tables 12(a) and (b).
- *The set containing BI+MM and BI+MF reaches 99% of maximum AUC for simulated data as shown in Section 6.2.* BI+MM scores the 98.7% of the maximum AUC for MCAR data and 98.99% for MAR data. To surpass 99% of the maximum AUC, the addition of BI+MF is needed. This addition allows the tool to reach 99.62% and 99.43% on MCAR and MAR data, respectively. However, BI+MF has to be tuned, leading to a total 3 $\times$ increase in pipeline complexity.
- A meta-learning methodology to correlate meta-features with performance is presented in Section 7. It could allow scientists to select the appropriate sophisticated methods based on meta-features, saving training time and improving overall performance. In addition, it could provide insight into the design choice of an algorithm that leads to better or worse performance on a given dataset. Unfortunately, no statistically significant results were found. This means that either there are no correlations present with the selected meta-features, or these correlations are not strong enough to be found significant with the given sample size of 25 datasets.

There are, of course, several *limitations* of the study that we would like to point to. The results and conclusions stem from computational experiments with binary classification tasks within a range of a number of features, sample size, imbalance of the classes, and missingness percentage. MNAR missingness pattern is not included in our experiments. Also, the mechanism for generating MAR data is based on a linear model. Results may differ for non-linear missingness generation and

MNAR data. Despite the significant computational effort involved—optimizing over thousands of ML pipelines for each dataset—results stem from only 25 real-world datasets with native missing values and 60 complete datasets where missing values were introduced (10 original datasets times 2 missingness mechanisms (MCAR, MAR) times 3 missingness percentages). This fact limits the statistical power of our statistical tests. While JADBio is an effective AutoML tool, results should also be obtained from other AutoML tools to further generalize the conclusions. Another limitation of our work, concerns the comparison of methods on only binary classification data. Even though imputation algorithms are unsupervised learning methods and do not use information from the target variable (in our work), results may vary according to the supervised task. Finally, we selected models based on the AUC score in the training set. Optimizing for another metric, such as accuracy or F1-score, during training may yield different results.

10 CONCLUSIONS

In this article, we conducted experiments on real-world datasets with native missing values and simulated missing values. We compared six imputation methods extended by binary indicators on a state-of-the-art AutoML tool. BI+DAE is the best method on real-world datasets with native missing values. However, BI+MM is comparable to, if not better than, the more sophisticated imputation methods in terms of predictive performance and efficiency on real-world data. Increasing missingness leads to predictive performance deterioration. Additionally, simulation data lead to contradicting results compared to real-world datasets. BI+DAE and BI+MM are the best methods on real-world data; however, when simulated data are considered BI+MF is the best method on average followed by BI+MM. Finally, meta-learning was employed but could not successfully find any patterns to predict whether a sophisticated imputation method can be used instead of the simple BI+MM to improve the downstream performance.

The results make us question whether advanced, multivariate imputation algorithms are really necessary for predictive modeling with AutoML. The simple BI+MM imputation is surprisingly effective and computationally efficient when the ML pipeline is properly tuned within an AutoML setting. BI features allow advanced classifiers to learn when to trust a value or not. Multivariate Imputation algorithms try to learn the full joint distribution of the dataset, a task that is quite challenging with low sample, imbalanced, or high-dimensional data and prone to error. It is also a very computationally demanding task. Imputing values for features that are redundant or irrelevant to the final model is a waste of computations. When imputing using multivariate imputation, one needs to store not only the final model (e.g., RF, SVM, or a NN) but also the imputation model to impute test samples. For some imputation models (Deep Neural Networks, or one RF for each feature as in MF) the additional storage may be non-negligible. In addition, the imputation model requires measuring all features and invalidates the efforts of feature selection. Arguably, the research effort that goes into novel and better-performing imputation methods would be more productive to be spent on novel and better-performing ways to natively handle missing values in our classification and feature selection algorithms.

APPENDICES

A DATASETS APPENDIX

A.1 Real-World Datasets with Native Missing Values

This section presents the 25 real-world binary classification datasets with native missing values. See Table 6 for more details.

Table 6. Binary Classification Real-World Datasets Used in the Comparative Evaluation

Dataset	ID	Samples	Features	#Numerical	#Categorical	Missing %	Imbalance ratio	#Feat with miss>1%	%Missing/Feature
analcadata_reviewer	1,008	379	7	0	7	51.56	0.43	7	51.56
audiology	999	226	69	0	69	2.03	0.25	6	23.23
anneal	989	898	38	16	22	64.98	0.24	29	85.15
autoHorse	840	205	25	17	8	1.11	0.40	4	6.46
braziltourism	957	412	8	7	1	2.91	0.23	2	10.68
bridges	328	107	11	4	7	6.03	0.41	7	9.35
cjs	1,024	2,796	34	32	2	71.64	0.24	28	86.97
colic	27	368	22	7	15	23.80	0.37	19	27.50
colleges_aaup	897	1,161	15	13	2	1.47	0.30	6	3.68
cylinder-bands	6,332	540	39	24	15	4.74	0.42	23	7.93
dresses-sales	23,381	500	12	1	11	13.92	0.42	5	33.04
eucalyptus	990	736	19	14	5	3.21	0.29	6	9.95
hepatitis	55	155	19	6	13	5.67	0.21	11	9.56
hungarian	231	294	13	12	1	20.46	0.36	5	52.93
kdd_el_nino-small	839	782	8	8	0	7.45	0.35	4	14.90
mushroom	24	8,124	22	0	22	1.39	0.48	1	30.53
pbcsq	802	1,945	17	13	4	3.43	0.50	6	9.71
primary-tumor	1,003	339	17	0	17	3.90	0.25	2	32.74
profb	470	672	9	5	4	19.84	0.33	2	89.29
schizo	466	340	14	12	2	17.52	0.48	11	22.30
sick	38	3,772	29	7	22	5.54	0.06	7	22.96
soybean	1,023	683	35	0	35	9.78	0.13	32	10.68
stress	42,167	199	12	8	4	8.29	0.20	7	14.22
vote	56	435	16	0	16	5.63	0.39	16	5.63
water-treatment	940	527	36	36	0	2.86	0.15	22	4.53

The table below contains the dataset name, id, number of samples, number of features, number of categorical, and numeric features, Missingness percentage in the whole dataset, Minority Class with missing values over lfinally the outcome type of each dataset.

A.2 Complete Datasets for Missing Data Simulation

This section presents the 10 complete binary classification datasets used for the simulated missing value experiments. Table 7 contains the dataset names and their characteristics.

Table 7. Binary Complete Datasets in Which We Inject Missing Values

Dataset	ID	#Samples	#Features	#Numerical	#Categorical	Minority Class %
Australian	40,981	690	14	8	6	0.44
boston	853	506	13	12	1	0.41
churn	40,701	5,000	20	16	4	0.14
compas-two-years	42,193	5,278	13	7	6	0.47
image	40,592	2,000	135	135	0	0.21
page-blocks	1,021	5,473	10	10	0	0.1
parkinsons	1,488	195	22	22	0	0.25
segment	958	2,310	19	19	0	0.14
stock	841	950	9	9	0	0.49
zoo	965	101	16	1	15	0.41

The table reports the dataset name, ID, the number of samples, number of features, the imbalance ratio, and the number of numerical and categorical variables.

B MISSING VALUE SIMULATION SETUP APPENDIX

B.1 Datasets Selected to Determine the Percentage of Missing Values per Feature.

Realistic simulation of missing values requires selecting the missingness percentage for each feature; see Section 4.1. We sampled 64 real-world datasets with missing values from OpenML repository. Table 8 describes the datasets’ characteristics.

Table 8. Datasets Used for Missing Value Simulation Experimental Setup

Dataset	ID	Samples	Features	#Numerical	#Categorical	Missing %	Minority Class %	#Feat miss>1%	%Missing/Feature	Type
adult	179	48,842	14	6	8	0.95	0.24	3	4.41	Binary
albert	41,147	425,240	78	78	0	13.64	0.50	43	24.73	Binary
analcdata_reviewer	1,008	379	7	0	7	51.56	0.43	7	51.56	Binary
anneal	989	898	38	16	22	64.98	0.24	29	85.15	Binary
aps_failure	41,138	76,000	170	170	0	8.35	0.02	160	8.83	Binary
ASP-POTASSCO-classification	41,705	1,294	142	139	3	9.94	0.02	138	10.23	MultiClass
ASP-POTASSCO-regression	41,704	14,234	142	138	4	9.94	0.00	138	10.23	Regression
audiology	999	226	69	0	69	2.03	0.25	6	23.23	Binary
autoHorse	840	205	25	17	8	1.11	0.40	4	6.46	Binary
braziltourism	957	412	8	7	1	2.91	0.23	2	10.68	Binary
bridges	328	107	11	4	7	6.03	0.41	7	9.35	Binary
Census-Income-KDD	42,750	199,523	41	13	28	5.08	0.06	7	29.72	Binary
cjs	1,024	2,796	34	32	2	71.64	0.24	28	86.97	Binary
Code_Smells_Data_Class	43,079	86,467	66	66	0	49.99	0.00	62	53.20	Regression
colic	27	368	22	7	15	23.80	0.37	19	27.50	Binary
colleges	42,727	7,063	47	31	16	31.42	0.00	30	49.19	Regression
colleges_aaup	897	1,161	15	13	2	1.47	0.30	6	3.68	Binary
colleges_usnews	930	1,302	33	32	1	18.22	0.47	25	23.96	Binary
cylinder-bands	6,332	540	39	24	15	4.74	0.42	23	7.93	Binary
Domainome	41,533	1,623	9838	9838	0	82.17	0.35	9688	83.44	Binary
dresses-sales	23,381	500	12	1	11	13.92	0.42	5	33.04	Binary
echoMonths	222	130	9	7	2	8.29	0.00	6	12.31	Regression
eucalyptus	990	736	19	14	5	3.21	0.29	6	9.95	Binary
fishcatch	232	158	7	7	0	7.87	0.00	1	55.06	Regression
fps-in-video-games	42,737	425,833	44	33	11	6.94	0.00	12	25.44	Regression
hepatitis	55	155	19	6	13	5.67	0.21	11	9.56	Binary
house_prices_nominal	42,563	1,460	79	36	43	6.04	0.00	16	29.74	Regression
hungarian	231	294	13	12	1	20.46	0.36	5	52.93	Binary
ipums_la_97-small	993	7,019	60	34	26	11.42	0.04	18	38.06	MultiClass
ipums_la_98-small	381	7,485	60	34	26	11.59	0.01	17	40.91	MultiClass
ipums_la_99-small	378	8,844	60	34	26	9.71	0.02	18	32.36	MultiClass
jungle_chess_2pcs_endgame_rat_panther	41,002	5,880	46	18	28	1.30	0.23	6	10.00	MultiClass
KDD98	42,343	82,318	477	358	119	11.30	0.12	87	61.98	Binary
KDDCup09-Upselling	1,112	50,000	15000	13,391	1609	3.35	0.07	608	82.59	Binary
KDDCup09_churn	42,759	50,000	230	192	38	69.78	0.07	205	78.28	Binary
kdd_coil_1	567	316	11	8	3	1.61	0.00	3	4.85	Regression
kdd_el_nino-small	839	782	8	8	0	7.45	0.35	4	14.90	Binary
kick	41,162	72,983	32	17	15	6.39	0.12	5	40.51	Binary
lymphoma_2classes	1,101	45	4026	4,026	0	3.28	0.49	2116	6.25	Binary
meta	566	528	21	18	3	4.55	0.00	3	31.82	Regression
MiceProtein	40,966	1,080	81	77	4	1.60	0.10	8	14.69	MultiClass
Midwest_Survey_nominal	42,532	2,778	27	1	26	1.95	0.03	5	10.51	MultiClass
mlr_ranger_rng	42,458	278,863	14	8	6	3.56	0.00	1	49.69	Regression
mlr_svm_rng	42,456	540,576	13	7	6	9.38	0.00	2	60.95	Regression
Moneyball	41,021	1,232	14	11	3	20.87	0.00	4	73.05	Regression
mushroom	24	8,124	22	0	22	1.39	0.48	1	30.53	Binary
NewFuelCar	41,506	36,203	17	17	0	1.46	0.00	1	24.78	Regression
okcupid-stem	42,734	50,789	19	3	16	15.97	0.10	12	25.28	MultiClass
pbcc	524	418	18	17	1	16.47	0.00	12	24.66	Regression
pbccseq	802	1,945	17	13	4	3.43	0.50	6	9.71	Binary
porto-seguro	42,206	595,212	37	25	12	3.84	0.04	5	28.21	Binary
primary-tumor	1,003	339	17	0	17	3.90	0.25	2	32.74	Binary
profb	470	672	9	5	4	19.84	0.33	2	89.29	Binary
rl	41,160	31,406	22	22	0	10.45	0.10	8	28.71	Binary
road-safety	42,803	363,243	66	61	5	9.10	0.05	41	14.62	MultiClass
SAT11-HAND-runtime-regression	41,980	4,440	116	113	3	5.27	0.00	10	61.15	Regression
schizo	466	340	14	12	2	17.52	0.48	11	22.30	Binary
sick	38	3,772	29	7	22	5.54	0.06	7	22.96	Binary
soybean	1,023	683	35	0	35	9.78	0.13	32	10.68	Binary
speeddating	40,536	8,378	122	61	61	2.87	0.00	109	3.17	Binary
stress	42,167	199	12	8	4	8.29	0.20	7	14.22	Binary
us_crime	315	1,994	127	126	1	15.48	0.00	24	81.91	Regression
vote	56	435	16	0	16	5.63	0.39	16	5.63	Binary
water-treatment	940	527	36	36	0	2.86	0.15	22	4.53	Binary

The table contains the dataset name, id, number of samples, number of features, number of categorical and numeric features, Missingness percentage in the whole dataset, Minority Class %, the number of features with missing values over 1%, the missingness percentage over features with missing values, and, finally, the outcome type of each dataset.

B.2 Determining the Average Number of Features on Which a Missing Feature Depends.

In this section, we present the quantitative results for the experiments regarding the simulation of MAR mechanism presented in Section 4. Table 9 presents the dataset name, the randomly selected feature with missing values (target) and the result of the feature selection (# features selected). On average a missing feature is dependent on 12 features.

Table 9. Summary of the Feature Selection Experiments for MAR Simulation

Dataset	Missing Feature (target)	#Selected Features
aps_failure	cn_006	25
colleges_aaup	Average_salary-full_professors	5
colleges_usnews	Out-of-state_tuition	18
dresses-sales	V3	7
eucalyptus	PMCno	6
hepatitis	ALBUMIN	6
hungarian	thalach	3
mushroom	stalk-root	16
pbseq	presence_of_asictes	7
speeddating	attractive	24

On average, a missing feature depends on 12 other features.

C EXPERIMENTAL RESULTS APPENDIX

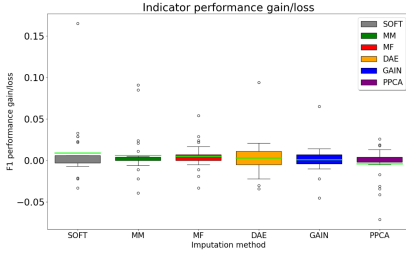
C.1 Real-world Results

C.1.1 BI Improve Performance across All Metrics. BI extended methods perform better than their base methods when the AUC score is measured (see Section 5.1). As seen in Figure 11(b) and (d), BI indeed improves the accuracy and F1-score of the downstream task, in the majority of the datasets. Figure 11(a) and (c) illustrate the gain or loss of including BI for each imputation method in the x-axis. BI improves the performance of the downstream task, on average.

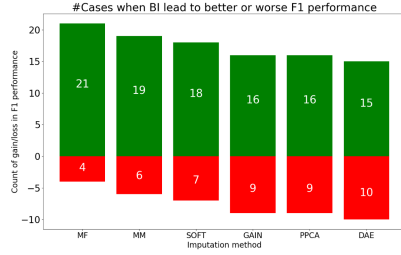
C.1.2 BI+DAE Is the Best Method Followed by BI+MM. Table 10(a), (b), and (c) show the quantitative results of the real-world experiments. Specifically, it depicts the number of wins (ties included) for each imputation method, as well as the average difference in AUC from the winning imputation method for each dataset. The table also reports the average AUC, average AUC difference from MM (set as baseline), and average ranking per method. For completeness, we include methods without BI as well. BI+DAE and BI+MM are the two best methods across all metrics, in terms of average AUC and average ranking. BI+MM gets the highest number of wins in all metrics. While BI+DAE closely follows with one win for each AUC and two wins for F1 and accuracy. However, BI+DAE is more consistent and has the highest average ranking and highest score for AUC and F1. Finally, BI+DAE exhibits the highest improvement over MM and the lowest difference from the best method in each dataset, on average.

C.1.3 BI+DAE Is the Best Method across All BI Extended Methods. As seen in Section 5.2, BI+DAE is the highest-ranked method for the AUC metric. Figure 12(a) and (b) illustrate the average ranking of BI extended methods for F1 and accuracy metrics, respectively. BI+DAE is the highest-ranked method for accuracy metric but is the third best for F1. BI+MM is the second-best in accuracy and the best in F1. Overall, across different metrics, the relative order varies slightly. Statistically significant results remain the same for AUC and accuracy. No statistically significant results are found for the F1-score.

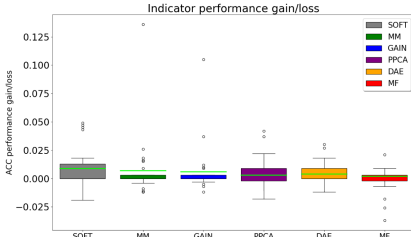
C.1.4 BI+MM and BI+DAE Score 99% of the Maximum across All Metrics. As shown in Tables 11(a), (b), (c), and Figure 13, BI+MM scores over 98% of the maximum performance for each metric. Adding BI+DAE, which is the best next method, to the imputation set that already contains BI+MM allows the tool to score over 99.5% of the maximum performance. However, the complexity increases by 10×. Finally, to reach 100% of the maximum, all imputation methods need to be



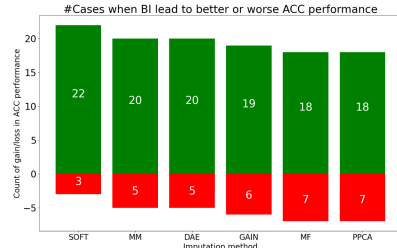
(a) F1 performance gain for each imputation method when BI are included. The horizontal line with lime color, denotes the mean F1 gain/loss of including binary indicators for the specified imputation method.



(b) For each method (x-axis), the green bars denote the count of datasets (y-axis) for which including BI in the base method improves f1 score. The red bars denote the count of datasets for which the f1 deteriorates by including BI variables.

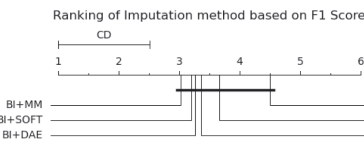


(c) Accuracy performance gain for each imputation method when BI are included. The horizontal line with lime color, denotes the mean accuracy gain/loss of including binary indicators for the specified imputation method.

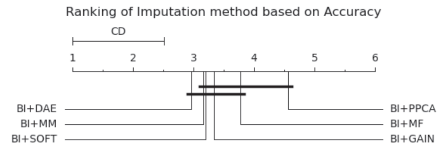


(d) For each method (x-axis), the green bars denote the count of datasets (y-axis) for which including BI in the base method improves accuracy score. The red bars denote the count of datasets for which the accuracy deteriorates by including BI variables.

Fig. 11. Panels (a) and (c) illustrate improvement of BI extended imputation methods over the base method. Panels (b) and (d) show the count of datasets where BI extended methods are scoring higher/lower than base methods.



(a) Average ranking for f1-score. BI+MM has the highest ranking.



(b) Average ranking for accuracy score. BI+DAE has the highest ranking.

Fig. 12. Panels (a) and (b) show the ranking of imputation methods for F1-score and accuracy score.

included in the imputation set. Including all methods in the pipeline of the tool, increases the original complexity by a factor of 24.

C.1.5 BI+MM Exhibits the Best Tradeoff between Effectiveness and Efficiency. This section presents the tradeoff between the effectiveness and efficiency of imputation methods against a baseline (BI+MM). For a detailed explanation of the illustration see Section 5.3. Figure 14(a) shows that BI+MM dominates the other imputation methods in both effectiveness and efficiency in 84 of 125 pairs for F1-score. In 41 pairs, BI+MM is dominated in effectiveness. For the accuracy metric,

Table 10. Number of Wins, Average Ranking, Average Metric Score, Average Difference from Best per Dataset and Average Difference from a Baseline Method (MM)

(a) Summarized Results for AUC

Method	Wins	Avg AUC	Avg Rank	Avg Diff	Avg Diff MM
BI+MM	8	0.883	5.12	0.015	0.007
BI+DAE	7	0.889	4.9	0.009	0.013
MM	4	0.876	6.16	0.022	—
BI+MF	4	0.88	6.14	0.018	0.004
DAE	3	0.889	5.4	0.009	0.013
GAIN	3	0.876	6.24	0.022	0.0
MF	3	0.875	7.0	0.023	-0.001
BI+PPCA	3	0.86	8.12	0.038	-0.016
BI+GAIN	3	0.881	6.02	0.017	0.006
PPCA	2	0.856	8.92	0.042	-0.02
BI+SOFT	2	0.881	6.52	0.017	0.005
SOFT	1	0.879	7.46	0.02	0.003

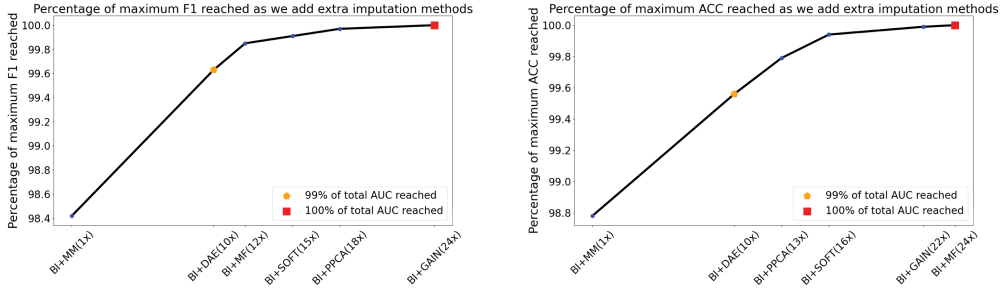
(b) Summarized Results for F1-score

Method	Wins	Avg F1	Avg Rank	Avg Diff	Avg Diff MM
MM	7	0.838	6.34	0.022	—
BI+MM	7	0.844	5.64	0.016	0.006
GAIN	6	0.842	6.14	0.017	0.004
DAE	5	0.844	5.76	0.016	0.006
BI+DAE	5	0.847	5.72	0.013	0.009
BI+GAIN	5	0.843	6.56	0.016	0.006
BI+MF	4	0.844	5.8	0.016	0.006
SOFT	3	0.835	6.54	0.025	-0.003
MF	3	0.839	7.36	0.021	0.001
BI+SOFT	3	0.843	5.86	0.016	0.006
PPCA	2	0.82	7.94	0.039	-0.017
BI+PPCA	2	0.817	8.34	0.043	-0.021

(c) Summarized Results for Accuracy

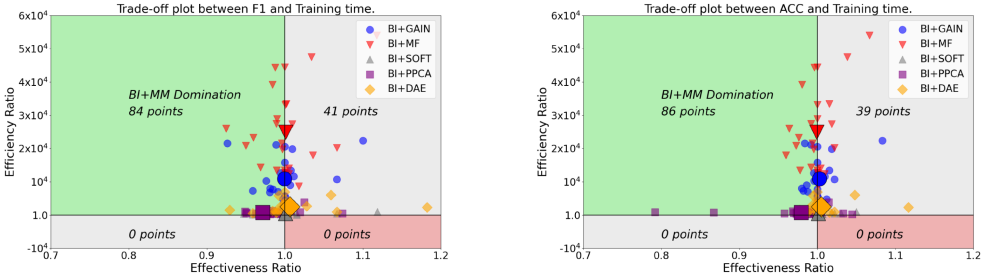
Method	Wins	Avg ACC	Avg Rank	Avg Diff	Avg Diff MM
BI+MM	10	0.872	5.42	0.012	0.007
MM	9	0.865	6.22	0.019	—
BI+DAE	8	0.875	5.1	0.009	0.01
BI+GAIN	6	0.874	5.78	0.01	0.009
DAE	5	0.871	6.26	0.013	0.006
GAIN	4	0.868	5.98	0.016	0.003
BI+MF	4	0.87	6.54	0.014	0.005
BI+PPCA	4	0.855	8.2	0.029	-0.01
BI+SOFT	4	0.872	5.62	0.012	0.007
PPCA	3	0.851	8.48	0.033	-0.014
MF	3	0.869	7.06	0.015	0.004
SOFT	1	0.864	7.34	0.02	-0.001

BI+MM, MM and BI+DAE are on average the best methods on various metrics.



(a) The percentage of maximum f1-score achieved by each subset of imputation methods. (b) The percentage of maximum accuracy score achieved by each subset of imputation methods.

Fig. 13. Panels (a) and (b) illustrate the maximum performance achieved by each imputation subset. For both F1 and accuracy, BI+MM and BI+DAE set scores over 99% of the maximum performance.



(a) The Figure illustrates the effectiveness-efficiency trade-off for f1-score. BI+MM dominates the other algorithms in 84 pairs. (b) The Figure illustrates the effectiveness efficiency trade-off for accuracy. BI+MM dominates the other algorithms in 86 pairs.

Fig. 14. Panels (a) and (b) denote the tradeoff in terms of relative effectiveness and relative efficiency between BI+MM and the other imputation methods for F1 and accuracy metrics, respectively.

Table 11. Table Denotes the Methods That Are Added to the Imputation Set, the % of the Maximum Score for the Specified Metric Reached by Each Set, and the Configuration Complexity Increase for Each Set

(a) AUC Subset Results			(b) F1 Subset Results			(c) Accuracy Subset Results		
Added Method	% of Max AUC	Mult. Config.	Added Method	% of Max F1	Mult. Config.	Added Method	% of Max ACC	Mult. Config.
BI+MM	98.68	1	BI+MM	98.42	1	BI+MM	98.78	1
BI+DAE	99.69	10	BI+DAE	99.63	10	BI+DAE	99.56	10
BI+MF	99.93	12	BI+MF	99.85	12	BI+PPCA	99.79	13
BI+PPCA	99.98	15	BI+SOFT	99.91	15	BI+SOFT	99.94	16
BI+GAIN	99.99	21	BI+PPCA	99.97	18	BI+GAIN	99.99	22
BI+SOFT	100.0	24	BI+GAIN	100.0	24	BI+MF	100.0	24

The set containing BI+MM and BI+DAE accounts for over 99% for each of the metrics.

BI+MM dominates the other methods in relative effectiveness and efficiency in 86 of 125 pairs. As seen in Figure 14(b), BI+MM is dominated in only 39 pairs.

C.1.6 Feature Selection–enforced Pipelines Degrade the Performance. As seen in Figure 15(a) and (b), feature selection deteriorates the performance of the pipelines. The average absolute difference between feature selection–enforced pipelines and non-enforced is less than 5% for accuracy. On average, the F1-score is lower by 5 points when enforcing feature selection.

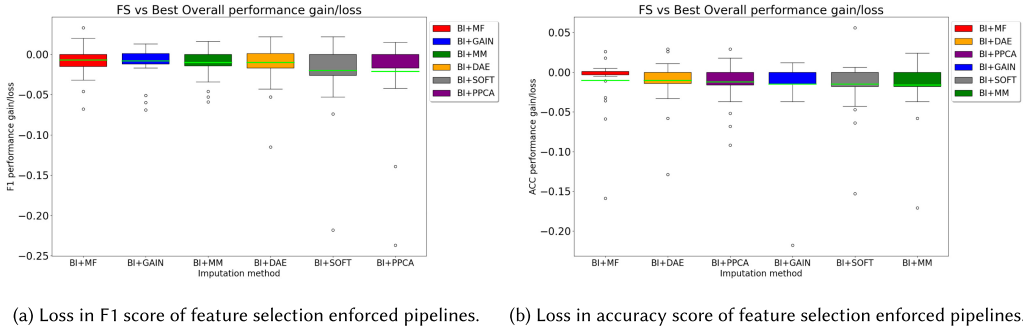


Fig. 15. Panels (a) and (b) denote the loss in predictive performance when enforcing feature selection in the pipeline for the real-world data. For all imputation methods, enforcing feature selection leads to a drop in predictive performance by less than 5% accuracy (on average).

C.2 Simulation Results

C.2.1 A Decline in Predictive Performance Is Caused by Increasing Missingness. In this section, we investigate the average performance drop in terms of multiple metrics compared to the complete dataset. Specifically, Table 12(a), (c), and (e) denote the results for the MCAR missingness and AUC, F1, and accuracy score, respectively. Table 12(b), (d), and (f) present the results for MAR missing data for AUC, F1, and accuracy score, respectively. Summarizing the results, across all metrics and both missingness mechanisms, as missingness increases, the performance of the tool deteriorates. For MCAR data, average AUC drops in absolute terms up to 0.023 at 10% missingness and 0.05 and 0.144 for 25% and 50%, respectively. While measuring F1, the loss is even bigger. At 10%, loss is up to 0.05, and at 25% loss can reach up to 0.078 average absolute difference, while average F1 loss at 50% missingness can be up to 0.158. The accuracy score deteriorates comparably to AUC. At 10% the loss can be up to 0.03, at 25% up to 0.05, and at 50% up to 0.118. For MAR data, results are similar to MCAR. It is noteworthy to mention that at 50% missingness, the performance does not deteriorate as much as for MCAR. This leads us to conclude that most methods in this comparative evaluation can recover information for high MAR missingness better than MCAR. This is theoretically sound, as every method except MM uses multiple variables for the imputation.

C.2.2 BI+MF Is the Best Method for MCAR Data. Table 12(a), (c), and (e) and Figure 16(b), (d), and (f) present the results for MCAR missing data at various missingness percentages and multiple metrics (AUC, F1, and Accuracy). Summarizing the results, across all missingness percentages and measured metrics, BI+MF is the best method in terms of average absolute loss to the complete data. The second best method, at 10%, is BI+DAE, while at 25% BI+MM is the second best for F1 and accuracy metrics. The relative order of imputation methods across metrics remains stable until 50% missingness. The order at 50% missingness may vary according to the performance metric. The two worst methods are BI+PPCA and BI+SOFT, while the positions of second-, third-, and fourth-best methods are shared by BI+MM, BI+GAIN, and BI+DAE.

C.2.3 BI+MF Is the Best Method for MAR Data. As seen in Section 6.1, BI+MF is the best method for MAR data. Table 12(b), (d), and (f) provide an overview of the results for AUC, F1, and accuracy score. The results are robust across all metrics. Figure 17(b) and (d) show that BI+MF has the lowest average loss for 25% and 50% missingness in both F1 and accuracy metric. At 10% missingness, BI+MF has comparable performance to BI+MM, which has the lowest loss at that missingness rate. A detailed review of the results for the AUC metric follows.

Table 12. The Average Loss from the Complete Datasets by Each Imputation Method when Missing Data Are MCAR (Left) and MAR (Right)

(a) Average AUC Loss for MCAR Data.				(b) Average AUC Loss for MAR Data.			
Method	10%	25%	50%	Method	10%	25%	50%
BI+MF	-0.006	-0.02	-0.064	BI+MM	-0.014	-0.03	-0.054
BI+DAE	-0.011	-0.023	-0.084	BI+SOFT	-0.014	-0.048	-0.103
BI+MM	-0.012	-0.024	-0.079	BI+MF	-0.015	-0.029	-0.052
BI+GAIN	-0.014	-0.026	-0.075	BI+GAIN	-0.016	-0.032	-0.06
BI+SOFT	-0.019	-0.044	-0.144	BI+DAE	-0.016	-0.038	-0.057
BI+PPCA	-0.023	-0.05	-0.119	BI+PPCA	-0.024	-0.049	-0.073

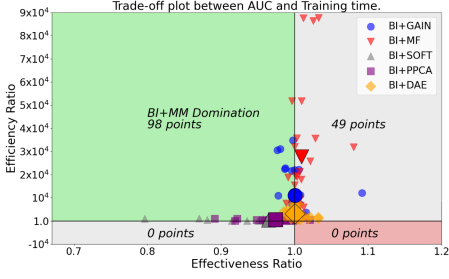
(c) Average F1 Loss for MCAR Data.				(d) Average F1 Loss for MAR Data.			
Method	10%	25%	50%	Method	10%	25%	50%
BI+MF	-0.007	-0.024	-0.083	BI+MM	-0.017	-0.045	-0.086
BI+DAE	-0.017	-0.042	-0.1	BI+MF	-0.019	-0.034	-0.071
BI+MM	-0.021	-0.037	-0.097	BI+GAIN	-0.021	-0.043	-0.086
BI+GAIN	-0.027	-0.041	-0.099	BI+DAE	-0.025	-0.059	-0.088
BI+SOFT	-0.034	-0.07	-0.15	BI+SOFT	-0.026	-0.068	-0.132
BI+PPCA	-0.052	-0.078	-0.158	BI+PPCA	-0.057	-0.085	-0.129

(e) Average ACC Loss for MCAR Data.				(f) Average ACC Loss for MAR Data.			
Method	10%	25%	50%	Method	10%	25%	50%
BI+MF	-0.003	-0.014	-0.058	BI+MM	-0.011	-0.028	-0.055
BI+DAE	-0.011	-0.029	-0.074	BI+MF	-0.012	-0.021	-0.047
BI+MM	-0.016	-0.026	-0.073	BI+GAIN	-0.013	-0.029	-0.06
BI+GAIN	-0.025	-0.028	-0.077	BI+DAE	-0.018	-0.043	-0.059
BI+SOFT	-0.025	-0.049	-0.118	BI+SOFT	-0.018	-0.05	-0.085
BI+PPCA	-0.03	-0.051	-0.106	BI+PPCA	-0.031	-0.047	-0.079

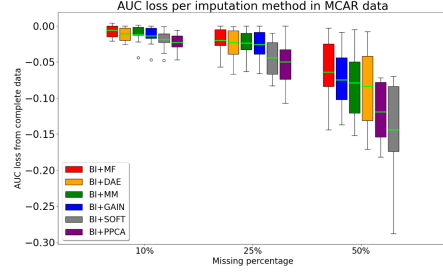
(a) and (b) present the results for the AUC metric; (c) and (d) present the results for F1, while (e) and (f) the classification accuracy results. BI+MF is better for MCAR data and MAR data (except for 10% missingness).

C.2.4 BI+MM Exhibits the Best Efficiency vs. Effectiveness Tradeoff for MCAR Missing data. Regarding the MCAR missing data, it is worth noting that BI+MM dominates in over 90 of 147 pairs, for each performance metric. Specifically, for AUC metric BI+MM dominates in 98 pairs, for F1-score in 90 pairs, and for accuracy in 95 pairs. BI+MM is never dominated in efficiency, as seen in Figure 16(a), (c), and (e). This is not surprising, as BI+MM is significantly faster to train than any other imputation method. BI+MM is only dominated by BI+MF across all metrics in average effectiveness. BI+PPCA and BI+SOFT are on average less effective and less efficient than BI+MM.

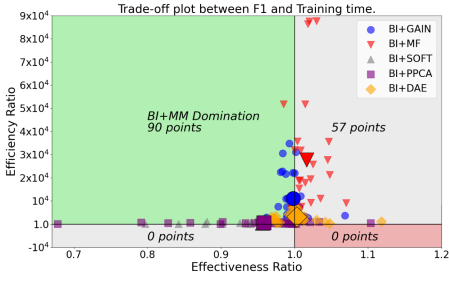
C.2.5 BI+MM Exhibits the Best Efficiency vs. Effectiveness Tradeoff for MAR Missing Data. In section 6.3, we presented the efficiency–effectiveness tradeoff for the MAR simulated data when AUC is reported. We extend and confirm our conclusion in this section by measuring and computing the tradeoff for F1-score and classification accuracy metrics. As seen in Figure 17(a) and (d), BI+MM is never dominated in terms of efficiency, as expected. BI+MM dominates the other methods in effectiveness and efficiency in 94 and 98 pairs for the F1-score and accuracy score, respectively. For F1, it is dominated in terms of relative effectiveness in 53 of 157 pairs, while it is



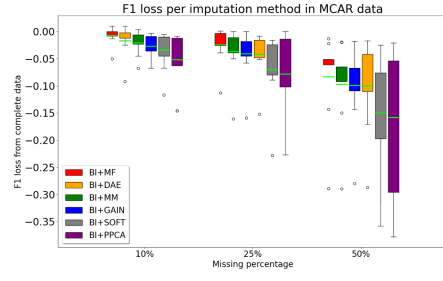
(a) The Figure illustrates the effectiveness-efficiency trade-off for the AUC. BI+MM dominates the other algorithms in 98 pairs.



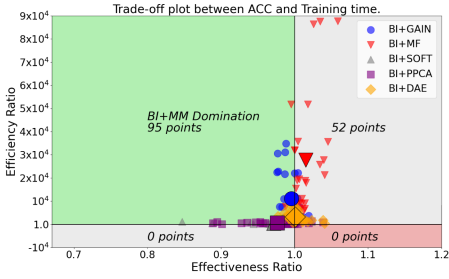
(b) AUC difference of each imputation method from the complete dataset.



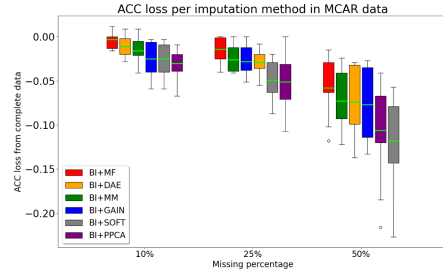
(c) The Figure illustrates the effectiveness-efficiency trade-off for f1-score. BI+MM dominates the other algorithms in 90 pairs.



(d) F1 difference of each imputation method from the complete dataset.



(e) The Figure illustrates the effectiveness-efficiency trade-off for accuracy score. BI+MM dominates the other algorithms in 95 pairs.

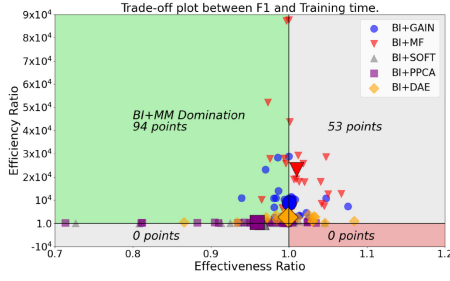


(f) Accuracy difference of each imputation method from the complete dataset.

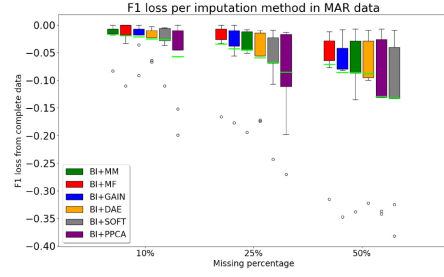
Fig. 16. Panels (a), (c), and (e) denote the tradeoff in terms of effectiveness and efficiency between BI+MM and the other imputation methods. Panels (b), (d), and (f) show the difference from complete data for each imputation method at various missingness levels. BI+MF is the best method for MCAR data. However, BI+MM exhibits good performance at a fraction of the cost.

dominated for accuracy in 49 pairs. BI+MF is on average more effective than BI+MM for MAR data. However, it is thousands of times less efficient (up to 90,000 for big datasets).

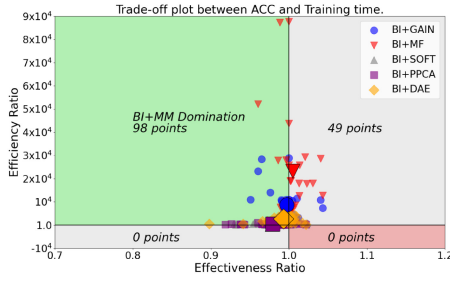
C.2.6 BI+MM and BI+MF Score over 99% of Maximum Performance for MCAR Data. The minimal-size subset of algorithms with close-to-optimal performance for MCAR missing data is *BI + MM*, *BI + MF*. We used the simple greedy algorithm introduced in Section 5.4. As illustrated



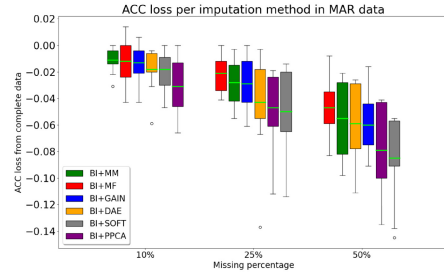
(a) The Figure illustrates the effectiveness-efficiency trade-off for the f1-score. BI+MM dominates the other algorithms in 94 pairs.



(b) F1 difference of each imputation method from the complete dataset.



(c) The Figure illustrates the effectiveness-efficiency trade-off for the accuracy score. BI+MM dominates the other algorithms in 98 pairs.



(d) Accuracy difference of each imputation method from the complete dataset.

Fig. 17. Panels (a) and (c) denote the tradeoff in terms of effectiveness and efficiency between BI+MM and the other imputation methods. Panels (b) and (d) show the difference from complete data for each imputation method at various missingness levels. BI+MF is the best method for MAR data. However, BI+MM exhibits good performance at a fraction of the cost.

in Figure 18(a), this subset achieves over 99% of the maximum achievable AUC for MCAR data. The subset containing $BI + MM$, $BI + MF$, also scores over 99% of the maximum performance score for both F1 and accuracy, as denoted in Figure 18(b) and (c). Detailed quantitative results are presented in Table 13(a), (c), and (e) for the AUC, F1, and accuracy score, respectively.

C.2.7 $BI+MM$ and $BI+MF$ Score over 99% of Maximum Performance for MAR Data. The minimal-size subset of algorithms with close-to-optimal performance for MCAR missing data is $BI + MM$, $BI + MF$ as presented in Section 6.2. In this section, we present results for F1 and accuracy score. We used the simple greedy algorithm introduced in Section 5.4. The subset containing $BI + MM$, $BI + MF$, also scores over 99% of the maximum performance score for both F1 and accuracy, as denoted in Figure 19(a) and (b). Detailed quantitative results are presented in Table 13(b), (d), and (f) for the AUC, F1, and accuracy score, respectively.

C.3 Evaluation of Imputation Accuracy

In this section, we present the results of imputation accuracy experiments. We measure the imputation accuracy for each imputation method, missing mechanism, and missingness percentage, by training the imputation methods on the default configurations (highlighted in Table 2). For each dataset, we measure the average R2-score between the imputed and the complete values (clamped

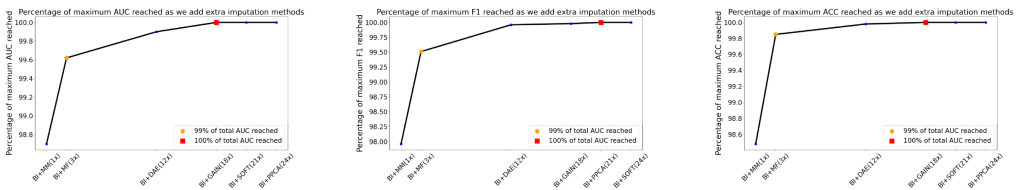
Table 13. The Methods That Are Added Sequentially to the Imputation Set, the % of Maximum Score for the Specified Metric Reached by Each Set, and the Configuration Complexity Increase for Each Set

(a) MCAR - AUC			(b) MAR - AUC		
Added Method	% of Max AUC	Mult. Config.	Added Method	% of Max AUC	Mult. Config.
BI+MM	98.7	1	BI+MM	98.99	1
BI+MF	99.62	3	BI+MF	99.43	3
BI+DAE	99.9	12	BI+DAE	99.67	12
BI+GAIN	100	18	BI+GAIN	99.84	18
BI+SOFT	100	21	BI+SOFT	99.95	21
BI+PPCA	100	24	BI+PPCA	100	24

(c) MCAR - F1			(d) MAR - F1		
Added Method	% of Max F1	Mult. Config.	Added Method	% of Max F1	Mult. Config.
BI+MM	97.96	1	BI+MM	98.19	1
BI+MF	99.51	3	BI+MF	99.43	3
BI+DAE	99.96	12	BI+DAE	99.88	12
BI+GAIN	99.98	18	BI+SOFT	99.94	15
BI+PPCA	100.0	21	BI+GAIN	99.97	21
BI+SOFT	100.0	24	BI+PPCA	100.0	24

(e) MCAR - Accuracy			(f) MAR - Accuracy		
Added Method	% of Max ACC	Mult. Config.	Added Method	% of Max ACC	Mult. Config.
BI+MM	98.48	1	BI+MM	98.95	1
BI+MF	99.85	3	BI+MF	99.72	3
BI+DAE	99.98	12	BI+DAE	99.88	12
BI+GAIN	100.0	18	BI+SOFT	99.96	15
BI+SOFT	100.0	21	BI+PPCA	100.0	18
BI+PPCA	100.0	24	BI+GAIN	100.0	24

Tables on the left concern MCAR data, while the tables on the right are for MAR data. For both MAR and MCAR, BI+MM and BI+MF reach over 99% of the maximum AUC across all metrics.



(a) The percentage of maximum AUC achieved by each subset of imputation methods.

(b) The percentage of maximum F1 achieved by each subset of imputation methods.

(c) The percentage of maximum accuracy achieved by each subset of imputation methods.

Fig. 18. MCAR: The percentage of maximum metric (AUC, ACC, F1) achieved by each set of imputation methods versus the increased complexity of the pipelines. On the x-axis, we denote the added method as well as the additional complexity to the configuration space. The addition of BI+MF to the set increases the complexity by $3\times$ compared to the original complexity. BI+MF and BI+MM allows us to recover over 99% of the maximum performance for MCAR data.

between the 0–1 range) for the continuous features. For the categorical features, we measure the accuracy score between the imputed and the complete values. We measure the imputation scores in both train and test sets.

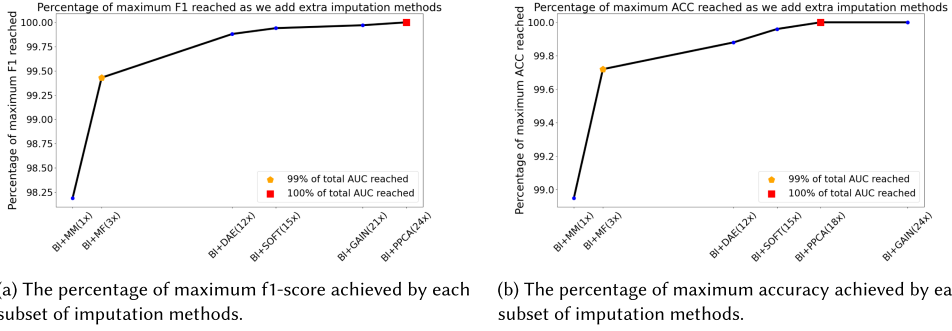


Fig. 19. MAR: The percentage of maximum metric (ACC, F1) achieved by each set of imputation methods versus the increased complexity of the pipelines. On the x-axis, we denote the added method as well as the additional complexity to the configuration space. The addition of BI+MF to the set increases the complexity by $3\times$ compared to the original complexity. BI+MF and BI+MM allow us to recover over 99% of the maximum metric for MAR data.

In summary, MF has the highest on-average imputation accuracy for categorical and continuous features. For both MCAR and MAR data, MF is the best method. Additionally, we observe that the differences between MF and the other methods are more prominent in MAR data. The result remains relatively the same across the train and test sets. MM is one of the worst performing in terms of imputation accuracy. However, MM is performing similarly to MF when measuring the downstream task performance, as seen in Sections 5 and 6. This observation further enhances our original hypothesis that imputation accuracy does not necessarily lead to better downstream task performance.

C.3.1 MF Has the Highest Imputation Accuracy for MAR Data. For MAR data, MF has the highest average R2 and accuracy score in the train data, as seen in Figure 20(a) and (b). Figure 20(c) and (d) show that MF imputes values more accurately on test MAR data. In general, MF is the most accurate method for all missingness percentages. MM, as expected, has the lowest R2-score as it does not predict any of the variance of the continuous data. One interesting observation is that SOFT imputation fails to generalize on test data. As missingness increases, the imputation methods make worse predictions leading to lower scores.

C.3.2 MF Has the Highest Imputation Accuracy for MCAR Data. For MCAR data, MF is the most accurate imputation method in the train data, as seen in Figure 21(a) and (b). Figure 21(c) and (d) show that MF has the highest average R2 and accuracy score on test MCAR data, across all missingness percentages. MM, as expected, has the lowest R2-score. SOFT fails to generalize on new unseen data. Finally, as missingness increases, the quality of imputed values deteriorates.

C.4 Real World: Downstream Task Results

In this section, we include the quantitative results of the real-world experiments. Table 14 contains AUC results. Table 15 the results of F1-score, and Table 16 contains the results for the accuracy metric.

C.5 MCAR: Downstream Task Results

This section presents the results for MCAR data under varying levels of missingness, and multiple metrics. Tables 17, 18, and 19 present the results for 10%, 25%, and 50% missingness for the AUC

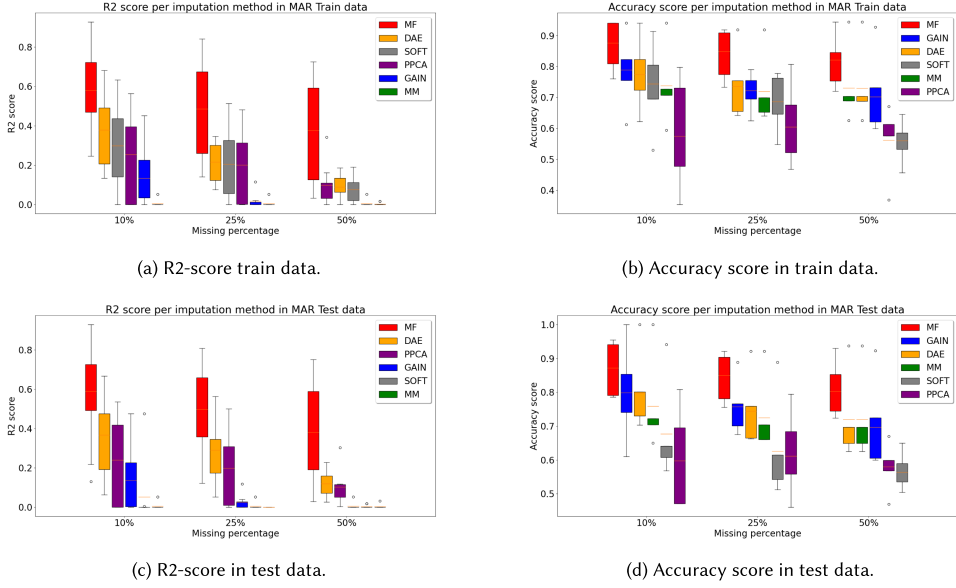


Fig. 20. Panel (a) depicts the imputation R2-score for the continuous variables in MAR train data. Panel (b) illustrates the imputation accuracy in MAR train data, for categorical variables. Panel (c) shows the R2-score for continuous variables in MAR test data and panel (d) the accuracy score for categorical data in MAR test data.

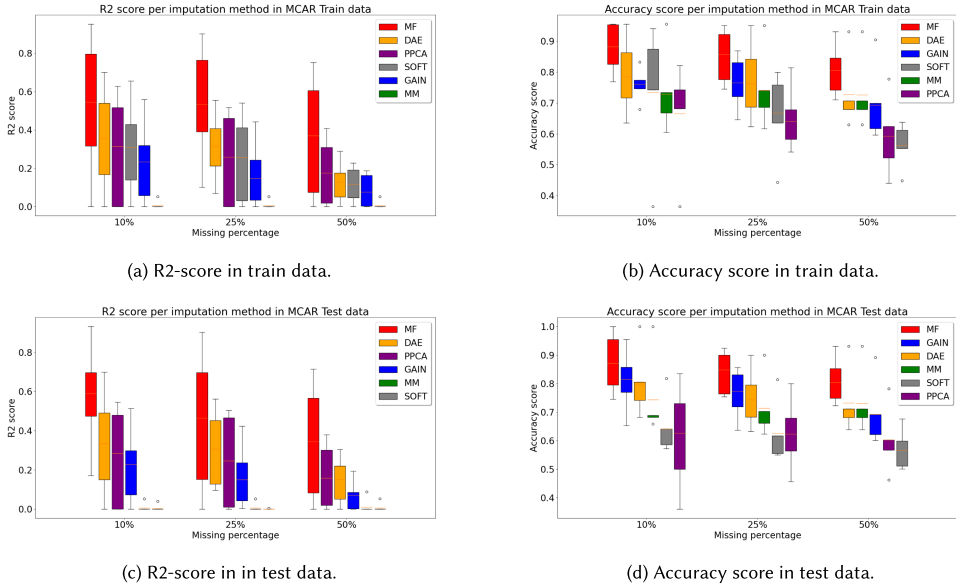


Fig. 21. Panel (a) depicts the imputation R2-score for the continuous variables in MCAR train data. Panel (b) illustrates the imputation accuracy in MCAR train data for categorical variables. Panel (c) shows the R2-score for continuous variables in MCAR test data and panel (d) shows the accuracy score for categorical data in MCAR test data.

Table 14. Real-world Results for the AUC Metric

Dataset	MM	BI-MM	MF	BI-MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
analcattdata_reviewer-FS	0.585	0.585	0.5	0.597	0.561	0.595	0.602	0.602	0.558	0.558	0.585	0.585
analcattdata_reviewer-NOFS	0.661	0.668	0.599	0.646	0.602	0.61	0.597	0.659	0.606	0.656	0.661	0.668
analcattdata_reviewer-Overall	0.661	0.668	0.605	0.643	0.607	0.63	0.597	0.659	0.606	0.656	0.661	0.668
anneal-FS	0.883	0.988	0.761	0.97	0.916	0.973	0.987	0.967	0.995	0.986	0.975	0.968
anneal-NOFS	0.881	0.996	0.938	0.97	0.931	0.983	0.972	0.991	0.996	0.996	0.982	0.982
anneal-Overall	0.883	0.996	0.943	0.969	0.896	0.991	0.972	0.991	0.996	0.996	0.975	0.982
audiology-FS	0.986	0.986	0.98	0.98	0.98	0.974	0.98	0.98	0.98	0.98	0.98	0.98
audiology-NOFS	0.998	0.998	0.992	0.993	0.994	0.991	0.992	0.998	0.995	0.989	0.993	0.995
audiology-Overall	0.998	0.998	0.98	0.981	0.993	0.992	0.992	0.98	0.995	0.989	0.993	0.98
autoHorse-FS	0.967	0.967	0.966	0.966	0.966	0.966	0.966	0.966	0.901	0.996	0.966	0.966
autoHorse-NOFS	0.99	0.989	0.983	0.988	0.982	0.988	0.981	0.99	0.989	0.993	0.976	0.976
autoHorse-Overall	0.99	0.989	0.966	0.966	0.987	0.988	0.981	0.99	0.989	0.993	0.966	0.966
braziltourism-FS	0.634	0.634	0.64	0.64	0.643	0.634	0.64	0.64	0.725	0.725	0.643	0.643
braziltourism-NOFS	0.616	0.727	0.716	0.725	0.721	0.709	0.709	0.725	0.669	0.668	0.731	0.715
braziltourism-Overall	0.616	0.727	0.643	0.64	0.632	0.64	0.709	0.64	0.669	0.668	0.731	0.715
bridges-FS	0.844	0.844	0.853	0.857	0.891	0.849	0.891	0.891	0.892	0.885	0.88	0.847
bridges-NOFS	0.909	0.911	0.902	0.901	0.882	0.916	0.902	0.889	0.901	0.916	0.915	0.912
bridges-Overall	0.909	0.911	0.902	0.909	0.905	0.909	0.902	0.889	0.901	0.916	0.915	0.912
cjs-FS	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.987	0.997	1.0	1.0
cjs-NOFS	1.0	1.0	0.994	1.0	0.985	0.998	0.996	0.991	0.987	0.99	0.999	0.999
cjs-Overall	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.987	0.997	1.0	1.0
colic-FS	0.829	0.829	0.837	0.855	0.845	0.839	0.836	0.836	0.839	0.839	0.83	0.83
colic-NOFS	0.839	0.838	0.853	0.863	0.845	0.872	0.842	0.865	0.848	0.858	0.852	0.856
colic-Overall	0.829	0.829	0.849	0.881	0.846	0.862	0.836	0.865	0.839	0.858	0.83	0.83
colleges_aaup-FS	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.996	0.996	0.999	0.999
colleges_aaup-NOFS	0.999	0.999	0.997	0.999	0.997	0.997	0.998	0.997	0.998	0.998	0.998	0.997
colleges_aaup-Overall	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.998	0.998	0.999	0.999
cylinder-bands-FS	0.808	0.785	0.81	0.788	0.808	0.797	0.808	0.798	0.723	0.723	0.832	0.826
cylinder-bands-NOFS	0.82	0.819	0.826	0.832	0.826	0.828	0.815	0.821	0.807	0.824	0.848	0.858
cylinder-bands-Overall	0.82	0.819	0.832	0.831	0.813	0.816	0.815	0.821	0.807	0.824	0.848	0.858
dresses-sales-FS	0.562	0.562	0.564	0.561	0.56	0.552	0.565	0.565	0.5	0.549	0.562	0.562
dresses-sales-NOFS	0.619	0.605	0.6	0.597	0.569	0.583	0.597	0.601	0.545	0.539	0.631	0.62
dresses-sales-Overall	0.619	0.562	0.564	0.561	0.567	0.552	0.565	0.565	0.545	0.539	0.631	0.562
eucalyptus-FS	0.833	0.833	0.821	0.82	0.823	0.835	0.842	0.817	0.75	0.816	0.807	0.834
eucalyptus-NOFS	0.777	0.777	0.778	0.778	0.778	0.777	0.779	0.844	0.82	0.824	0.849	0.855
eucalyptus-Overall	0.833	0.833	0.778	0.778	0.832	0.836	0.842	0.817	0.82	0.824	0.849	0.855
hepatitis-FS	0.748	0.748	0.834	0.83	0.803	0.807	0.673	0.673	0.799	0.799	0.826	0.826
hepatitis-NOFS	0.826	0.848	0.866	0.866	0.844	0.85	0.869	0.864	0.876	0.866	0.852	0.869
hepatitis-Overall	0.826	0.848	0.867	0.858	0.858	0.866	0.869	0.864	0.799	0.799	0.852	0.869
hungarian-FS	0.899	0.899	0.883	0.884	0.893	0.867	0.871	0.871	0.897	0.897	0.875	0.875
hungarian-NOFS	0.918	0.915	0.901	0.901	0.917	0.915	0.881	0.895	0.895	0.898	0.914	0.912
hungarian-Overall	0.918	0.915	0.887	0.888	0.913	0.918	0.871	0.871	0.895	0.897	0.914	0.912
kdd_el_nino-small-FS	0.983	0.983	0.98	0.981	0.988	0.987	0.983	0.981	0.95	0.926	0.983	0.986
kdd_el_nino-small-NOFS	0.987	0.989	0.984	0.985	0.988	0.988	0.985	0.988	0.98	0.985	0.986	0.987
kdd_el_nino-small-Overall	0.987	0.989	0.982	0.986	0.988	0.987	0.985	0.988	0.98	0.985	0.986	0.987
mushroom-FS	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
mushroom-NOFS	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
mushroom-Overall	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
pbseq-FS	0.849	0.849	0.851	0.852	0.84	0.849	0.836	0.831	0.849	0.849	0.846	0.843
pbseq-NOFS	0.849	0.85	0.857	0.844	0.856	0.848	0.846	0.845	0.841	0.838	0.848	0.842
pbseq-Overall	0.849	0.85	0.851	0.85	0.851	0.849	0.846	0.845	0.841	0.838	0.848	0.842
primary-tumor-FS	0.875	0.887	0.855	0.875	0.846	0.874	0.829	0.882	0.829	0.864	0.786	0.887
primary-tumor-NOFS	0.88	0.892	0.875	0.875	0.871	0.88	0.877	0.889	0.861	0.87	0.88	0.892
primary-tumor-Overall	0.88	0.892	0.867	0.892	0.886	0.87	0.877	0.889	0.861	0.87	0.88	0.892
prof-FS	0.642	0.642	0.642	0.642	0.642	0.642	0.642	0.642	0.631	0.631	0.642	0.642
prof-NOFS	0.695	0.696	0.691	0.693	0.695	0.692	0.695	0.696	0.579	0.58	0.693	0.695
prof-Overall	0.695	0.696	0.692	0.694	0.692	0.686	0.695	0.696	0.631	0.631	0.693	0.695
schizo-FS	0.526	0.526	0.556	0.557	0.534	0.515	0.556	0.556	0.543	0.543	0.623	0.623
schizo-NOFS	0.719	0.684	0.764	0.765	0.717	0.74	0.76	0.76	0.56	0.559	0.794	0.805
schizo-Overall	0.719	0.684	0.781	0.772	0.736	0.751	0.76	0.76	0.543	0.543	0.794	0.805
sick-FS	0.984	0.984	0.992	0.993	0.993	0.992	0.977	0.969	0.991	0.991	0.992	0.992
sick-NOFS	0.99	0.989	0.994	0.994	0.992	0.995	0.986	0.988	0.992	0.989	0.993	0.992
sick-Overall	0.99	0.989	0.994	0.994	0.992	0.993	0.986	0.988	0.992	0.989	0.993	0.992
soybean-FS	0.981	0.983	0.983	0.992	0.991	0.981	0.985	0.985	0.973	0.973	0.991	0.991
soybean-NOFS	0.991	0.994	0.989	0.986	0.989	0.992	0.987	0.991	0.991	0.985	0.989	0.993
soybean-Overall	0.981	0.994	0.985	0.991	0.99	0.991	0.987	0.991	0.991	0.985	0.989	0.993
stress-FS	0.916	0.916	0.932	0.932	0.932	0.932	0.932	0.932	0.933	0.933	0.932	0.932
stress-NOFS	0.902	0.904	0.899	0.903	0.906	0.904	0.902	0.901	0.948	0.946	0.909	0.909
stress-Overall	0.916	0.916	0.932	0.932	0.932	0.932	0.932	0.932	0.933	0.933	0.909	0.932
vote-FS	0.983	0.985	0.992	0.995	0.986	0.99	0.991	0.991	0.989	0.991	0.978	0.986
vote-NOFS	0.992	0.991	0.994	0.991	0.994	0.99	0.995	0.991	0.995	0.992	0.992	0.992
vote-Overall	0.992	0.991	0.992	0.991	0.995	0.992	0.991	0.991	0.989	0.991	0.992	0.992
water-treatment-FS	0.916	0.988	0.986	0.987	0.958	0.988	0.943	0.943	0.5	0.5	0.962	0.979
water-treatment-NOFS	0.988	0.988	0.988	0.987	0.988	0.988	0.954	0.986	0.788	0.774	0.98	0.981
water-treatment-Overall	0.988	0.988	0.987	0.987	0.988	0.988	0.954	0.986	0.788	0.774	0.98	0.981

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 15. Real-world Results for the F1 Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
analcadata_reviewer-FS	0.603	0.603	0.603	0.603	0.603	0.603	0.608	0.608	0.603	0.603	0.603	0.603
analcadata_reviewer-NOFS	0.635	0.656	0.609	0.653	0.606	0.636	0.624	0.653	0.632	0.645	0.635	0.656
analcadata_reviewer-Overall	0.635	0.656	0.614	0.635	0.612	0.607	0.624	0.653	0.632	0.645	0.635	0.656
anneal-FS	0.905	0.988	0.88	0.99	0.928	0.991	0.971	0.987	0.981	0.976	0.966	0.99
anneal-NOFS	0.903	0.99	0.93	0.987	0.942	0.977	0.961	0.99	0.985	0.993	0.975	0.983
anneal-Overall	0.905	0.99	0.933	0.987	0.924	0.99	0.961	0.99	0.985	0.993	0.966	0.983
audiology-FS	0.931	0.931	0.926	0.926	0.926	0.909	0.926	0.926	0.926	0.926	0.926	0.926
audiology-NOFS	0.966	0.966	0.931	0.945	0.931	0.931	0.933	0.966	0.949	0.918	0.931	0.966
audiology-Overall	0.966	0.966	0.926	0.893	0.926	0.926	0.933	0.926	0.949	0.918	0.931	0.926
autoHorse-FS	0.976	0.976	0.968	0.968	0.968	0.968	0.968	0.968	0.889	0.984	0.968	0.968
autoHorse-NOFS	0.984	0.984	0.976	0.984	0.976	0.984	0.976	0.976	0.976	0.976	0.976	0.976
autoHorse-Overall	0.984	0.984	0.968	0.968	0.976	0.984	0.976	0.976	0.976	0.976	0.968	0.968
braziltourism-FS	0.893	0.893	0.885	0.885	0.893	0.891	0.885	0.885	0.895	0.895	0.893	0.893
braziltourism-NOFS	0.872	0.877	0.883	0.876	0.882	0.875	0.879	0.881	0.881	0.88	0.878	0.876
braziltourism-Overall	0.872	0.877	0.885	0.885	0.885	0.885	0.879	0.885	0.881	0.88	0.878	0.876
bridges-FS	0.778	0.778	0.8	0.8	0.837	0.809	0.829	0.829	0.809	0.816	0.818	0.783
bridges-NOFS	0.8	0.824	0.829	0.8	0.792	0.83	0.83	0.808	0.816	0.821	0.83	0.8
bridges-Overall	0.8	0.824	0.826	0.815	0.83	0.808	0.83	0.808	0.816	0.821	0.83	0.8
cjs-FS	1.0	1.0	1.0	1.0	0.999	0.997	1.0	1.0	0.931	0.949	0.988	1.0
cjs-NOFS	1.0	1.0	0.984	0.981	0.973	0.974	0.974	0.956	0.931	0.943	0.976	0.981
cjs-Overall	1.0	1.0	1.0	1.0	0.999	0.999	1.0	1.0	0.931	0.949	0.988	1.0
colic-FS	0.906	0.906	0.891	0.895	0.886	0.899	0.876	0.876	0.877	0.877	0.896	0.896
colic-NOFS	0.886	0.892	0.877	0.897	0.902	0.89	0.893	0.909	0.878	0.889	0.881	0.873
colic-Overall	0.906	0.906	0.898	0.902	0.902	0.898	0.876	0.909	0.877	0.889	0.896	0.896
colleges_aaup-FS	0.994	0.988	0.993	0.988	0.988	0.988	0.993	0.993	0.985	0.985	0.994	0.993
colleges_aaup-NOFS	0.987	0.987	0.985	0.987	0.985	0.983	0.987	0.985	0.989	0.989	0.989	0.983
colleges_aaup-Overall	0.994	0.988	0.993	0.988	0.988	0.988	0.993	0.993	0.989	0.989	0.994	0.993
cylinder-bands-FS	0.796	0.81	0.8	0.81	0.797	0.813	0.793	0.819	0.81	0.81	0.832	0.827
cylinder-bands-NOFS	0.816	0.805	0.827	0.826	0.814	0.808	0.806	0.802	0.799	0.825	0.844	0.852
cylinder-bands-Overall	0.816	0.805	0.829	0.832	0.81	0.8	0.806	0.802	0.799	0.825	0.844	0.852
dresses-sales-FS	0.592	0.592	0.592	0.592	0.595	0.592	0.592	0.592	0.592	0.592	0.592	0.592
dresses-sales-NOFS	0.597	0.601	0.592	0.598	0.594	0.592	0.605	0.601	0.593	0.596	0.602	0.602
dresses-sales-Overall	0.597	0.592	0.592	0.592	0.592	0.592	0.592	0.592	0.593	0.596	0.602	0.592
eucalyptus-FS	0.672	0.672	0.644	0.644	0.664	0.677	0.68	0.659	0.603	0.634	0.649	0.654
eucalyptus-NOFS	0.638	0.638	0.638	0.638	0.638	0.638	0.64	0.678	0.65	0.652	0.697	0.691
eucalyptus-Overall	0.672	0.672	0.638	0.638	0.664	0.676	0.68	0.659	0.65	0.652	0.697	0.691
hepatitis-FS	0.912	0.912	0.902	0.932	0.932	0.917	0.912	0.912	0.896	0.896	0.932	0.932
hepatitis-NOFS	0.91	0.91	0.916	0.928	0.91	0.904	0.919	0.924	0.932	0.925	0.912	0.913
hepatitis-Overall	0.91	0.91	0.931	0.912	0.913	0.917	0.919	0.924	0.896	0.896	0.912	0.913
hungarian-FS	0.81	0.81	0.79	0.789	0.803	0.752	0.78	0.78	0.783	0.783	0.77	0.77
hungarian-NOFS	0.826	0.824	0.804	0.804	0.814	0.826	0.817	0.807	0.8	0.81	0.81	0.814
hungarian-Overall	0.826	0.824	0.79	0.79	0.806	0.821	0.78	0.78	0.8	0.783	0.81	0.814
kdd_el_nino-small-FS	0.897	0.897	0.908	0.91	0.917	0.913	0.919	0.897	0.83	0.777	0.918	0.932
kdd_el_nino-small-NOFS	0.925	0.928	0.919	0.915	0.924	0.927	0.9	0.923	0.898	0.916	0.922	0.921
kdd_el_nino-small-Overall	0.925	0.928	0.911	0.918	0.919	0.918	0.9	0.923	0.898	0.916	0.922	0.921
mushroom-FS	1.0	1.0	0.999	1.0	1.0	1.0	0.999	0.999	0.996	0.997	1.0	1.0
mushroom-NOFS	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.999	0.998	1.0	1.0
mushroom-Overall	1.0	1.0	0.999	1.0	1.0	1.0	0.999	0.999	0.996	0.998	1.0	1.0
pbccseq-FS	0.776	0.776	0.788	0.796	0.77	0.779	0.763	0.772	0.779	0.779	0.786	0.767
pbccseq-NOFS	0.775	0.779	0.787	0.779	0.785	0.792	0.779	0.78	0.773	0.774	0.785	0.775
pbccseq-Overall	0.775	0.779	0.781	0.783	0.785	0.777	0.779	0.78	0.773	0.774	0.785	0.775
primary-tumor-FS	0.753	0.703	0.688	0.692	0.705	0.711	0.652	0.704	0.652	0.673	0.585	0.703
primary-tumor-NOFS	0.735	0.712	0.717	0.717	0.744	0.703	0.736	0.703	0.736	0.702	0.735	0.712
primary-tumor-Overall	0.735	0.712	0.714	0.738	0.744	0.699	0.736	0.703	0.736	0.702	0.735	0.712
profb-FS	0.548	0.548	0.548	0.548	0.548	0.548	0.548	0.548	0.552	0.552	0.548	0.548
profb-NOFS	0.573	0.573	0.562	0.576	0.562	0.571	0.573	0.573	0.514	0.51	0.572	0.573
profb-Overall	0.573	0.573	0.569	0.576	0.568	0.56	0.573	0.573	0.552	0.552	0.572	0.573
schizo-FS	0.653	0.653	0.653	0.653	0.645	0.65	0.648	0.648	0.658	0.658	0.648	0.648
schizo-NOFS	0.684	0.645	0.735	0.729	0.694	0.701	0.728	0.722	0.653	0.653	0.75	0.763
schizo-Overall	0.684	0.645	0.719	0.722	0.696	0.71	0.728	0.722	0.658	0.658	0.75	0.763
sick-FS	0.837	0.837	0.838	0.836	0.86	0.851	0.841	0.823	0.843	0.843	0.849	0.849
sick-NOFS	0.839	0.839	0.847	0.849	0.842	0.851	0.855	0.851	0.843	0.839	0.845	0.848
sick-Overall	0.839	0.839	0.837	0.854	0.842	0.849	0.855	0.851	0.843	0.839	0.845	0.848
soybean-FS	0.807	0.839	0.889	0.901	0.918	0.833	0.843	0.843	0.833	0.833	0.897	0.905
soybean-NOFS	0.913	0.898	0.884	0.867	0.894	0.882	0.875	0.897	0.911	0.871	0.882	0.899
soybean-Overall	0.807	0.898	0.86	0.889	0.887	0.884	0.875	0.897	0.911	0.871	0.882	0.899
stress-FS	0.792	0.792	0.844	0.844	0.844	0.844	0.844	0.844	0.85	0.85	0.844	0.844
stress-NOFS	0.75	0.75	0.769	0.756	0.756	0.75	0.756	0.75	0.878	0.837	0.75	0.75
stress-Overall	0.792	0.792	0.844	0.844	0.844	0.844	0.844	0.844	0.85	0.85	0.75	0.844
vote-FS	0.948	0.948	0.948	0.948	0.948	0.949	0.948	0.948	0.949	0.948	0.929	0.948
vote-NOFS	0.943	0.954	0.948	0.949	0.949	0.953	0.959	0.955	0.959	0.954	0.948	0.959
vote-Overall	0.943	0.954	0.943	0.955	0.955	0.965	0.948	0.948	0.949	0.948	0.948	0.959
water-treatment-FS	0.8	0.987	0.975	0.975	0.821	0.987	0.732	0.732	0.263	0.263	0.961	0.94
water-treatment-NOFS	0.987	0.987	0.987	0.975	0.987	0.987	0.785	0.95	0.571	0.5	0.951	0.918
water-treatment-Overall	0.987	0.987	0.975	0.975	0.987	0.987	0.785	0.95	0.571	0.5	0.951	0.918

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 16. Real-world Results for the ACC Metric

Dataset	MM	BI-MM	MF	BI-MF	GAIN	BI-GAIN	SOFT	BI-SOFT	PPCA	BI-PPCA	DAE	BI-DAE
analcattdata_reviewer-FS	0.589	0.589	0.568	0.611	0.6	0.6	0.6	0.6	0.568	0.568	0.589	0.589
analcattdata_reviewer-NOFS	0.632	0.647	0.621	0.637	0.626	0.6	0.6	0.647	0.595	0.637	0.632	0.647
analcattdata_reviewer-Overall	0.632	0.647	0.621	0.642	0.6	0.637	0.6	0.647	0.595	0.637	0.632	0.647
anneal-FS	0.849	0.982	0.795	0.984	0.884	0.987	0.955	0.98	0.971	0.964	0.947	0.984
anneal-NOFS	0.844	0.984	0.895	0.98	0.906	0.964	0.94	0.984	0.978	0.989	0.962	0.973
anneal-Overall	0.849	0.984	0.895	0.98	0.88	0.984	0.94	0.984	0.978	0.989	0.947	0.973
audiology-FS	0.965	0.965	0.965	0.965	0.965	0.965	0.965	0.965	0.965	0.965	0.965	0.965
audiology-NOFS	0.982	0.982	0.965	0.973	0.965	0.965	0.965	0.982	0.973	0.956	0.965	0.982
audiology-Overall	0.982	0.982	0.965	0.947	0.965	0.965	0.965	0.965	0.973	0.956	0.965	0.965
autoHorse-FS	0.971	0.971	0.961	0.961	0.961	0.961	0.961	0.961	0.874	0.981	0.961	0.961
autoHorse-NOFS	0.981	0.981	0.971	0.981	0.971	0.981	0.971	0.971	0.971	0.971	0.971	0.971
autoHorse-Overall	0.981	0.981	0.961	0.961	0.971	0.981	0.971	0.971	0.971	0.971	0.961	0.961
braziltourism-FS	0.816	0.816	0.806	0.806	0.816	0.811	0.806	0.806	0.82	0.82	0.816	0.816
braziltourism-NOFS	0.777	0.791	0.801	0.791	0.801	0.791	0.796	0.796	0.796	0.791	0.796	0.786
braziltourism-Overall	0.777	0.791	0.806	0.806	0.806	0.806	0.796	0.806	0.796	0.791	0.796	0.786
bridges-FS	0.796	0.796	0.833	0.815	0.87	0.833	0.87	0.87	0.833	0.833	0.852	0.815
bridges-NOFS	0.833	0.833	0.87	0.833	0.815	0.852	0.833	0.815	0.833	0.87	0.833	0.833
bridges-Overall	0.833	0.833	0.852	0.815	0.833	0.833	0.833	0.815	0.833	0.87	0.833	0.833
cjs-FS	1.0	1.0	1.0	1.0	0.999	0.999	1.0	1.0	0.967	0.976	0.994	1.0
cjs-NOFS	1.0	1.0	0.992	0.991	0.987	0.988	0.987	0.979	0.967	0.973	0.989	0.991
cjs-Overall	1.0	1.0	1.0	1.0	0.999	0.999	1.0	1.0	0.967	0.976	0.994	1.0
colic-FS	0.875	0.875	0.853	0.859	0.848	0.864	0.837	0.837	0.837	0.837	0.864	0.864
colic-NOFS	0.848	0.853	0.837	0.859	0.87	0.853	0.859	0.88	0.837	0.853	0.837	0.837
colic-Overall	0.875	0.875	0.864	0.87	0.87	0.864	0.837	0.88	0.837	0.853	0.864	0.864
colleges_aaup-FS	0.991	0.983	0.99	0.983	0.983	0.983	0.99	0.99	0.979	0.979	0.991	0.99
colleges_aaup-NOFS	0.981	0.981	0.979	0.981	0.979	0.976	0.981	0.979	0.985	0.985	0.985	0.976
colleges_aaup-Overall	0.991	0.983	0.99	0.983	0.983	0.983	0.99	0.99	0.985	0.985	0.991	0.99
cylinder-bands-FS	0.752	0.741	0.748	0.741	0.756	0.741	0.756	0.756	0.73	0.73	0.785	0.778
cylinder-bands-NOFS	0.781	0.77	0.793	0.796	0.781	0.774	0.767	0.763	0.759	0.781	0.807	0.807
cylinder-bands-Overall	0.781	0.77	0.804	0.8	0.774	0.767	0.767	0.763	0.759	0.781	0.807	0.807
dresses-sales-FS	0.632	0.632	0.632	0.632	0.632	0.632	0.632	0.632	0.58	0.596	0.632	0.632
dresses-sales-NOFS	0.644	0.648	0.632	0.624	0.628	0.608	0.644	0.648	0.624	0.612	0.644	0.648
dresses-sales-Overall	0.644	0.632	0.632	0.632	0.632	0.632	0.632	0.632	0.624	0.612	0.644	0.632
eucalyptus-FS	0.791	0.791	0.774	0.774	0.772	0.783	0.783	0.769	0.712	0.761	0.75	0.774
eucalyptus-NOFS	0.769	0.769	0.769	0.769	0.769	0.769	0.758	0.785	0.791	0.791	0.785	0.788
eucalyptus-Overall	0.791	0.791	0.769	0.769	0.777	0.78	0.783	0.769	0.791	0.791	0.785	0.788
hepatitis-FS	0.846	0.846	0.833	0.885	0.885	0.859	0.846	0.846	0.821	0.821	0.885	0.885
hepatitis-NOFS	0.846	0.846	0.859	0.885	0.846	0.833	0.859	0.872	0.885	0.872	0.846	0.859
hepatitis-Overall	0.846	0.846	0.885	0.859	0.859	0.859	0.859	0.872	0.821	0.821	0.846	0.859
hungarian-FS	0.844	0.844	0.83	0.837	0.837	0.823	0.85	0.85	0.844	0.844	0.83	0.83
hungarian-NOFS	0.857	0.857	0.857	0.857	0.857	0.864	0.857	0.864	0.844	0.844	0.85	0.857
hungarian-Overall	0.857	0.857	0.837	0.837	0.857	0.857	0.85	0.85	0.844	0.844	0.85	0.857
kdd_el_nino-small-FS	0.928	0.928	0.934	0.939	0.944	0.936	0.944	0.931	0.88	0.849	0.944	0.954
kdd_el_nino-small-NOFS	0.946	0.949	0.944	0.939	0.946	0.949	0.931	0.946	0.926	0.941	0.944	0.944
kdd_el_nino-small-Overall	0.946	0.949	0.936	0.941	0.944	0.941	0.931	0.946	0.926	0.941	0.944	0.944
mushroom-FS	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.996	0.998	1.0	1.0
mushroom-NOFS	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.998	1.0	1.0
mushroom-Overall	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.996	0.998	1.0	1.0
pbseq-FS	0.773	0.773	0.779	0.783	0.764	0.78	0.758	0.757	0.782	0.782	0.772	0.769
pbseq-NOFS	0.776	0.772	0.777	0.776	0.782	0.777	0.776	0.776	0.766	0.764	0.782	0.775
pbseq-Overall	0.776	0.772	0.781	0.779	0.781	0.781	0.776	0.776	0.766	0.764	0.782	0.775
primary-tumor-FS	0.865	0.871	0.835	0.841	0.853	0.871	0.841	0.876	0.841	0.859	0.812	0.871
primary-tumor-NOFS	0.859	0.876	0.847	0.853	0.871	0.841	0.865	0.871	0.865	0.853	0.859	0.876
primary-tumor-Overall	0.859	0.876	0.847	0.841	0.871	0.859	0.865	0.871	0.865	0.853	0.859	0.876
profb-FS	0.673	0.67	0.67	0.673	0.673	0.673	0.673	0.673	0.676	0.676	0.673	0.673
profb-NOFS	0.705	0.705	0.708	0.702	0.711	0.711	0.705	0.705	0.667	0.67	0.705	0.705
profb-Overall	0.705	0.705	0.705	0.708	0.705	0.702	0.705	0.705	0.676	0.676	0.705	0.705
schizo-FS	0.535	0.535	0.594	0.594	0.576	0.547	0.588	0.588	0.553	0.559	0.659	0.659
schizo-NOFS	0.718	0.706	0.741	0.776	0.735	0.759	0.724	0.741	0.606	0.594	0.776	0.788
schizo-Overall	0.718	0.706	0.753	0.753	0.753	0.765	0.724	0.741	0.553	0.559	0.776	0.788
sick-FS	0.979	0.979	0.981	0.981	0.983	0.981	0.979	0.977	0.98	0.98	0.981	0.981
sick-NOFS	0.979	0.979	0.982	0.981	0.98	0.98	0.981	0.981	0.98	0.98	0.98	0.981
sick-Overall	0.979	0.979	0.98	0.98	0.98	0.98	0.981	0.981	0.98	0.98	0.98	0.981
soybean-FS	0.944	0.956	0.971	0.974	0.98	0.956	0.962	0.962	0.959	0.959	0.974	0.974
soybean-NOFS	0.977	0.971	0.968	0.965	0.971	0.968	0.965	0.974	0.977	0.968	0.968	0.974
soybean-Overall	0.944	0.971	0.962	0.971	0.968	0.971	0.965	0.974	0.977	0.968	0.968	0.974
stress-FS	0.91	0.91	0.93	0.93	0.93	0.93	0.93	0.93	0.94	0.94	0.93	0.93
stress-NOFS	0.9	0.9	0.91	0.9	0.9	0.9	0.89	0.9	0.95	0.93	0.9	0.9
stress-Overall	0.91	0.91	0.93	0.93	0.93	0.93	0.93	0.93	0.94	0.94	0.9	0.93
vote-FS	0.959	0.959	0.959	0.959	0.959	0.959	0.959	0.959	0.959	0.959	0.945	0.959
vote-NOFS	0.954	0.963	0.959	0.959	0.959	0.963	0.968	0.963	0.968	0.963	0.959	0.968
vote-Overall	0.954	0.963	0.954	0.963	0.963	0.972	0.959	0.959	0.959	0.959	0.959	0.968
water-treatment-FS	0.943	0.996	0.992	0.992	0.947	0.996	0.92	0.92	0.848	0.848	0.989	0.981
water-treatment-NOFS	0.996	0.996	0.996	0.992	0.996	0.996	0.936	0.985	0.879	0.864	0.985	0.973
water-treatment-Overall	0.996	0.996	0.992	0.992	0.996	0.996	0.936	0.985	0.879	0.864	0.985	0.973

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 17. MCAR Results at 10% Missingness for the AUC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.902	0.905	0.922	0.914	0.913	0.919	0.885	0.915	0.89	0.897	0.906	0.906
Australian-NOFS	0.916	0.911	0.917	0.915	0.911	0.904	0.886	0.896	0.892	0.896	0.915	0.91
Australian-Overall	0.902	0.911	0.922	0.914	0.909	0.908	0.885	0.896	0.892	0.896	0.906	0.906
boston-FS	0.933	0.933	0.926	0.926	0.924	0.924	0.912	0.917	0.89	0.89	0.926	0.926
boston-NOFS	0.931	0.928	0.94	0.935	0.927	0.931	0.933	0.908	0.913	0.921	0.928	0.926
boston-Overall	0.933	0.933	0.938	0.941	0.922	0.921	0.912	0.908	0.913	0.921	0.926	0.926
churn-FS	0.914	0.914	0.92	0.916	0.907	0.916	0.887	0.883	0.884	0.885	0.906	0.906
churn-NOFS	0.909	0.912	0.914	0.916	0.91	0.916	0.904	0.91	0.886	0.886	0.906	0.909
churn-Overall	0.909	0.912	0.917	0.916	0.908	0.919	0.904	0.91	0.884	0.886	0.906	0.909
compas-two-years-FS	0.704	0.704	0.698	0.692	0.699	0.705	0.693	0.693	0.688	0.688	0.697	0.697
compas-two-years-NOFS	0.702	0.698	0.703	0.701	0.701	0.698	0.702	0.702	0.703	0.699	0.692	0.692
compas-two-years-Overall	0.702	0.704	0.703	0.696	0.7	0.699	0.693	0.702	0.688	0.688	0.692	0.697
image-FS	0.87	0.845	—	—	0.883	0.875	0.85	0.85	0.845	0.849	0.877	0.878
image-NOFS	0.885	0.884	—	—	0.881	0.878	0.877	0.877	0.863	0.859	0.892	0.887
image-Overall	0.885	0.884	—	—	0.882	0.883	0.877	0.877	0.863	0.859	0.892	0.887
page-blocks-FS	0.99	0.989	0.99	0.989	0.988	0.99	0.983	0.985	0.969	0.969	0.988	0.988
page-blocks-NOFS	0.988	0.988	0.99	0.99	0.989	0.988	0.983	0.982	0.973	0.974	0.987	0.987
page-blocks-Overall	0.988	0.989	0.99	0.99	0.987	0.989	0.983	0.982	0.973	0.974	0.988	0.987
parkinsons-FS	0.85	0.846	0.871	0.871	0.849	0.849	0.85	0.845	0.848	0.861	0.868	0.866
parkinsons-NOFS	0.896	0.896	0.935	0.923	0.896	0.895	0.898	0.892	0.899	0.907	0.919	0.914
parkinsons-Overall	0.896	0.896	0.932	0.925	0.907	0.894	0.898	0.892	0.899	0.907	0.919	0.914
segment-FS	0.999	0.999	1.0	1.0	0.999	0.999	0.996	0.997	0.978	0.977	0.999	0.999
segment-NOFS	0.999	1.0	1.0	1.0	0.999	1.0	0.998	0.999	0.985	0.986	1.0	0.999
segment-Overall	0.999	1.0	1.0	1.0	1.0	1.0	0.998	0.999	0.985	0.986	1.0	0.999
stock-FS	0.99	0.99	0.993	0.993	0.984	0.987	0.975	0.975	0.954	0.954	0.99	0.99
stock-NOFS	0.989	0.99	0.992	0.993	0.99	0.989	0.979	0.98	0.969	0.97	0.991	0.991
stock-Overall	0.99	0.99	0.993	0.993	0.986	0.986	0.979	0.98	0.969	0.97	0.99	0.99
zoo-FS	0.993	0.993	0.895	0.895	0.929	0.929	0.986	0.986	0.898	0.895	0.995	0.995
zoo-NOFS	0.979	0.979	0.992	0.998	1.0	1.0	0.973	0.989	0.897	0.994	0.903	1.0
zoo-Overall	0.979	0.993	0.994	1.0	0.929	0.989	0.986	0.989	0.897	0.994	0.903	1.0

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

metric. Results for F1-score are presented in Tables 20, 21, and 22. Finally, Tables 23, 24, and 25 display the results for the classification accuracy.

Table 18. MCAR Results at 25% Missingness for the AUC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.876	0.877	0.896	0.882	0.872	0.88	0.845	0.864	0.831	0.832	0.887	0.887
Australian-NOFS	0.879	0.884	0.898	0.879	0.881	0.895	0.865	0.875	0.842	0.824	0.882	0.881
Australian-Overall	0.876	0.877	0.891	0.883	0.893	0.884	0.865	0.864	0.842	0.832	0.887	0.887
boston-FS	0.925	0.917	0.916	0.913	0.912	0.917	0.878	0.878	0.911	0.911	0.907	0.907
boston-NOFS	0.919	0.906	0.927	0.917	0.916	0.907	0.888	0.887	0.919	0.909	0.906	0.903
boston-Overall	0.925	0.917	0.917	0.914	0.926	0.923	0.878	0.878	0.919	0.909	0.907	0.907
churn-FS	0.866	0.87	0.874	0.871	0.866	0.874	0.834	0.819	0.85	0.849	0.862	0.86
churn-NOFS	0.864	0.87	0.867	0.865	0.868	0.869	0.847	0.851	0.856	0.859	0.866	0.866
churn-Overall	0.866	0.87	0.872	0.876	0.862	0.867	0.847	0.851	0.856	0.859	0.866	0.866
compas-two-years-FS	0.683	0.676	0.69	0.691	0.687	0.656	0.642	0.645	0.678	0.678	0.67	0.675
compas-two-years-NOFS	0.685	0.685	0.69	0.684	0.68	0.673	0.664	0.663	0.682	0.681	0.677	0.674
compas-two-years-Overall	0.685	0.685	0.69	0.69	0.674	0.678	0.664	0.663	0.682	0.681	0.67	0.675
image-FS	0.801	0.801	—	—	0.839	0.845	0.817	0.817	0.851	0.845	0.878	0.878
image-NOFS	0.868	0.866	—	—	0.862	0.864	0.837	0.844	0.864	0.856	0.88	0.882
image-Overall	0.868	0.866	—	—	0.862	0.866	0.837	0.844	0.864	0.856	0.88	0.882
page-blocks-FS	0.983	0.982	0.986	0.985	0.982	0.981	0.966	0.964	0.912	0.912	0.983	0.979
page-blocks-NOFS	0.982	0.983	0.983	0.985	0.983	0.983	0.966	0.968	0.961	0.966	0.984	0.982
page-blocks-Overall	0.983	0.982	0.983	0.986	0.983	0.983	0.966	0.968	0.961	0.966	0.984	0.982
parkinsons-FS	0.793	0.832	0.847	0.85	0.806	0.838	0.78	0.808	0.833	0.833	0.849	0.933
parkinsons-NOFS	0.893	0.904	0.915	0.919	0.891	0.899	0.86	0.873	0.837	0.887	0.918	0.93
parkinsons-Overall	0.893	0.904	0.919	0.914	0.901	0.882	0.86	0.873	0.833	0.833	0.918	0.933
segment-FS	0.999	0.999	0.999	1.0	0.999	0.999	0.978	0.985	0.94	0.941	0.998	0.998
segment-NOFS	0.999	0.999	1.0	1.0	0.999	0.998	0.994	0.99	0.962	0.967	0.998	0.999
segment-Overall	0.999	0.999	1.0	1.0	0.999	0.999	0.994	0.99	0.962	0.967	0.998	0.999
stock-FS	0.976	0.976	0.988	0.99	0.979	0.98	0.937	0.937	0.93	0.93	0.983	0.981
stock-NOFS	0.983	0.981	0.99	0.992	0.981	0.982	0.958	0.959	0.938	0.939	0.983	0.981
stock-Overall	0.983	0.981	0.991	0.99	0.977	0.98	0.958	0.959	0.938	0.939	0.983	0.981
zoo-FS	0.973	0.974	1.0	1.0	0.973	0.997	0.998	0.998	1.0	1.0	0.99	0.994
zoo-NOFS	0.998	1.0	1.0	1.0	0.99	1.0	1.0	0.989	0.938	1.0	1.0	0.995
zoo-Overall	0.973	1.0	1.0	1.0	0.998	1.0	0.998	0.989	1.0	1.0	1.0	0.995

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 19. MCAR Results at 50% Missingness for the AUC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.864	0.864	0.845	0.855	0.846	0.856	0.806	0.825	0.856	0.827	0.865	0.868
Australian-NOFS	0.843	0.859	0.851	0.858	0.852	0.836	0.792	0.821	0.853	0.838	0.86	0.866
Australian-Overall	0.843	0.859	0.852	0.852	0.854	0.865	0.806	0.825	0.853	0.838	0.865	0.868
boston-FS	0.863	0.863	0.893	0.868	0.839	0.845	0.754	0.754	0.862	0.836	0.854	0.851
boston-NOFS	0.853	0.831	0.885	0.884	0.848	0.851	0.765	0.761	0.86	0.808	0.839	0.842
boston-Overall	0.853	0.863	0.884	0.881	0.853	0.844	0.754	0.761	0.86	0.836	0.854	0.842
churn-FS	0.785	0.785	0.777	0.787	0.76	0.79	0.733	0.739	0.768	0.762	0.777	0.777
churn-NOFS	0.789	0.79	0.785	0.789	0.787	0.787	0.772	0.781	0.776	0.784	0.784	0.785
churn-Overall	0.789	0.79	0.79	0.789	0.789	0.799	0.772	0.781	0.776	0.762	0.784	0.785
compas-two-years-FS	0.626	0.626	0.647	0.639	0.635	0.642	0.594	0.604	0.632	0.632	0.633	0.633
compas-two-years-NOFS	0.648	0.646	0.642	0.639	0.646	0.639	0.611	0.614	0.638	0.639	0.642	0.643
compas-two-years-Overall	0.626	0.646	0.651	0.647	0.649	0.646	0.594	0.614	0.632	0.639	0.633	0.643
image-FS	0.778	0.754	—	—	0.781	0.754	0.681	0.671	0.797	0.817	0.823	0.823
image-NOFS	0.826	0.82	—	—	0.841	0.824	0.735	0.75	0.845	0.838	0.843	0.847
image-Overall	0.826	0.82	—	—	0.82	0.833	0.735	0.75	0.845	0.838	0.843	0.847
page-blocks-FS	0.943	0.934	0.963	0.963	0.952	0.956	0.896	0.879	0.791	0.791	0.955	0.934
page-blocks-NOFS	0.96	0.956	0.969	0.967	0.96	0.958	0.899	0.91	0.906	0.917	0.957	0.958
page-blocks-Overall	0.96	0.956	0.967	0.968	0.959	0.955	0.899	0.91	0.906	0.917	0.957	0.958
parkinsons-FS	0.731	0.704	0.817	0.842	0.747	0.693	0.7	0.652	0.812	0.795	0.816	0.809
parkinsons-NOFS	0.842	0.819	0.839	0.869	0.84	0.843	0.753	0.677	0.821	0.814	0.847	0.822
parkinsons-Overall	0.842	0.819	0.82	0.84	0.837	0.803	0.753	0.652	0.812	0.795	0.847	0.809
segment-FS	0.99	0.992	0.997	0.986	0.986	0.992	0.864	0.909	0.852	0.86	0.992	0.991
segment-NOFS	0.995	0.995	0.995	0.998	0.992	0.994	0.923	0.93	0.912	0.914	0.993	0.992
segment-Overall	0.995	0.995	0.997	0.997	0.994	0.991	0.923	0.93	0.912	0.914	0.993	0.992
stock-FS	0.903	0.903	0.972	0.949	0.949	0.918	0.784	0.784	0.8	0.8	0.935	0.923
stock-NOFS	0.948	0.944	0.965	0.971	0.92	0.938	0.809	0.822	0.858	0.842	0.935	0.95
stock-Overall	0.948	0.944	0.965	0.971	0.933	0.932	0.809	0.822	0.858	0.842	0.935	0.95
zoo-FS	0.965	0.918	0.925	0.81	0.86	0.901	0.921	0.821	0.833	0.818	0.894	0.699
zoo-NOFS	0.97	0.848	0.938	0.946	0.902	0.902	0.841	0.844	0.821	0.819	0.884	0.829
zoo-Overall	0.97	0.848	0.93	0.916	0.854	0.925	0.841	0.844	0.833	0.818	0.884	0.829

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 20. MCAR Results at 10% Missingness for the F1 Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.902	0.905	0.922	0.914	0.913	0.919	0.885	0.915	0.89	0.897	0.906	0.906
Australian-NOFS	0.916	0.911	0.917	0.915	0.911	0.904	0.886	0.896	0.892	0.896	0.915	0.91
Australian-Overall	0.902	0.911	0.922	0.914	0.909	0.908	0.885	0.896	0.892	0.896	0.906	0.906
boston-FS	0.933	0.933	0.926	0.926	0.924	0.924	0.912	0.917	0.89	0.89	0.926	0.926
boston-NOFS	0.931	0.928	0.94	0.935	0.927	0.931	0.933	0.908	0.913	0.921	0.928	0.926
boston-Overall	0.933	0.933	0.938	0.941	0.922	0.921	0.912	0.908	0.913	0.921	0.926	0.926
churn-FS	0.914	0.914	0.92	0.916	0.907	0.916	0.887	0.883	0.884	0.885	0.906	0.906
churn-NOFS	0.909	0.912	0.914	0.916	0.91	0.916	0.904	0.91	0.886	0.886	0.906	0.909
churn-Overall	0.909	0.912	0.917	0.916	0.908	0.919	0.904	0.91	0.884	0.886	0.906	0.909
compas-two-years-FS	0.704	0.704	0.698	0.692	0.699	0.705	0.693	0.693	0.688	0.688	0.697	0.697
compas-two-years-NOFS	0.702	0.698	0.703	0.701	0.701	0.698	0.702	0.702	0.703	0.699	0.692	0.692
compas-two-years-Overall	0.702	0.704	0.703	0.696	0.7	0.699	0.693	0.702	0.688	0.688	0.692	0.697
image-FS	0.87	0.845	—	—	0.883	0.875	0.85	0.85	0.845	0.849	0.877	0.878
image-NOFS	0.885	0.884	—	—	0.881	0.878	0.877	0.877	0.863	0.859	0.892	0.887
image-Overall	0.885	0.884	—	—	0.882	0.883	0.877	0.877	0.863	0.859	0.892	0.887
page-blocks-FS	0.99	0.989	0.99	0.989	0.989	0.989	0.983	0.985	0.969	0.969	0.988	0.988
page-blocks-NOFS	0.988	0.988	0.99	0.99	0.989	0.988	0.983	0.982	0.973	0.974	0.987	0.987
page-blocks-Overall	0.988	0.989	0.99	0.99	0.987	0.989	0.983	0.982	0.973	0.974	0.988	0.987
parkinsons-FS	0.85	0.846	0.871	0.871	0.849	0.849	0.85	0.845	0.848	0.861	0.868	0.866
parkinsons-NOFS	0.896	0.896	0.935	0.923	0.896	0.895	0.898	0.892	0.899	0.907	0.919	0.914
parkinsons-Overall	0.896	0.896	0.932	0.925	0.907	0.894	0.898	0.892	0.899	0.907	0.919	0.914
segment-FS	0.999	0.999	1.0	1.0	0.999	0.999	0.996	0.997	0.978	0.977	0.999	0.999
segment-NOFS	0.999	1.0	1.0	1.0	0.999	1.0	0.998	0.999	0.985	0.986	1.0	0.999
segment-Overall	0.999	1.0	1.0	1.0	1.0	1.0	0.998	0.999	0.985	0.986	1.0	0.999
stock-FS	0.99	0.99	0.993	0.993	0.984	0.987	0.975	0.975	0.954	0.954	0.99	0.99
stock-NOFS	0.989	0.99	0.992	0.993	0.99	0.989	0.979	0.98	0.969	0.97	0.991	0.991
stock-Overall	0.99	0.99	0.993	0.993	0.986	0.986	0.979	0.98	0.969	0.97	0.99	0.99
zoo-FS	0.993	0.993	0.895	0.895	0.929	0.929	0.986	0.986	0.898	0.895	0.995	0.995
zoo-NOFS	0.979	0.979	0.992	0.998	1.0	1.0	0.973	0.989	0.897	0.994	0.903	1.0
zoo-Overall	0.979	0.993	0.994	1.0	0.929	0.989	0.986	0.989	0.897	0.994	0.903	1.0

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 21. MCAR Results at 25% Missingness for the F1 Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.876	0.877	0.896	0.882	0.872	0.88	0.845	0.864	0.831	0.832	0.887	0.887
Australian-NOFS	0.879	0.884	0.898	0.879	0.881	0.895	0.865	0.875	0.842	0.824	0.882	0.881
Australian-Overall	0.876	0.877	0.891	0.883	0.893	0.884	0.865	0.864	0.842	0.832	0.887	0.887
boston-FS	0.925	0.917	0.916	0.913	0.912	0.917	0.878	0.878	0.911	0.911	0.907	0.907
boston-NOFS	0.919	0.906	0.927	0.917	0.916	0.907	0.888	0.887	0.919	0.909	0.906	0.903
boston-Overall	0.925	0.917	0.917	0.914	0.926	0.923	0.878	0.878	0.919	0.909	0.907	0.907
churn-FS	0.866	0.87	0.874	0.871	0.866	0.874	0.834	0.819	0.85	0.849	0.862	0.86
churn-NOFS	0.864	0.87	0.867	0.865	0.868	0.869	0.847	0.851	0.856	0.859	0.866	0.866
churn-Overall	0.866	0.87	0.872	0.876	0.862	0.867	0.847	0.851	0.856	0.859	0.866	0.866
compas-two-years-FS	0.683	0.676	0.69	0.691	0.687	0.656	0.642	0.645	0.678	0.678	0.67	0.675
compas-two-years-NOFS	0.685	0.685	0.69	0.684	0.68	0.673	0.664	0.663	0.682	0.681	0.677	0.674
compas-two-years-Overall	0.685	0.685	0.69	0.69	0.674	0.678	0.664	0.663	0.682	0.681	0.67	0.675
image-FS	0.801	0.801	—	—	0.839	0.845	0.817	0.817	0.851	0.845	0.878	0.878
image-NOFS	0.868	0.866	—	—	0.862	0.864	0.837	0.844	0.864	0.856	0.88	0.882
image-Overall	0.868	0.866	—	—	0.862	0.866	0.837	0.844	0.864	0.856	0.88	0.882
page-blocks-FS	0.983	0.982	0.986	0.985	0.982	0.981	0.966	0.964	0.912	0.912	0.983	0.979
page-blocks-NOFS	0.982	0.983	0.983	0.985	0.983	0.983	0.966	0.968	0.961	0.966	0.984	0.982
page-blocks-Overall	0.983	0.982	0.983	0.986	0.983	0.983	0.966	0.968	0.961	0.966	0.984	0.982
parkinsons-FS	0.793	0.832	0.847	0.85	0.806	0.838	0.78	0.808	0.833	0.833	0.849	0.933
parkinsons-NOFS	0.893	0.904	0.915	0.919	0.891	0.899	0.86	0.873	0.837	0.887	0.918	0.93
parkinsons-Overall	0.893	0.904	0.919	0.914	0.901	0.882	0.86	0.873	0.833	0.833	0.918	0.933
segment-FS	0.999	0.999	0.999	1.0	0.999	0.999	0.978	0.985	0.94	0.941	0.998	0.998
segment-NOFS	0.999	0.999	1.0	1.0	0.999	0.998	0.994	0.99	0.962	0.967	0.998	0.999
segment-Overall	0.999	0.999	1.0	1.0	0.999	0.999	0.994	0.99	0.962	0.967	0.998	0.999
stock-FS	0.976	0.976	0.988	0.99	0.979	0.98	0.937	0.937	0.93	0.93	0.983	0.981
stock-NOFS	0.983	0.981	0.99	0.992	0.981	0.982	0.958	0.959	0.938	0.939	0.983	0.981
stock-Overall	0.983	0.981	0.991	0.99	0.977	0.98	0.958	0.959	0.938	0.939	0.983	0.981
zoo-FS	0.973	0.974	1.0	1.0	0.973	0.997	0.998	0.998	1.0	1.0	0.99	0.994
zoo-NOFS	0.998	1.0	1.0	1.0	0.99	1.0	1.0	0.989	0.938	1.0	1.0	0.995
zoo-Overall	0.973	1.0	1.0	1.0	0.998	1.0	0.998	0.989	1.0	1.0	1.0	0.995

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 22. MCAR Results at 50% Missingness for the F1 Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.864	0.864	0.845	0.855	0.846	0.856	0.806	0.825	0.856	0.827	0.865	0.868
Australian-NOFS	0.843	0.859	0.851	0.858	0.852	0.836	0.792	0.821	0.853	0.838	0.86	0.866
Australian-Overall	0.843	0.859	0.852	0.852	0.854	0.865	0.806	0.825	0.853	0.838	0.865	0.868
boston-FS	0.863	0.863	0.893	0.868	0.839	0.845	0.754	0.754	0.862	0.836	0.854	0.851
boston-NOFS	0.853	0.831	0.885	0.884	0.848	0.851	0.765	0.761	0.86	0.808	0.839	0.842
boston-Overall	0.853	0.863	0.884	0.881	0.853	0.844	0.754	0.761	0.86	0.836	0.854	0.842
churn-FS	0.785	0.785	0.777	0.787	0.76	0.79	0.733	0.739	0.768	0.762	0.777	0.777
churn-NOFS	0.789	0.79	0.785	0.789	0.787	0.787	0.772	0.781	0.776	0.784	0.784	0.785
churn-Overall	0.789	0.79	0.79	0.789	0.789	0.799	0.772	0.781	0.776	0.762	0.784	0.785
compas-two-years-FS	0.626	0.626	0.647	0.639	0.635	0.642	0.594	0.604	0.632	0.632	0.633	0.633
compas-two-years-NOFS	0.648	0.646	0.642	0.639	0.646	0.639	0.611	0.614	0.638	0.639	0.642	0.643
compas-two-years-Overall	0.626	0.646	0.651	0.647	0.649	0.646	0.594	0.614	0.632	0.639	0.633	0.643
image-FS	0.778	0.754	—	—	0.781	0.754	0.681	0.671	0.797	0.817	0.823	0.823
image-NOFS	0.826	0.82	—	—	0.841	0.824	0.735	0.75	0.845	0.838	0.843	0.847
image-Overall	0.826	0.82	—	—	0.82	0.833	0.735	0.75	0.845	0.838	0.843	0.847
page-blocks-FS	0.943	0.934	0.963	0.963	0.952	0.956	0.896	0.879	0.791	0.791	0.955	0.934
page-blocks-NOFS	0.96	0.956	0.969	0.967	0.96	0.958	0.899	0.91	0.906	0.917	0.957	0.958
page-blocks-Overall	0.96	0.956	0.967	0.968	0.959	0.955	0.899	0.91	0.906	0.917	0.957	0.958
parkinsons-FS	0.731	0.704	0.817	0.842	0.747	0.693	0.7	0.652	0.812	0.795	0.816	0.809
parkinsons-NOFS	0.842	0.819	0.839	0.869	0.84	0.843	0.753	0.677	0.821	0.814	0.847	0.822
parkinsons-Overall	0.842	0.819	0.82	0.84	0.837	0.803	0.753	0.652	0.812	0.795	0.847	0.809
segment-FS	0.99	0.992	0.997	0.986	0.986	0.992	0.864	0.909	0.852	0.86	0.992	0.991
segment-NOFS	0.995	0.995	0.995	0.998	0.992	0.994	0.923	0.93	0.912	0.914	0.993	0.992
segment-Overall	0.995	0.995	0.997	0.997	0.994	0.991	0.923	0.93	0.912	0.914	0.993	0.992
stock-FS	0.903	0.903	0.972	0.949	0.949	0.918	0.784	0.784	0.8	0.8	0.935	0.923
stock-NOFS	0.948	0.944	0.965	0.971	0.92	0.938	0.809	0.822	0.858	0.842	0.935	0.95
stock-Overall	0.948	0.944	0.965	0.971	0.933	0.932	0.809	0.822	0.858	0.842	0.935	0.95
zoo-FS	0.965	0.918	0.925	0.81	0.86	0.901	0.921	0.821	0.833	0.818	0.894	0.699
zoo-NOFS	0.97	0.848	0.938	0.946	0.902	0.902	0.841	0.844	0.821	0.819	0.884	0.829
zoo-Overall	0.97	0.848	0.93	0.916	0.854	0.925	0.841	0.844	0.833	0.818	0.884	0.829

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 23. MCAR Results at 10% Missingness for the ACC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian	0.846	0.855	0.878	0.858	0.861	0.841	0.841	0.841	0.829	0.838	0.855	0.855
Australian-FS	0.846	0.849	0.875	0.867	0.864	0.864	0.841	0.861	0.835	0.843	0.855	0.855
Australian-NOFS	0.858	0.855	0.87	0.864	0.861	0.849	0.843	0.841	0.829	0.838	0.858	0.849
boston	0.881	0.881	0.893	0.897	0.874	0.862	0.858	0.862	0.874	0.881	0.881	0.874
boston-FS	0.881	0.881	0.874	0.874	0.881	0.881	0.858	0.862	0.862	0.862	0.881	0.874
boston-NOFS	0.881	0.881	0.885	0.893	0.877	0.874	0.881	0.862	0.874	0.881	0.889	0.874
churn	0.936	0.936	0.944	0.942	0.943	0.943	0.928	0.924	0.924	0.922	0.931	0.932
churn-FS	0.936	0.936	0.944	0.943	0.936	0.943	0.923	0.926	0.924	0.922	0.934	0.94
churn-NOFS	0.936	0.936	0.944	0.942	0.944	0.944	0.928	0.924	0.924	0.922	0.931	0.932
compas-two-years	0.665	0.663	0.658	0.655	0.658	0.653	0.648	0.662	0.65	0.65	0.655	0.653
compas-two-years-FS	0.663	0.663	0.656	0.655	0.653	0.662	0.648	0.648	0.65	0.65	0.653	0.653
compas-two-years-NOFS	0.665	0.659	0.657	0.654	0.663	0.651	0.662	0.662	0.665	0.657	0.655	0.651
image	0.863	0.869	—	—	0.865	0.866	0.862	0.866	0.858	0.861	0.873	0.867
image-FS	0.856	0.845	—	—	0.863	0.862	0.859	0.846	0.847	0.846	0.864	0.864
image-NOFS	0.863	0.869	—	—	0.859	0.872	0.862	0.866	0.858	0.861	0.873	0.867
page-blocks	0.972	0.972	0.974	0.973	0.97	0.972	0.966	0.967	0.961	0.961	0.973	0.973
page-blocks-FS	0.971	0.972	0.974	0.974	0.973	0.972	0.966	0.968	0.959	0.959	0.973	0.971
page-blocks-NOFS	0.972	0.972	0.974	0.972	0.97	0.971	0.966	0.967	0.961	0.961	0.973	0.973
parkinsons	0.867	0.867	0.908	0.918	0.867	0.857	0.878	0.888	0.898	0.888	0.898	0.888
parkinsons-FS	0.837	0.827	0.867	0.867	0.837	0.827	0.867	0.857	0.867	0.888	0.857	0.857
parkinsons-NOFS	0.867	0.867	0.929	0.898	0.867	0.857	0.878	0.888	0.898	0.888	0.898	0.888
segment	0.996	0.997	0.996	0.998	0.997	0.996	0.993	0.995	0.958	0.957	0.997	0.997
segment-FS	0.996	0.994	0.998	0.998	0.993	0.995	0.992	0.992	0.947	0.951	0.994	0.993
segment-NOFS	0.996	0.997	0.995	0.996	0.995	0.997	0.993	0.995	0.958	0.957	0.997	0.997
stock	0.956	0.956	0.96	0.96	0.937	0.943	0.922	0.924	0.901	0.905	0.956	0.96
stock-FS	0.956	0.956	0.96	0.956	0.933	0.947	0.924	0.924	0.872	0.872	0.956	0.96
stock-NOFS	0.947	0.945	0.956	0.958	0.943	0.937	0.922	0.924	0.901	0.905	0.954	0.954
zoo	0.961	0.961	0.961	1.0	0.941	0.941	0.941	0.941	0.902	0.961	0.863	1.0
zoo-FS	0.961	0.961	0.941	0.941	0.941	0.941	0.941	0.941	0.922	0.902	0.98	0.98
zoo-NOFS	0.961	0.922	0.961	0.98	1.0	1.0	0.98	0.941	0.902	0.961	0.863	1.0

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 24. MCAR Results at 25% Missingness for the ACC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian	0.8	0.806	0.823	0.838	0.814	0.809	0.803	0.791	0.771	0.771	0.814	0.814
Australian-FS	0.8	0.806	0.832	0.826	0.809	0.809	0.797	0.791	0.774	0.771	0.814	0.814
Australian-NOFS	0.803	0.806	0.835	0.806	0.817	0.832	0.803	0.812	0.771	0.762	0.809	0.806
boston	0.877	0.866	0.87	0.862	0.866	0.87	0.814	0.814	0.874	0.862	0.846	0.846
boston-FS	0.877	0.866	0.862	0.85	0.858	0.866	0.814	0.814	0.858	0.858	0.846	0.846
boston-NOFS	0.881	0.846	0.881	0.87	0.862	0.846	0.814	0.814	0.874	0.862	0.862	0.85
churn	0.919	0.918	0.922	0.931	0.922	0.919	0.904	0.894	0.914	0.916	0.924	0.921
churn-FS	0.919	0.923	0.922	0.924	0.925	0.924	0.895	0.891	0.907	0.907	0.922	0.923
churn-NOFS	0.924	0.918	0.931	0.925	0.925	0.916	0.904	0.894	0.914	0.916	0.924	0.921
compas-two-years	0.644	0.642	0.651	0.651	0.636	0.641	0.623	0.629	0.638	0.639	0.629	0.638
compas-two-years-FS	0.639	0.637	0.654	0.653	0.645	0.619	0.61	0.609	0.635	0.635	0.629	0.638
compas-two-years-NOFS	0.644	0.642	0.653	0.642	0.643	0.637	0.623	0.629	0.638	0.639	0.638	0.631
image	0.853	0.853	—	—	0.853	0.862	0.844	0.846	0.861	0.86	0.866	0.862
image-FS	0.815	0.815	—	—	0.849	0.849	0.845	0.845	0.85	0.852	0.862	0.862
image-NOFS	0.853	0.853	—	—	0.862	0.86	0.844	0.846	0.861	0.86	0.866	0.862
page-blocks	0.966	0.965	0.965	0.968	0.966	0.965	0.947	0.948	0.95	0.95	0.963	0.963
page-blocks-FS	0.966	0.965	0.967	0.967	0.961	0.966	0.948	0.947	0.936	0.936	0.963	0.961
page-blocks-NOFS	0.965	0.963	0.967	0.967	0.967	0.965	0.947	0.948	0.95	0.95	0.963	0.963
parkinsons	0.878	0.878	0.898	0.908	0.878	0.857	0.898	0.867	0.837	0.837	0.867	0.888
parkinsons-FS	0.827	0.837	0.878	0.878	0.806	0.857	0.857	0.827	0.837	0.837	0.857	0.888
parkinsons-NOFS	0.878	0.878	0.908	0.898	0.888	0.888	0.898	0.867	0.867	0.857	0.867	0.898
segment	0.988	0.991	0.997	0.997	0.992	0.993	0.976	0.973	0.922	0.927	0.988	0.99
segment-FS	0.988	0.991	0.997	0.997	0.99	0.989	0.958	0.965	0.897	0.896	0.985	0.988
segment-NOFS	0.989	0.99	0.997	0.997	0.99	0.99	0.976	0.973	0.922	0.927	0.988	0.99
stock	0.931	0.933	0.949	0.947	0.924	0.926	0.893	0.893	0.865	0.865	0.931	0.924
stock-FS	0.924	0.924	0.952	0.952	0.918	0.924	0.859	0.859	0.853	0.853	0.931	0.933
stock-NOFS	0.931	0.933	0.947	0.952	0.92	0.931	0.893	0.893	0.865	0.865	0.931	0.924
zoo	0.961	1.0	1.0	1.0	0.98	1.0	0.98	0.98	1.0	1.0	1.0	0.98
zoo-FS	0.961	0.961	1.0	1.0	0.98	0.98	0.98	0.98	1.0	1.0	0.98	0.98
zoo-NOFS	0.98	1.0	1.0	1.0	0.98	1.0	1.0	0.98	0.902	1.0	1.0	0.98

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 25. MCAR Results at 50% Missingness for the ACC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian	0.803	0.806	0.826	0.817	0.78	0.812	0.73	0.777	0.8	0.768	0.8	0.817
Australian-FS	0.8	0.8	0.797	0.806	0.783	0.783	0.73	0.777	0.803	0.771	0.8	0.817
Australian-NOFS	0.803	0.806	0.809	0.814	0.768	0.78	0.736	0.774	0.8	0.768	0.803	0.809
boston	0.802	0.81	0.834	0.838	0.806	0.798	0.731	0.719	0.822	0.81	0.783	0.791
boston-FS	0.81	0.81	0.858	0.814	0.791	0.787	0.731	0.731	0.838	0.81	0.783	0.787
boston-NOFS	0.802	0.787	0.85	0.834	0.818	0.818	0.711	0.719	0.822	0.779	0.779	0.791
churn	0.897	0.887	0.898	0.895	0.892	0.889	0.872	0.872	0.888	0.89	0.893	0.887
churn-FS	0.892	0.892	0.899	0.894	0.887	0.89	0.863	0.866	0.89	0.89	0.89	0.89
churn-NOFS	0.897	0.887	0.904	0.895	0.893	0.892	0.872	0.872	0.888	0.885	0.893	0.887
compas-two-years	0.593	0.612	0.615	0.617	0.614	0.609	0.581	0.589	0.606	0.605	0.603	0.606
compas-two-years-FS	0.593	0.593	0.612	0.613	0.603	0.612	0.581	0.579	0.606	0.606	0.603	0.603
compas-two-years-NOFS	0.613	0.612	0.61	0.61	0.615	0.609	0.584	0.589	0.608	0.605	0.612	0.606
image	0.846	0.838	—	—	0.837	0.854	0.807	0.809	0.855	0.854	0.85	0.853
image-FS	0.817	0.812	—	—	0.826	0.812	0.807	0.804	0.828	0.848	0.844	0.844
image-NOFS	0.846	0.838	—	—	0.851	0.843	0.807	0.809	0.855	0.854	0.85	0.853
page-blocks	0.942	0.941	0.953	0.953	0.945	0.944	0.911	0.92	0.933	0.936	0.941	0.946
page-blocks-FS	0.94	0.945	0.955	0.954	0.948	0.941	0.91	0.912	0.923	0.923	0.942	0.942
page-blocks-NOFS	0.942	0.941	0.952	0.953	0.943	0.946	0.911	0.92	0.933	0.936	0.941	0.946
parkinsons	0.796	0.786	0.878	0.806	0.806	0.776	0.776	0.765	0.847	0.816	0.816	0.816
parkinsons-FS	0.786	0.776	0.857	0.806	0.837	0.776	0.786	0.765	0.847	0.816	0.827	0.816
parkinsons-NOFS	0.796	0.786	0.847	0.847	0.827	0.827	0.776	0.765	0.857	0.816	0.816	0.806
segment	0.975	0.974	0.985	0.984	0.976	0.971	0.912	0.919	0.872	0.878	0.967	0.966
segment-FS	0.972	0.97	0.987	0.983	0.965	0.969	0.89	0.902	0.864	0.858	0.964	0.965
segment-NOFS	0.975	0.974	0.986	0.987	0.97	0.973	0.912	0.919	0.872	0.878	0.967	0.966
stock	0.88	0.88	0.916	0.92	0.848	0.859	0.731	0.745	0.766	0.787	0.851	0.874
stock-FS	0.823	0.823	0.916	0.886	0.865	0.842	0.718	0.722	0.722	0.722	0.851	0.844
stock-NOFS	0.88	0.88	0.909	0.918	0.842	0.851	0.731	0.745	0.766	0.787	0.851	0.874
zoo	0.961	0.882	0.902	0.882	0.784	0.882	0.824	0.863	0.784	0.784	0.863	0.863
zoo-FS	0.941	0.882	0.922	0.824	0.843	0.863	0.902	0.765	0.784	0.784	0.882	0.725
zoo-NOFS	0.961	0.882	0.922	0.902	0.824	0.863	0.824	0.863	0.784	0.784	0.863	0.863

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

C.6 MAR: Downstream Task Results

This section presents the results for MAR data under varying levels of missingness, and multiple metrics. Tables 26, 27, and 28 present the results for 10%, 25%, and 50% missingness for the AUC metric. Results for F1-score are presented in Tables 29, 30, and 31. Finally, Tables 32, 33, and 34 display the results for the classification accuracy.

Table 26. MAR Results at 10% Missingness for the AUC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.899	0.907	0.893	0.895	0.905	0.907	0.88	0.886	0.873	0.873	0.895	0.911
Australian-NOFS	0.921	0.912	0.91	0.915	0.901	0.918	0.912	0.904	0.886	0.89	0.919	0.899
Australian-Overall	0.899	0.907	0.907	0.915	0.918	0.912	0.912	0.904	0.886	0.89	0.895	0.911
boston-FS	0.94	0.94	0.926	0.926	0.928	0.918	0.918	0.918	0.926	0.926	0.933	0.94
boston-NOFS	0.936	0.932	0.947	0.943	0.938	0.939	0.936	0.929	0.93	0.923	0.94	0.932
boston-Overall	0.94	0.94	0.926	0.921	0.942	0.922	0.918	0.918	0.93	0.923	0.933	0.94
churn-FS	0.893	0.897	0.899	0.896	0.896	0.897	0.89	0.881	0.871	0.873	0.896	0.896
churn-NOFS	0.894	0.896	0.899	0.902	0.898	0.902	0.891	0.894	0.873	0.875	0.902	0.898
churn-Overall	0.893	0.896	0.899	0.897	0.898	0.9	0.891	0.894	0.873	0.875	0.896	0.896
compas-two-years-FS	0.704	0.704	0.694	0.71	0.707	0.702	0.687	0.689	0.705	0.702	0.706	0.693
compas-two-years-NOFS	0.707	0.705	0.711	0.706	0.703	0.7	0.699	0.703	0.703	0.697	0.709	0.702
compas-two-years-Overall	0.704	0.704	0.711	0.71	0.7	0.699	0.699	0.703	0.703	0.697	0.706	0.702
image-FS	0.855	0.858	—	—	0.868	0.857	0.873	0.873	0.82	0.82	0.872	0.868
image-NOFS	0.878	0.879	—	—	0.879	0.888	0.875	0.875	0.862	0.865	0.882	0.885
image-Overall	0.878	0.879	—	—	0.875	0.879	0.875	0.875	0.862	0.865	0.882	0.885
page-blocks-FS	0.987	0.988	0.989	0.988	0.988	0.988	0.985	0.986	0.976	0.954	0.987	0.987
page-blocks-NOFS	0.988	0.988	0.989	0.99	0.988	0.988	0.985	0.985	0.982	0.983	0.987	0.988
page-blocks-Overall	0.988	0.988	0.988	0.988	0.987	0.987	0.985	0.985	0.982	0.983	0.987	0.988
parkinsons-FS	0.873	0.873	0.87	0.871	0.875	0.876	0.874	0.874	0.889	0.801	0.87	0.87
parkinsons-NOFS	0.908	0.911	0.923	0.926	0.925	0.912	0.918	0.918	0.93	0.894	0.917	0.912
parkinsons-Overall	0.873	0.873	0.936	0.87	0.905	0.873	0.918	0.918	0.93	0.894	0.87	0.87
segment-FS	0.999	0.999	1.0	1.0	0.999	1.0	0.999	0.998	0.96	0.954	0.998	0.998
segment-NOFS	0.999	1.0	1.0	1.0	1.0	1.0	0.999	1.0	0.968	0.97	1.0	0.999
segment-Overall	0.999	1.0	1.0	1.0	0.999	1.0	0.999	1.0	0.968	0.97	0.998	0.998
stock-FS	0.993	0.985	0.995	0.995	0.991	0.992	0.991	0.987	0.965	0.965	0.991	0.991
stock-NOFS	0.993	0.994	0.995	0.995	0.993	0.995	0.991	0.989	0.98	0.981	0.994	0.994
stock-Overall	0.993	0.994	0.995	0.994	0.993	0.994	0.991	0.989	0.98	0.981	0.994	0.994
zoo-FS	1.0	1.0	1.0	1.0	0.824	1.0	1.0	1.0	1.0	1.0	0.987	0.993
zoo-NOFS	0.992	1.0	1.0	1.0	1.0	0.997	1.0	0.997	0.99	1.0	0.983	1.0
zoo-Overall	0.992	1.0	1.0	1.0	0.997	1.0	1.0	0.997	0.99	1.0	0.983	0.993

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 27. MAR Results at 25% Missingness for the AUC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.901	0.905	0.888	0.886	0.883	0.891	0.864	0.87	0.877	0.869	0.909	0.909
Australian-NOFS	0.903	0.898	0.887	0.888	0.887	0.896	0.884	0.888	0.87	0.875	0.908	0.902
Australian-Overall	0.903	0.905	0.886	0.884	0.88	0.898	0.864	0.87	0.877	0.869	0.909	0.909
boston-FS	0.907	0.907	0.903	0.901	0.904	0.908	0.841	0.844	0.887	0.887	0.891	0.891
boston-NOFS	0.894	0.891	0.903	0.905	0.9	0.905	0.858	0.852	0.895	0.887	0.901	0.89
boston-Overall	0.894	0.891	0.901	0.91	0.914	0.9	0.858	0.852	0.895	0.887	0.901	0.89
churn-FS	0.841	0.841	0.847	0.841	0.844	0.839	0.804	0.811	0.808	0.816	0.837	0.837
churn-NOFS	0.833	0.836	0.846	0.842	0.843	0.831	0.84	0.827	0.817	0.814	0.832	0.843
churn-Overall	0.833	0.836	0.845	0.846	0.847	0.841	0.84	0.827	0.808	0.814	0.837	0.843
compas-two-years-FS	0.701	0.697	0.688	0.701	0.688	0.696	0.685	0.7	0.685	0.681	0.682	0.697
compas-two-years-NOFS	0.701	0.695	0.693	0.695	0.69	0.69	0.673	0.692	0.692	0.697	0.694	0.691
compas-two-years-Overall	0.701	0.695	0.695	0.693	0.69	0.685	0.673	0.7	0.685	0.681	0.682	0.691
image-FS	0.806	0.806	—	—	0.798	0.84	0.835	0.839	0.826	0.823	0.871	0.871
image-NOFS	0.871	0.875	—	—	0.871	0.865	0.86	0.862	0.869	0.86	0.878	0.884
image-Overall	0.871	0.875	—	—	0.879	0.869	0.86	0.862	0.869	0.86	0.878	0.884
page-blocks-FS	0.976	0.979	0.982	0.984	0.977	0.978	0.95	0.951	0.932	0.94	0.979	0.98
page-blocks-NOFS	0.976	0.98	0.982	0.984	0.975	0.978	0.95	0.956	0.866	0.947	0.978	0.978
page-blocks-Overall	0.976	0.979	0.984	0.982	0.979	0.976	0.95	0.956	0.932	0.947	0.978	0.978
parkinsons-FS	0.889	0.876	0.872	0.87	0.889	0.864	0.803	0.803	0.844	0.829	0.849	0.881
parkinsons-NOFS	0.904	0.892	0.909	0.918	0.898	0.892	0.883	0.871	0.886	0.883	0.905	0.905
parkinsons-Overall	0.904	0.892	0.872	0.876	0.891	0.875	0.883	0.871	0.886	0.883	0.905	0.881
segment-FS	0.994	0.997	1.0	1.0	0.998	0.997	0.974	0.961	0.941	0.963	0.998	0.999
segment-NOFS	0.999	0.999	1.0	1.0	0.997	0.998	0.989	0.995	0.972	0.975	0.999	0.999
segment-Overall	0.999	0.999	1.0	1.0	0.999	0.998	0.989	0.995	0.972	0.975	0.999	0.999
stock-FS	0.979	0.968	0.98	0.98	0.975	0.977	0.882	0.908	0.886	0.903	0.974	0.974
stock-NOFS	0.979	0.978	0.981	0.986	0.973	0.977	0.887	0.933	0.931	0.942	0.976	0.975
stock-Overall	0.979	0.978	0.982	0.981	0.975	0.978	0.887	0.933	0.931	0.942	0.976	0.974
zoo-FS	0.991	0.984	1.0	0.99	0.986	0.967	0.973	0.989	0.96	0.99	0.923	0.925
zoo-NOFS	0.99	0.989	0.973	1.0	0.998	1.0	0.997	0.998	0.986	0.997	0.952	0.99
zoo-Overall	0.99	0.989	0.965	1.0	1.0	1.0	0.973	0.998	0.96	0.997	0.952	0.925

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 28. MAR Results at 50% Missingness for the AUC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.855	0.887	0.867	0.878	0.852	0.84	0.785	0.838	0.821	0.821	0.863	0.884
Australian-NOFS	0.862	0.873	0.879	0.858	0.851	0.861	0.772	0.821	0.822	0.829	0.86	0.874
Australian-Overall	0.855	0.887	0.882	0.861	0.843	0.849	0.785	0.838	0.822	0.821	0.863	0.884
boston-FS	0.867	0.897	0.884	0.884	0.868	0.877	0.814	0.786	0.869	0.856	0.853	0.863
boston-NOFS	0.887	0.897	0.894	0.899	0.885	0.875	0.818	0.832	0.866	0.864	0.849	0.867
boston-Overall	0.867	0.897	0.895	0.893	0.871	0.875	0.818	0.832	0.866	0.856	0.853	0.863
churn-FS	0.758	0.773	0.764	0.774	0.754	0.747	0.704	0.711	0.765	0.776	0.766	0.757
churn-NOFS	0.755	0.766	0.775	0.793	0.756	0.752	0.72	0.737	0.773	0.764	0.759	0.76
churn-Overall	0.755	0.766	0.775	0.775	0.761	0.762	0.72	0.737	0.765	0.764	0.766	0.757
compas-two-years-FS	0.662	0.668	0.665	0.674	0.642	0.664	0.643	0.658	0.663	0.671	0.659	0.666
compas-two-years-NOFS	0.666	0.682	0.663	0.674	0.658	0.678	0.637	0.662	0.666	0.682	0.665	0.679
compas-two-years-Overall	0.666	0.682	0.668	0.676	0.647	0.678	0.643	0.662	0.666	0.682	0.665	0.679
image-FS	0.771	0.76	—	—	0.742	0.779	0.721	0.735	0.812	0.812	0.805	0.81
image-NOFS	0.824	0.824	—	—	0.836	0.824	0.731	0.749	0.841	0.828	0.857	0.852
image-Overall	0.824	0.824	—	—	0.828	0.83	0.731	0.749	0.841	0.828	0.857	0.852
page-blocks-FS	0.963	0.964	0.952	0.959	0.963	0.962	0.89	0.902	0.878	0.87	0.961	0.957
page-blocks-NOFS	0.963	0.962	0.959	0.96	0.96	0.966	0.896	0.911	0.898	0.918	0.961	0.958
page-blocks-Overall	0.963	0.964	0.958	0.961	0.959	0.964	0.896	0.911	0.898	0.918	0.961	0.957
parkinsons-FS	0.809	0.855	0.861	0.88	0.839	0.813	0.8	0.801	0.865	0.884	0.803	0.85
parkinsons-NOFS	0.875	0.847	0.883	0.876	0.855	0.865	0.778	0.77	0.908	0.903	0.887	0.869
parkinsons-Overall	0.875	0.847	0.876	0.87	0.855	0.84	0.8	0.77	0.908	0.903	0.887	0.869
segment-FS	0.995	0.993	0.991	0.991	0.995	0.991	0.877	0.909	0.877	0.895	0.991	0.993
segment-NOFS	0.995	0.993	0.992	0.994	0.992	0.994	0.886	0.903	0.907	0.934	0.993	0.993
segment-Overall	0.995	0.993	0.994	0.997	0.985	0.995	0.886	0.903	0.907	0.934	0.991	0.993
stock-FS	0.962	0.935	0.966	0.973	0.957	0.95	0.876	0.869	0.713	0.803	0.954	0.939
stock-NOFS	0.962	0.959	0.972	0.971	0.956	0.962	0.871	0.897	0.925	0.928	0.956	0.956
stock-Overall	0.962	0.959	0.969	0.975	0.953	0.954	0.871	0.897	0.925	0.928	0.954	0.956
zoo-FS	0.96	0.905	0.96	0.956	0.949	0.99	0.946	0.948	0.896	0.863	0.967	0.884
zoo-NOFS	0.954	0.952	0.973	0.981	0.908	0.924	0.948	0.952	0.943	0.965	0.954	0.968
zoo-Overall	0.954	0.952	0.954	0.957	0.916	0.979	0.948	0.952	0.943	0.965	0.954	0.968

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 29. MAR Results at 10% Missingness for the F1 Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.899	0.907	0.893	0.895	0.905	0.907	0.88	0.886	0.873	0.873	0.895	0.911
Australian-NOFS	0.921	0.912	0.91	0.915	0.901	0.918	0.912	0.904	0.886	0.89	0.919	0.899
Australian-Overall	0.899	0.907	0.907	0.915	0.918	0.912	0.912	0.904	0.886	0.89	0.895	0.911
boston-FS	0.94	0.94	0.926	0.926	0.928	0.918	0.918	0.918	0.926	0.926	0.933	0.94
boston-NOFS	0.936	0.932	0.947	0.943	0.938	0.939	0.936	0.929	0.93	0.923	0.94	0.932
boston-Overall	0.94	0.94	0.926	0.921	0.942	0.922	0.918	0.918	0.93	0.923	0.933	0.94
churn-FS	0.893	0.897	0.899	0.896	0.896	0.897	0.89	0.881	0.871	0.873	0.896	0.896
churn-NOFS	0.894	0.896	0.899	0.902	0.898	0.902	0.891	0.894	0.873	0.875	0.902	0.898
churn-Overall	0.893	0.896	0.899	0.897	0.898	0.9	0.891	0.894	0.873	0.875	0.896	0.896
compas-two-years-FS	0.704	0.704	0.694	0.71	0.707	0.702	0.687	0.689	0.705	0.702	0.706	0.693
compas-two-years-NOFS	0.707	0.705	0.711	0.706	0.703	0.7	0.699	0.703	0.703	0.697	0.709	0.702
compas-two-years-Overall	0.704	0.704	0.711	0.71	0.7	0.699	0.699	0.703	0.703	0.697	0.706	0.702
image-FS	0.855	0.858	—	—	0.868	0.857	0.873	0.873	0.82	0.82	0.872	0.868
image-NOFS	0.878	0.879	—	—	0.879	0.888	0.875	0.875	0.862	0.865	0.882	0.885
image-Overall	0.878	0.879	—	—	0.875	0.879	0.875	0.875	0.862	0.865	0.882	0.885
page-blocks-FS	0.987	0.988	0.989	0.988	0.988	0.988	0.985	0.986	0.976	0.954	0.987	0.987
page-blocks-NOFS	0.988	0.988	0.989	0.99	0.988	0.988	0.985	0.985	0.982	0.983	0.987	0.988
page-blocks-Overall	0.988	0.988	0.988	0.988	0.987	0.987	0.985	0.985	0.982	0.983	0.987	0.988
parkinsons-FS	0.873	0.873	0.87	0.871	0.875	0.876	0.874	0.874	0.889	0.801	0.87	0.87
parkinsons-NOFS	0.908	0.911	0.923	0.926	0.925	0.912	0.918	0.918	0.93	0.894	0.917	0.912
parkinsons-Overall	0.873	0.873	0.936	0.87	0.905	0.873	0.918	0.918	0.93	0.894	0.87	0.87
segment-FS	0.999	0.999	1.0	1.0	0.999	1.0	0.999	0.998	0.96	0.954	0.998	0.998
segment-NOFS	0.999	1.0	1.0	1.0	1.0	1.0	0.999	1.0	0.968	0.97	1.0	0.999
segment-Overall	0.999	1.0	1.0	1.0	0.999	1.0	0.999	1.0	0.968	0.97	0.998	0.998
stock-FS	0.993	0.985	0.995	0.995	0.991	0.992	0.991	0.987	0.965	0.965	0.991	0.991
stock-NOFS	0.993	0.994	0.995	0.995	0.993	0.995	0.991	0.989	0.98	0.981	0.994	0.994
stock-Overall	0.993	0.994	0.995	0.994	0.993	0.994	0.991	0.989	0.98	0.981	0.994	0.994
zoo-FS	1.0	1.0	1.0	1.0	0.824	1.0	1.0	1.0	1.0	1.0	0.987	0.993
zoo-NOFS	0.992	1.0	1.0	1.0	1.0	0.997	1.0	0.997	0.99	1.0	0.983	1.0
zoo-Overall	0.992	1.0	1.0	1.0	0.997	1.0	1.0	0.997	0.99	1.0	0.983	0.993

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 30. MAR Results at 25% Missingness for the F1 Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.901	0.905	0.888	0.886	0.883	0.89	0.864	0.87	0.877	0.869	0.909	0.909
Australian-NOFS	0.903	0.898	0.887	0.888	0.887	0.896	0.884	0.888	0.87	0.875	0.908	0.902
Australian-Overall	0.903	0.905	0.886	0.884	0.88	0.898	0.864	0.87	0.877	0.869	0.909	0.909
boston-FS	0.907	0.907	0.903	0.901	0.904	0.908	0.841	0.844	0.887	0.887	0.891	0.891
boston-NOFS	0.894	0.891	0.903	0.905	0.9	0.905	0.858	0.852	0.895	0.887	0.901	0.89
boston-Overall	0.894	0.891	0.901	0.91	0.914	0.9	0.858	0.852	0.895	0.887	0.901	0.89
churn-FS	0.841	0.841	0.847	0.841	0.844	0.839	0.804	0.811	0.808	0.816	0.837	0.837
churn-NOFS	0.833	0.836	0.846	0.842	0.843	0.831	0.84	0.827	0.817	0.814	0.832	0.843
churn-Overall	0.833	0.836	0.845	0.846	0.847	0.841	0.84	0.827	0.808	0.814	0.837	0.843
compas-two-years-FS	0.701	0.697	0.688	0.701	0.688	0.696	0.685	0.7	0.685	0.681	0.682	0.697
compas-two-years-NOFS	0.701	0.695	0.693	0.695	0.69	0.69	0.673	0.692	0.692	0.697	0.694	0.691
compas-two-years-Overall	0.701	0.695	0.695	0.693	0.69	0.685	0.673	0.7	0.685	0.681	0.682	0.691
image-FS	0.806	0.806	—	—	0.798	0.84	0.835	0.839	0.826	0.823	0.871	0.871
image-NOFS	0.871	0.875	—	—	0.871	0.865	0.86	0.862	0.869	0.86	0.878	0.884
image-Overall	0.871	0.875	—	—	0.879	0.869	0.86	0.862	0.869	0.86	0.878	0.884
page-blocks-FS	0.976	0.979	0.982	0.984	0.977	0.978	0.95	0.951	0.932	0.94	0.979	0.98
page-blocks-NOFS	0.976	0.98	0.982	0.984	0.975	0.978	0.95	0.956	0.866	0.947	0.978	0.978
page-blocks-Overall	0.976	0.979	0.984	0.982	0.979	0.976	0.95	0.956	0.932	0.947	0.978	0.978
parkinsons-FS	0.889	0.876	0.872	0.87	0.889	0.864	0.803	0.803	0.844	0.829	0.849	0.881
parkinsons-NOFS	0.904	0.892	0.909	0.918	0.898	0.892	0.883	0.871	0.886	0.883	0.905	0.905
parkinsons-Overall	0.904	0.892	0.872	0.876	0.891	0.875	0.883	0.871	0.886	0.883	0.905	0.881
segment-FS	0.994	0.997	1.0	1.0	0.998	0.997	0.974	0.961	0.941	0.963	0.998	0.999
segment-NOFS	0.999	0.999	1.0	1.0	0.997	0.998	0.989	0.995	0.972	0.975	0.999	0.999
segment-Overall	0.999	0.999	1.0	1.0	0.999	0.998	0.989	0.995	0.972	0.975	0.999	0.999
stock-FS	0.979	0.968	0.98	0.98	0.975	0.977	0.882	0.908	0.886	0.903	0.974	0.974
stock-NOFS	0.979	0.978	0.981	0.986	0.973	0.977	0.887	0.933	0.931	0.942	0.976	0.975
stock-Overall	0.979	0.978	0.982	0.981	0.975	0.978	0.887	0.933	0.931	0.942	0.976	0.974
zoo-FS	0.991	0.984	1.0	0.99	0.986	0.967	0.973	0.989	0.96	0.99	0.923	0.925
zoo-NOFS	0.99	0.989	0.973	1.0	0.998	1.0	0.997	0.998	0.986	0.997	0.952	0.99
zoo-Overall	0.99	0.989	0.965	1.0	1.0	1.0	0.973	0.998	0.96	0.997	0.952	0.925

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 31. MAR Results at 50% Missingness for the F1 Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian-FS	0.855	0.887	0.867	0.878	0.852	0.84	0.785	0.838	0.821	0.821	0.863	0.884
Australian-NOFS	0.862	0.873	0.879	0.858	0.851	0.861	0.772	0.821	0.822	0.829	0.86	0.874
Australian-Overall	0.855	0.887	0.882	0.861	0.843	0.849	0.785	0.838	0.822	0.821	0.863	0.884
boston-FS	0.867	0.897	0.884	0.884	0.868	0.877	0.814	0.786	0.869	0.856	0.853	0.863
boston-NOFS	0.887	0.897	0.894	0.899	0.885	0.875	0.818	0.832	0.866	0.864	0.849	0.867
boston-Overall	0.867	0.897	0.895	0.893	0.871	0.875	0.818	0.832	0.866	0.856	0.853	0.863
churn-FS	0.758	0.773	0.764	0.774	0.754	0.747	0.704	0.711	0.765	0.776	0.766	0.757
churn-NOFS	0.755	0.766	0.775	0.793	0.756	0.752	0.72	0.737	0.773	0.764	0.759	0.76
churn-Overall	0.755	0.766	0.775	0.775	0.761	0.762	0.72	0.737	0.765	0.764	0.766	0.757
compas-two-years-FS	0.662	0.668	0.665	0.674	0.642	0.664	0.643	0.658	0.663	0.671	0.659	0.666
compas-two-years-NOFS	0.666	0.682	0.663	0.674	0.658	0.678	0.637	0.662	0.666	0.682	0.665	0.679
compas-two-years-Overall	0.666	0.682	0.668	0.676	0.647	0.678	0.643	0.662	0.666	0.682	0.665	0.679
image-FS	0.771	0.76	—	—	0.742	0.779	0.721	0.735	0.812	0.812	0.805	0.81
image-NOFS	0.824	0.824	—	—	0.836	0.824	0.731	0.749	0.841	0.828	0.857	0.852
image-Overall	0.824	0.824	—	—	0.828	0.83	0.731	0.749	0.841	0.828	0.857	0.852
page-blocks-FS	0.963	0.964	0.952	0.959	0.963	0.962	0.89	0.902	0.878	0.87	0.961	0.957
page-blocks-NOFS	0.963	0.962	0.959	0.96	0.96	0.966	0.896	0.911	0.898	0.918	0.961	0.958
page-blocks-Overall	0.963	0.964	0.958	0.961	0.959	0.964	0.896	0.911	0.898	0.918	0.961	0.957
parkinsons-FS	0.809	0.855	0.861	0.88	0.839	0.813	0.8	0.801	0.865	0.884	0.803	0.85
parkinsons-NOFS	0.875	0.847	0.883	0.876	0.855	0.865	0.778	0.77	0.908	0.903	0.887	0.869
parkinsons-Overall	0.875	0.847	0.876	0.87	0.855	0.84	0.8	0.77	0.908	0.903	0.887	0.869
segment-FS	0.995	0.993	0.991	0.991	0.995	0.991	0.877	0.909	0.877	0.895	0.991	0.993
segment-NOFS	0.995	0.993	0.992	0.994	0.992	0.994	0.886	0.903	0.907	0.934	0.993	0.993
segment-Overall	0.995	0.993	0.994	0.997	0.985	0.995	0.886	0.903	0.907	0.934	0.991	0.993
stock-FS	0.962	0.935	0.966	0.973	0.957	0.95	0.876	0.869	0.713	0.803	0.954	0.939
stock-NOFS	0.962	0.959	0.972	0.971	0.956	0.962	0.871	0.897	0.925	0.928	0.956	0.956
stock-Overall	0.962	0.959	0.969	0.975	0.953	0.954	0.871	0.897	0.925	0.928	0.954	0.956
zoo-FS	0.96	0.905	0.96	0.956	0.949	0.99	0.946	0.948	0.896	0.863	0.967	0.884
zoo-NOFS	0.954	0.952	0.973	0.981	0.908	0.924	0.948	0.952	0.943	0.965	0.954	0.968
zoo-Overall	0.954	0.952	0.954	0.957	0.916	0.979	0.948	0.952	0.943	0.965	0.954	0.968

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 32. MAR Results at 10% Missingness for the ACC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian	0.849	0.843	0.855	0.861	0.861	0.852	0.852	0.846	0.832	0.838	0.829	0.841
Australian-FS	0.849	0.843	0.838	0.841	0.852	0.849	0.829	0.835	0.82	0.82	0.829	0.841
Australian-NOFS	0.852	0.852	0.852	0.858	0.829	0.858	0.852	0.846	0.832	0.838	0.855	0.829
boston	0.893	0.893	0.877	0.858	0.889	0.858	0.854	0.854	0.874	0.858	0.87	0.881
boston-FS	0.893	0.893	0.87	0.881	0.877	0.858	0.854	0.854	0.877	0.877	0.87	0.881
boston-NOFS	0.905	0.885	0.897	0.885	0.877	0.889	0.881	0.87	0.874	0.858	0.897	0.87
churn	0.94	0.935	0.94	0.933	0.943	0.936	0.932	0.928	0.921	0.922	0.944	0.944
churn-FS	0.94	0.942	0.93	0.932	0.94	0.943	0.93	0.926	0.92	0.914	0.944	0.944
churn-NOFS	0.941	0.935	0.943	0.943	0.943	0.936	0.932	0.928	0.921	0.922	0.942	0.933
compas-two-years	0.657	0.657	0.662	0.658	0.656	0.66	0.666	0.661	0.656	0.658	0.664	0.662
compas-two-years-FS	0.657	0.657	0.654	0.659	0.658	0.653	0.649	0.652	0.651	0.649	0.664	0.644
compas-two-years-NOFS	0.66	0.665	0.664	0.663	0.652	0.661	0.666	0.661	0.656	0.658	0.666	0.662
image	0.873	0.864	—	—	0.872	0.872	0.859	0.86	0.854	0.862	0.865	0.871
image-FS	0.851	0.855	—	—	0.857	0.858	0.859	0.859	0.84	0.84	0.869	0.859
image-NOFS	0.873	0.864	—	—	0.867	0.869	0.859	0.86	0.854	0.862	0.865	0.871
page-blocks	0.971	0.971	0.972	0.972	0.972	0.972	0.966	0.968	0.965	0.963	0.968	0.972
page-blocks-FS	0.972	0.971	0.971	0.973	0.973	0.972	0.97	0.966	0.961	0.958	0.968	0.973
page-blocks-NOFS	0.971	0.972	0.972	0.973	0.971	0.972	0.966	0.968	0.965	0.963	0.968	0.972
parkinsons	0.878	0.878	0.898	0.878	0.867	0.878	0.908	0.898	0.908	0.857	0.878	0.878
parkinsons-FS	0.878	0.878	0.878	0.878	0.878	0.878	0.878	0.878	0.867	0.857	0.878	0.878
parkinsons-NOFS	0.898	0.867	0.888	0.898	0.898	0.888	0.908	0.898	0.908	0.857	0.898	0.898
segment	0.993	0.994	0.996	0.997	0.994	0.994	0.994	0.996	0.932	0.932	0.994	0.994
segment-FS	0.994	0.994	0.997	0.996	0.995	0.994	0.991	0.989	0.923	0.916	0.994	0.994
segment-NOFS	0.993	0.994	0.997	0.998	0.995	0.995	0.994	0.996	0.932	0.932	0.996	0.995
stock	0.96	0.96	0.973	0.973	0.956	0.964	0.945	0.941	0.924	0.926	0.962	0.962
stock-FS	0.952	0.935	0.975	0.975	0.949	0.962	0.964	0.947	0.901	0.901	0.956	0.956
stock-NOFS	0.96	0.96	0.964	0.966	0.962	0.971	0.945	0.941	0.924	0.926	0.962	0.962
zoo	0.961	1.0	1.0	1.0	0.98	1.0	1.0	0.98	0.961	1.0	0.941	0.941
zoo-FS	1.0	1.0	1.0	1.0	0.843	1.0	1.0	1.0	1.0	1.0	0.941	0.941
zoo-NOFS	0.961	1.0	1.0	1.0	1.0	0.98	1.0	0.98	0.961	1.0	0.941	1.0

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 33. MAR Results at 25% Missingness for the ACC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian	0.832	0.838	0.841	0.835	0.82	0.835	0.8	0.791	0.814	0.812	0.835	0.843
Australian-FS	0.829	0.838	0.841	0.835	0.826	0.826	0.8	0.791	0.814	0.812	0.835	0.843
Australian-NOFS	0.832	0.82	0.835	0.823	0.812	0.812	0.809	0.814	0.809	0.812	0.832	0.826
boston	0.862	0.858	0.85	0.87	0.885	0.858	0.814	0.818	0.854	0.834	0.85	0.834
boston-FS	0.862	0.862	0.85	0.866	0.866	0.854	0.802	0.802	0.834	0.834	0.842	0.842
boston-NOFS	0.862	0.858	0.862	0.858	0.85	0.85	0.814	0.818	0.854	0.858	0.85	0.834
churn	0.921	0.915	0.926	0.922	0.921	0.918	0.898	0.892	0.9	0.9	0.918	0.918
churn-FS	0.919	0.922	0.921	0.92	0.918	0.921	0.901	0.887	0.9	0.901	0.918	0.918
churn-NOFS	0.921	0.915	0.928	0.924	0.92	0.913	0.898	0.892	0.903	0.9	0.918	0.918
compas-two-years	0.656	0.656	0.653	0.648	0.64	0.646	0.641	0.656	0.649	0.652	0.649	0.653
compas-two-years-FS	0.656	0.649	0.653	0.656	0.639	0.649	0.656	0.656	0.649	0.652	0.649	0.647
compas-two-years-NOFS	0.659	0.656	0.649	0.659	0.641	0.654	0.641	0.654	0.647	0.65	0.652	0.653
image	0.861	0.866	—	—	0.869	0.862	0.859	0.86	0.861	0.852	0.873	0.869
image-FS	0.838	0.838	—	—	0.819	0.846	0.843	0.841	0.843	0.842	0.867	0.867
image-NOFS	0.861	0.866	—	—	0.866	0.868	0.859	0.86	0.861	0.852	0.873	0.869
page-blocks	0.962	0.962	0.966	0.964	0.961	0.959	0.945	0.946	0.947	0.953	0.959	0.958
page-blocks-FS	0.962	0.962	0.966	0.965	0.961	0.962	0.946	0.942	0.947	0.949	0.963	0.959
page-blocks-NOFS	0.962	0.961	0.965	0.965	0.96	0.959	0.945	0.946	0.947	0.953	0.959	0.958
parkinsons	0.867	0.878	0.867	0.867	0.888	0.847	0.857	0.857	0.888	0.878	0.888	0.867
parkinsons-FS	0.867	0.857	0.867	0.867	0.867	0.837	0.837	0.837	0.867	0.837	0.847	0.867
parkinsons-NOFS	0.867	0.878	0.888	0.898	0.888	0.888	0.857	0.857	0.888	0.878	0.888	0.888
segment	0.991	0.995	0.997	0.997	0.992	0.99	0.973	0.979	0.941	0.938	0.994	0.993
segment-FS	0.99	0.991	0.997	0.997	0.991	0.99	0.958	0.945	0.897	0.939	0.993	0.993
segment-NOFS	0.991	0.995	0.997	0.997	0.989	0.992	0.973	0.979	0.941	0.938	0.994	0.993
stock	0.926	0.918	0.941	0.939	0.914	0.916	0.813	0.859	0.846	0.861	0.914	0.918
stock-FS	0.926	0.901	0.931	0.931	0.92	0.916	0.794	0.844	0.8	0.825	0.918	0.918
stock-NOFS	0.926	0.918	0.945	0.939	0.916	0.918	0.813	0.859	0.846	0.861	0.914	0.922
zoo	0.961	0.961	0.922	1.0	1.0	1.0	0.941	0.98	0.922	0.98	0.922	0.863
zoo-FS	0.961	0.941	1.0	0.961	0.961	0.941	0.941	0.961	0.922	0.941	0.843	0.863
zoo-NOFS	0.961	0.961	0.922	1.0	0.98	1.0	0.98	0.98	0.98	0.98	0.922	0.98

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

Table 34. MAR Results at 50% Missingness for the ACC Metric

Dataset	MM	BI+MM	MF	BI+MF	GAIN	BI+GAIN	SOFT	BI+SOFT	PPCA	BI+PPCA	DAE	BI+DAE
Australian	0.791	0.823	0.832	0.812	0.794	0.783	0.748	0.791	0.774	0.762	0.814	0.82
Australian-FS	0.791	0.823	0.817	0.809	0.803	0.783	0.748	0.791	0.762	0.762	0.814	0.82
Australian-NOFS	0.806	0.823	0.843	0.812	0.771	0.812	0.722	0.762	0.774	0.774	0.806	0.803
boston	0.798	0.818	0.838	0.818	0.798	0.81	0.763	0.763	0.787	0.791	0.787	0.791
boston-FS	0.798	0.818	0.806	0.822	0.806	0.798	0.767	0.739	0.791	0.791	0.787	0.791
boston-NOFS	0.826	0.834	0.822	0.826	0.822	0.822	0.763	0.763	0.787	0.802	0.794	0.798
churn	0.894	0.889	0.896	0.894	0.888	0.882	0.865	0.866	0.887	0.883	0.89	0.888
churn-FS	0.892	0.89	0.893	0.899	0.89	0.884	0.87	0.869	0.887	0.889	0.89	0.888
churn-NOFS	0.894	0.889	0.892	0.891	0.888	0.882	0.865	0.866	0.888	0.883	0.894	0.883
compas-two-years	0.621	0.632	0.629	0.628	0.608	0.627	0.607	0.614	0.62	0.629	0.621	0.632
compas-two-years-FS	0.611	0.623	0.623	0.624	0.598	0.62	0.607	0.618	0.619	0.622	0.626	0.627
compas-two-years-NOFS	0.621	0.632	0.629	0.626	0.618	0.631	0.604	0.614	0.62	0.629	0.621	0.632
image	0.84	0.839	—	—	0.845	0.849	0.809	0.817	0.854	0.847	0.856	0.853
image-FS	0.813	0.815	—	—	0.811	0.817	0.804	0.804	0.846	0.846	0.831	0.837
image-NOFS	0.84	0.839	—	—	0.846	0.846	0.809	0.817	0.854	0.847	0.856	0.853
page-blocks	0.948	0.949	0.955	0.952	0.949	0.947	0.921	0.92	0.933	0.934	0.949	0.949
page-blocks-FS	0.948	0.949	0.954	0.954	0.95	0.948	0.916	0.921	0.932	0.93	0.949	0.949
page-blocks-NOFS	0.948	0.95	0.953	0.953	0.95	0.949	0.921	0.92	0.933	0.934	0.949	0.949
parkinsons	0.837	0.857	0.857	0.857	0.847	0.837	0.827	0.847	0.878	0.867	0.867	0.857
parkinsons-FS	0.796	0.857	0.816	0.867	0.847	0.837	0.827	0.816	0.847	0.837	0.837	0.837
parkinsons-NOFS	0.837	0.857	0.878	0.867	0.847	0.837	0.806	0.847	0.878	0.867	0.867	0.857
segment	0.982	0.977	0.99	0.99	0.961	0.983	0.91	0.911	0.882	0.898	0.967	0.969
segment-FS	0.983	0.977	0.989	0.99	0.983	0.98	0.899	0.915	0.861	0.872	0.967	0.969
segment-NOFS	0.982	0.977	0.99	0.988	0.983	0.979	0.91	0.911	0.882	0.898	0.971	0.969
stock	0.901	0.891	0.922	0.918	0.863	0.882	0.811	0.827	0.834	0.838	0.882	0.874
stock-FS	0.901	0.857	0.924	0.918	0.897	0.857	0.815	0.798	0.695	0.743	0.882	0.859
stock-NOFS	0.901	0.891	0.924	0.924	0.876	0.884	0.811	0.827	0.834	0.838	0.882	0.874
zoo	0.902	0.902	0.941	0.941	0.863	0.941	0.902	0.922	0.882	0.922	0.922	0.922
zoo-FS	0.902	0.882	0.961	0.922	0.902	0.961	0.882	0.922	0.863	0.843	0.941	0.882
zoo-NOFS	0.902	0.902	0.98	0.961	0.843	0.863	0.902	0.922	0.863	0.922	0.922	0.922

-FS: denotes the result with enforced feature selection, -NOFS: denotes the results without feature selection, -Overall: denotes the results with and without feature selection.

C.7 Imputation Accuracy Results

In this section, we include the quantitative results of the experiments regarding the imputation quality. We split this section, into two subsections for each missingness mechanism. We report results for the train and test set using the default configurations for each imputation method (for details see Section C.3).

C.7.1 MCAR. Tables 35 and 36 present the results at 10% missingness for R2 and accuracy scores, respectively. Tables 37 and 38 show the imputation R2 and accuracy score at 25% missingness, while Tables 39 and 40 show these at 50% missingness.

Table 35. Imputation R2-score for MCAR 10% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.073	0.0	0.171	0.08	0.0	0.0
Australian-Train	0.033	0.0	0.174	0.118	0.0	0.0
boston-Test	0.246	0.0	0.636	0.491	0.0	0.302
boston-Train	0.224	0.252	0.651	0.521	0.0	0.336
churn-Test	0.292	0.0	0.474	0.33	0.0	0.348
churn-Train	0.259	0.429	0.46	0.322	0.0	0.337
compas-two-years-Test	0.06	0.0	0.235	0.209	0.0	0.0
compas-two-years-Train	0.058	0.139	0.316	0.225	0.0	0.0
image-Test	0.711	0.0	—	0.798	0.0	0.827
image-Train	0.46	0.938	—	0.775	0.0	0.829
page-blocks-Test	0.299	0.0	0.815	0.15	0.0	0.4
page-blocks-Train	0.319	0.309	0.796	0.168	0.0	0.431
parkinsons-Test	0.194	0.04	0.697	0.578	0.0	0.489
parkinsons-Train	0.221	0.592	0.815	0.6	0.0	0.628
segment-Test	0.364	0.0	0.665	0.483	0.053	0.48
segment-Train	0.433	0.41	0.729	0.539	0.053	0.517
stock-Test	0.514	0.0	0.931	0.699	0.0	0.546
stock-Train	0.559	0.656	0.951	0.701	0.0	0.588
zoo-Test	0.0	0.0	0.687	0.0	0.0	0.0
zoo-Train	0.0	0.0	0.0	0.0	0.0	0.0

Table 36. Imputation Accuracy Score for MCAR 10% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.653	0.594	0.745	0.682	0.659	0.36
Australian-Train	0.679	0.743	0.825	0.635	0.605	0.364
boston-Test	0.955	0.818	1.0	1.0	1.0	0.5
boston-Train	0.773	0.364	0.955	0.955	0.955	0.682
churn-Test	0.77	0.642	0.795	0.742	0.689	0.708
churn-Train	0.746	0.747	0.769	0.716	0.673	0.715
compas-two-years-Test	0.858	0.586	0.955	0.766	0.682	0.835
compas-two-years-Train	0.832	0.94	0.953	0.75	0.668	0.821
zoo-Test	0.837	0.572	0.861	0.806	0.689	0.73
zoo-Train	0.774	0.874	0.909	0.863	0.734	0.743

Table 37. Imputation R2-score for MCAR 25% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.043	0.0	0.125	0.097	0.0	0.01
Australian-Train	0.034	0.0	0.101	0.069	0.0	0.001
boston-Test	0.14	0.0	0.585	0.413	0.0	0.201
boston-Train	0.047	0.159	0.604	0.382	0.0	0.175
churn-Test	0.237	0.0	0.378	0.211	0.0	0.283
churn-Train	0.243	0.376	0.391	0.213	0.0	0.286
compas-two-years-Test	0.003	0.0	0.152	0.129	0.0	0.0
compas-two-years-Train	0.007	0.031	0.162	0.153	0.0	0.0
image-Test	0.256	0.0	—	0.712	0.0	0.757
image-Train	0.181	0.88	—	0.644	0.0	0.765
page-blocks-Test	0.183	0.0	0.614	0.228	0.0	0.279
page-blocks-Train	0.227	0.292	0.768	0.236	0.0	0.399
parkinsons-Test	0.01	0.005	0.723	0.561	0.0	0.466
parkinsons-Train	0.067	0.54	0.764	0.353	0.0	0.497
segment-Test	0.256	0.0	0.696	0.452	0.053	0.473
segment-Train	0.243	0.412	0.677	0.446	0.053	0.46
stock-Test	0.423	0.0	0.902	0.562	0.0	0.504
stock-Train	0.443	0.496	0.901	0.554	0.0	0.516
zoo-Test	0.059	0.0	0.0	0.107	0.0	0.0
zoo-Train	0.0	0.0	0.421	0.407	0.0	0.0

Table 38. Imputation Accuracy Score for MCAR 25% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.637	0.593	0.754	0.632	0.624	0.622
Australian-Train	0.646	0.635	0.745	0.623	0.616	0.592
boston-Test	0.857	0.814	0.9	0.9	0.9	0.457
boston-Train	0.869	0.443	0.951	0.951	0.951	0.541
churn-Test	0.719	0.617	0.764	0.71	0.703	0.679
churn-Train	0.721	0.704	0.776	0.712	0.706	0.678
compas-two-years-Test	0.831	0.55	0.925	0.683	0.682	0.8
compas-two-years-Train	0.831	0.758	0.922	0.687	0.686	0.814
zoo-Test	0.824	0.555	0.897	0.795	0.661	0.564
zoo-Train	0.759	0.799	0.893	0.842	0.741	0.582

Table 39. Imputation R2-score for MCAR 50% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.02	0.0	0.046	0.039	0.0	0.013
Australian-Train	0.0	0.004	0.032	0.027	0.0	0.01
boston-Test	0.079	0.0	0.403	0.213	0.0	0.192
boston-Train	0.089	0.047	0.385	0.176	0.0	0.186
churn-Test	0.059	0.0	0.25	0.077	0.0	0.02
churn-Train	0.057	0.191	0.243	0.075	0.0	0.019
compas-two-years-Test	0.002	0.0	0.083	0.052	0.0	0.042
compas-two-years-Train	0.008	0.059	0.075	0.051	0.0	0.04
image-Test	0.009	0.0	—	0.455	0.0	0.574
image-Train	0.011	0.24	—	0.393	0.0	0.573
page-blocks-Test	0.086	0.0	0.411	0.171	0.0	0.154
page-blocks-Train	0.174	0.113	0.605	0.154	0.0	0.21
parkinsons-Test	0.0	0.0	0.566	0.305	0.0	0.323
parkinsons-Train	0.003	0.228	0.595	0.167	0.0	0.407
segment-Test	0.189	0.0	0.621	0.22	0.053	0.301
segment-Train	0.187	0.188	0.631	0.21	0.053	0.309
stock-Test	0.194	0.0	0.714	0.29	0.0	0.38
stock-Train	0.164	0.224	0.751	0.289	0.0	0.401
zoo-Test	0.0	0.089	0.0	0.0	0.0	0.0
zoo-Train	0.0	0.0	0.0	0.002	0.0	0.0

Table 40. Imputation Accuracy for MCAR 50% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.622	0.546	0.723	0.639	0.639	0.599
Australian-Train	0.617	0.553	0.71	0.629	0.629	0.624
boston-Test	0.892	0.677	0.931	0.931	0.931	0.462
boston-Train	0.905	0.448	0.931	0.931	0.931	0.44
churn-Test	0.601	0.599	0.749	0.711	0.711	0.603
churn-Train	0.596	0.638	0.742	0.707	0.707	0.601
compas-two-years-Test	0.658	0.501	0.853	0.681	0.681	0.782
compas-two-years-Train	0.646	0.562	0.845	0.679	0.679	0.777
zoo-Test	0.692	0.511	0.77	0.696	0.689	0.567
zoo-Train	0.699	0.612	0.797	0.687	0.685	0.522

C.7.2 *MAR*. Tables 41 and 42 present the results at 10% missingness for R2 and Accuracy scores, respectively. Tables 43 and 44 show the R2-score and accuracy score at 25% missingness, while Tables 45 and 46 show these at 50% missingness.

Table 41. Imputation R2-score for MAR 10% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.004	0.0	0.131	0.064	0.0	0.0
Australian-Train	0.072	0.0	0.246	0.142	0.0	0.0
boston-Test	0.051	0.0	0.554	0.445	0.0	0.171
boston-Train	0.035	0.199	0.717	0.382	0.0	0.278
churn-Test	0.226	0.0	0.493	0.324	0.0	0.344
churn-Train	0.226	0.415	0.47	0.321	0.0	0.336
compas-two-years-Test	0.0	0.0	0.219	0.193	0.0	0.0
compas-two-years-Train	0.0	0.142	0.247	0.207	0.0	0.0
image-Test	0.312	0.0	—	0.751	0.0	0.816
image-Train	0.03	0.943	—	0.706	0.0	0.816
page-blocks-Test	0.031	0.0	0.698	0.133	0.0	0.212
page-blocks-Train	0.084	0.286	0.684	0.134	0.0	0.22
parkinsons-Test	0.158	0.005	0.726	0.632	0.0	0.536
parkinsons-Train	0.065	0.633	0.724	0.491	0.0	0.564
segment-Test	0.281	0.0	0.638	0.476	0.053	0.418
segment-Train	0.268	0.437	0.722	0.447	0.053	0.395
stock-Test	0.476	0.0	0.925	0.668	0.0	0.479
stock-Train	0.452	0.573	0.927	0.608	0.0	0.481
zoo-Test	0.0	0.476	0.929	0.381	0.0	0.0
zoo-Train	0.0	0.0	0.48	0.681	0.0	0.0

Table 42. Imputation Accuracy Score for MAR 10% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.61	0.625	0.786	0.703	0.65	0.54
Australian-Train	0.612	0.695	0.76	0.622	0.594	0.518
boston-Test	1.0	0.941	0.941	1.0	1.0	0.471
boston-Train	0.941	0.529	0.941	0.941	0.941	0.353
churn-Test	0.741	0.641	0.791	0.756	0.719	0.695
churn-Train	0.755	0.778	0.809	0.758	0.727	0.731
compas-two-years-Test	0.854	0.608	0.955	0.73	0.704	0.808
compas-two-years-Train	0.815	0.914	0.941	0.724	0.707	0.798
zoo-Test	0.789	0.568	0.888	0.801	0.722	0.471
zoo-Train	0.823	0.805	0.929	0.823	0.72	0.477

Table 43. Imputation R2-score for MAR 25% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.007	0.0	0.123	0.053	0.0	0.01
Australian-Train	0.007	0.0	0.142	0.076	0.0	0.002
boston-Test	0.041	0.0	0.641	0.288	0.0	0.152
boston-Train	0.01	0.057	0.54	0.301	0.0	0.168
churn-Test	0.118	0.0	0.405	0.19	0.0	0.289
churn-Train	0.115	0.38	0.411	0.19	0.0	0.296
compas-two-years-Test	0.0	0.0	0.169	0.121	0.0	0.0
compas-two-years-Train	0.0	0.164	0.157	0.102	0.0	0.0
image-Test	0.0	0.0	—	0.621	0.0	0.732
image-Train	0.0	0.867	—	0.54	0.0	0.73
page-blocks-Test	0.026	0.0	0.628	0.174	0.0	0.159
page-blocks-Train	0.013	0.11	0.678	0.162	0.0	0.16
parkinsons-Test	0.016	0.0	0.66	0.536	0.0	0.501
parkinsons-Train	0.005	0.513	0.671	0.301	0.0	0.482
segment-Test	0.0	0.0	0.686	0.346	0.053	0.309
segment-Train	0.0	0.298	0.675	0.344	0.053	0.313
stock-Test	0.028	0.0	0.809	0.346	0.0	0.366
stock-Train	0.021	0.326	0.841	0.346	0.0	0.388
zoo-Test	0.0	0.0	0.358	0.564	0.0	0.003
zoo-Train	0.0	0.0	0.261	0.123	0.0	0.0

Table 44. Imputation Accuracy Score for MAR 25% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.675	0.574	0.756	0.665	0.66	0.558
Australian-Train	0.624	0.646	0.733	0.642	0.64	0.522
boston-Test	0.889	0.889	0.921	0.921	0.921	0.46
boston-Train	0.79	0.548	0.919	0.919	0.919	0.468
churn-Test	0.752	0.615	0.781	0.719	0.704	0.684
churn-Train	0.746	0.696	0.774	0.708	0.699	0.676
compas-two-years-Test	0.701	0.542	0.904	0.663	0.66	0.795
compas-two-years-Train	0.695	0.778	0.903	0.655	0.652	0.807
zoo-Test	0.766	0.512	0.889	0.759	0.678	0.561
zoo-Train	0.755	0.763	0.909	0.754	0.685	0.546

Table 45. Imputation R2-score for MAR 50% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.0	0.0	0.029	0.027	0.0	0.003
Australian-Train	0.0	0.015	0.034	0.036	0.0	0.001
boston-Test	0.0	0.0	0.468	0.093	0.0	0.104
boston-Train	0.0	0.021	0.484	0.096	0.0	0.162
churn-Test	0.019	0.0	0.26	0.064	0.0	0.051
churn-Train	0.017	0.14	0.258	0.064	0.0	0.048
compas-two-years-Test	0.0	0.0	0.115	0.072	0.0	0.027
compas-two-years-Train	0.0	0.029	0.104	0.067	0.0	0.032
image-Test	0.0	0.0	—	0.364	0.0	0.515
image-Train	0.0	0.176	—	0.292	0.0	0.514
page-blocks-Test	0.0	0.0	0.437	0.076	0.0	0.116
page-blocks-Train	0.0	0.103	0.449	0.079	0.0	0.11
parkinsons-Test	0.018	0.0	0.6	0.228	0.0	0.304
parkinsons-Train	0.017	0.19	0.608	0.186	0.0	0.342
segment-Test	0.0	0.0	0.59	0.144	0.053	0.098
segment-Train	0.0	0.069	0.593	0.135	0.053	0.097
stock-Test	0.0	0.0	0.751	0.159	0.0	0.118
stock-Train	0.0	0.113	0.725	0.136	0.0	0.098
zoo-Test	0.0	0.032	0.191	0.22	0.0	0.102
zoo-Train	0.0	0.0	0.127	0.0	0.0	0.0

Table 46. Imputation Accuracy Score for MAR 50% Missingness

Dataset	GAIN	SOFT	MF	DAE	MM	PPCA
Australian-Test	0.605	0.541	0.724	0.649	0.649	0.568
Australian-Train	0.621	0.532	0.72	0.625	0.625	0.582
boston-Test	0.923	0.65	0.93	0.937	0.937	0.469
boston-Train	0.928	0.456	0.944	0.944	0.944	0.368
churn-Test	0.725	0.589	0.745	0.697	0.697	0.595
churn-Train	0.732	0.645	0.753	0.704	0.704	0.576
compas-two-years-Test	0.634	0.504	0.853	0.691	0.69	0.67
compas-two-years-Train	0.629	0.581	0.842	0.69	0.689	0.671
zoo-Test	0.6	0.535	0.764	0.625	0.625	0.599
zoo-Train	0.599	0.585	0.846	0.688	0.691	0.613

REFERENCES

- [1] 2023. (unpublished).
- [2] Deepak Adhikari, Wei Jiang, Jinyu Zhan, Zhiyuan He, Danda B. Rawat, Uwe Aickelin, and Hadi A. Khorshidi. 2022. A comprehensive survey on imputation of missing data in internet of things. *ACM Comput. Surv.* 55, 7, Article 133 (Dec. 2022), 38 pages. <https://doi.org/10.1145/3533381>
- [3] Ahmed Alaa and Mihaela van der Schaar. 2018. AutoPrognosis: Automated clinical prognostic modeling via bayesian optimization with structured kernel learning. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer Dy and Andreas Krause (Eds.). PMLR, 139–148. <https://proceedings.mlr.press/v80/alaa18b.html>
- [4] Mustafa Alabadla, Fatimah Sidi, Iskandar Ishak, Hamidah Ibrahim, Lilly Suriani Affendey, Zafienas Che Ani, Marzanah A. Jabar, Umar Ali Bukar, Navin Kumar Devaraj, Ahmad Sobri Muda, Anas Tharek, Noritah Omar, and M. Izham Mohd Jaya. 2022. Systematic review of using machine learning in imputing missing values. *IEEE Access* 10 (2022), 44483–44502. <https://doi.org/10.1109/ACCESS.2022.3160841>

- [5] Edesio Alcobaça, Felipe Siqueira, Adriano Rivolli, Luís P. F. Garcia, Jefferson T. Oliva, and André C. P. L. F. de Carvalho. 2020. MFE: Towards reproducible meta-feature extraction. *J. Mach. Learn. Res.* 21, 111 (2020), 1–5. <http://jmlr.org/papers/v21/19-348.html>
- [6] Rebecca R. Andridge and Roderick J. A. Little. 2010. A review of hot deck imputation for survey non-response. *Int. Stat. Rev.* 78, 1 (Apr. 2010), 40–64.
- [7] Yoav Benjamini and Yoel Hochberg. 1995. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *J. Roy. Stat. Soc.: Ser. B (Methodol.)* 57, 1 (1995), 289–300. <https://doi.org/10.1111/j.2517-6161.1995.tb02031.x>
- [8] Dimitris Bertsimas, Colin Pawlowski, and Ying Daisy Zhuo. 2018. From predictive methods to missing data imputation: An optimization approach. *J. Mach. Learn. Res.* 18, 196 (2018), 1–39.
- [9] Felix Biessmann, Tammo Rukat, Philipp Schmidt, Prathik Naidu, Sebastian Schelter, Andrey Taptunov, Dustin Lange, and David Salinas. 2019. DataWig: Missing value imputation for tables. *J. Mach. Learn. Res.* 20, 175 (2019), 1–6.
- [10] Christopher Bishop. 1998. Bayesian PCA. In *Advances in Neural Information Processing Systems*, M. Kearns, S.olla, and D. Cohn (Eds.), Vol. 11. MIT Press.
- [11] Ramiro Daniel Camino, Christian A. Hammerschmidt, and Radu State. 2019. Improving missing data imputation with deep generative models. arXiv:1902.10666. Retrieved from <http://arxiv.org/abs/1902.10666>
- [12] James R. Carpenter and Melanie Smuk. 2021. Missing data: A statistical framework for practice. *Biometr. J.* 63, 5 (2021), 915–947. <https://doi.org/10.1002/bimj.202000196>
- [13] Tlameo Emmanuel, Thabiso Maupong, Dimane Mpoeleng, Thabo Semong, Banyatsang Mphago, and Oteng Tabona. 2021. A survey on missing data in machine learning. *J. Big Data* 8, 1 (27 Oct. 2021), 140. <https://doi.org/10.1186/s40537-021-00516-9>
- [14] Nick Erickson, Jonas Mueller, Alexander Shirkov, Hang Zhang, Pedro Larroy, Mu Li, and Alexander Smola. 2020. AutoGluon-Tabular: Robust and accurate AutoML for structured data. arXiv:2003.06505. Retrieved from <https://arxiv.org/abs/2003.06505>
- [15] Shahla Faisal and Gerhard Tutz. 2021. Multiple imputation using nearest neighbor methods. *Inf. Sci.* 570 (2021), 500–516. <https://doi.org/10.1016/j.ins.2021.04.009>
- [16] Matthias Feurer, Katharina Eggensperger, Stefan Falkner, Marius Lindauer, and Frank Hutter. 2020. Auto-Sklearn 2.0: Hands-free AutoML via meta-learning.
- [17] Vincent Fortuin, Dmitry Baranchuk, Gunnar Rätsch, and Stephan Mandt. 2020. GP-VAE: Deep probabilistic time series imputation. arXiv:1907.04155 [stat.ML]. Retrieved from <https://arxiv.org/abs/1907.04155>
- [18] João Gama and Pavel Brazdil. 1995. Characterization of classification algorithms. 189–200. https://doi.org/10.1007/3-540-60428-6_16
- [19] Unai Garcíarena, Roberto Santana, and Alexander Mendiburu. 2017. Evolving imputation strategies for missing data in classification problems with TPOT. arXiv:1706.01120. Retrieved from <http://arxiv.org/abs/1706.01120>
- [20] Pieter Gijsbers and Joaquin Vanschoren. 2020. GAMA: A general automated machine learning assistant. arXiv:2007.04911. Retrieved from <https://arxiv.org/abs/2007.04911>
- [21] Lovedeep Gondara and Ke Wang. 2018. MIDA: Multiple Imputation Using Denoising Autoencoders. 260–272. https://doi.org/10.1007/978-3-319-93040-4_21
- [22] H2O.ai. 2022. DriverlessAI. Retrieved from <https://www.h2o.ai/products/h2o-driverless-ai/>
- [23] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. 2020. Array programming with NumPy. *Nature* 585, 7825 (Sept. 2020), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- [24] Md. Kamrul Hasan, Md. Ashrafal Alam, Shidhartho Roy, Aishwariya Dutta, Md. Tasnim Jawad, and Sunanda Das. 2021. Missing value imputation affects the performance of machine learning: A review and analysis of the literature (2010–2021). *Inf. Med. Unlock.* 27 (2021), 100799. <https://doi.org/10.1016/j.imu.2021.100799>
- [25] Harshad Hegde, Neel Shimpi, Alokshagar Panny, Ingrid Glurich, Pamela Christie, and Amit Acharya. 2019. MICE vs PPCA: Missing data imputation in healthcare. *Inf. Med. Unlock.* 17 (2019), 100275. <https://doi.org/10.1016/j.imu.2019.100275>
- [26] Steffen Herbold. 2020. Autorank: A Python package for automated ranking of classifiers. *J. of Open Source Softw.* 5, 48 (2020), 2173. <https://doi.org/10.21105/joss.02173>
- [27] James Honaker, Gary King, and Matthew Blackwell. 2011. Amelia II: A program for missing data. *J. Stat. Softw.* 45, 7 (2011), 1–47. <https://doi.org/10.18637/jss.v045.i07>
- [28] Md Hamidul Huque, John B. Carlin, Julie A. Simpson, and Katherine J. Lee. 2018. A comparison of multiple imputation methods for missing data in longitudinal studies. *BMC Med. Res. Methodol.* 18, 1 (12 Dec. 2018), 168. <https://doi.org/10.1186/s12874-018-0615-6>

- [29] Anil Jadhav, Dhanya Pramod, and Krishnan Ramanathan. 2019. Comparison of performance of data imputation methods for numeric dataset. *Appl. Artif. Intell.* 33 (07 2019), 1–21. <https://doi.org/10.1080/08839514.2019.1637138>
- [30] Sebastian Jäger, Arndt Allhorn, and Felix Bießmann. 2021. A benchmark for data imputation methods. *Front. Big Data* 4 (2021). <https://doi.org/10.3389/fdata.2021.693674>
- [31] Jintao Ke, Shuaichao Zhang, Hai Yang, and Xiqun (Michael) Chen. 2019. PCA-based missing information imputation for real-time crash likelihood prediction under imbalanced data. *Transportmetr. A: Transp. Sci.* 15, 2 (2019), 872–895. <https://doi.org/10.1080/23249935.2018.1542414> <https://doi.org/10.1080/23249935.2018.1542414>
- [32] Diederik P. Kingma and Max Welling. 2013. Auto-encoding variational Bayes. [arXiv:1312.6114](https://arxiv.org/abs/1312.6114). Retrieved from <https://arxiv.org/abs/1312.6114>
- [33] Alexander Kowarik and Matthias Templ. 2016. Imputation with the R package VIM. *J. Stat. Softw.* 74, 7 (2016), 1–16. <https://doi.org/10.18637/jss.v074.i07>
- [34] Gayaneh Kyureghian, Oral Capps, and Rodolfo M. Nayga. 2011. *A Missing Variable Imputation Methodology with an Empirical Application*. 313–337. [https://doi.org/10.1108/S0731-9053\(2011\)000027A015](https://doi.org/10.1108/S0731-9053(2011)000027A015)
- [35] Ranjit Lall and Thomas Robinson. 2022. The MIDAS touch: Accurate and scalable missing-data imputation with deep learning. *Polit. Anal.* 30, 2 (2022), 179–196. <https://doi.org/10.1017/pan.2020.49>
- [36] Trang T. Le, Weixuan Fu, and Jason H. Moore. 2020. Scaling tree-based automated machine learning to biomedical big data with a feature set selector. *Bioinformatics* 36, 1 (2020), 250–256.
- [37] Dan Li, Jitender Deogun, William Spaulding, and Bill Shuart. 2004. Towards missing data imputation: A study of fuzzy k-means clustering method. In *Rough Sets and Current Trends in Computing*, Shusaku Tsumoto, Roman Ślowiński, Jan Komorowski, and Jerzy W. Grzymała-Busse (Eds.). Springer, Berlin, 573–579.
- [38] Peng Li, Xi Rao, Jennifer Blase, Yue Zhang, Xu Chu, and Ce Zhang. 2019. CleanML: A benchmark for joint data cleaning and machine learning [experiments and analysis]. [arXiv:1904.09483](https://arxiv.org/abs/1904.09483). Retrieved from <http://arxiv.org/abs/1904.09483>
- [39] Yuebiao Li, Zhiheng Li, and Li Li. 2014. Missing traffic data: Comparison of imputation methods. *Intell. Transp. Syst. IET* 8 (02 2014), 51–57. <https://doi.org/10.1049/iet-its.2013.0052>
- [40] R. J. A. Little and D. B. Rubin. 2002. *Statistical Analysis with Missing Data*. Wiley.
- [41] Haw-minn Lu, Giancarlo Perrone, and José Unpingco. 2020. Multiple imputation with denoising autoencoder using metamorphic truth and imputation feedback. [arXiv:2002.08338](https://arxiv.org/abs/2002.08338). Retrieved from <https://arxiv.org/abs/2002.08338>
- [42] Ms. R. Malarvizhi. 2012. KNN classifier performs better than k-means clustering in missing value imputation. *IOSR J. Comput. Eng.* 6 (2012), 12–15.
- [43] Behrooz Mamandipoor, Mahshid Majd, Monica Moz, and Venet Osmani. 2019. Blood lactate concentration prediction in critical care patients: Handling missing values. [arXiv:1910.01473](https://arxiv.org/abs/1910.01473). Retrieved from <http://arxiv.org/abs/1910.01473>
- [44] Rahul Mazumder, Trevor Hastie, and Robert Tibshirani. 2010. Spectral regularization algorithms for learning large incomplete matrices. *J. Mach. Learn. Res.* 11, 80 (2010), 2287–2322. <http://jmlr.org/papers/v11/mazumder10a.html>
- [45] John McCoy, Steve Kroon, and Lidia Auret. 2018. Variational autoencoders for missing data imputation with application to a simulated milling circuit. *IFAC-PapersOnLine* 51 (01 2018), 141–146. <https://doi.org/10.1016/j.ifacol.2018.09.406>
- [46] Karthika Mohan and Judea Pearl. 2021. Graphical models for processing missing data. *J. Am. Stat. Assoc.* 116, 534 (2021), 1023–1037. <https://doi.org/10.1080/01621459.2021.1874961>
- [47] Carol M. Musil, Camille B. Warner, Piyanee Klainin Yobas, and Susan L. Jones. 2002. A comparison of imputation techniques for handling missing data. *West. J. Nurs. Res.* 24, 7 (2002), 815–829. <https://doi.org/10.1177/019394502762477004>
- [48] Boris Muzellec, Julie Josse, Claire Boyer, and Marco Cuturi. 2020. Missing data imputation using optimal transport. [arXiv:2002.03860](https://arxiv.org/abs/2002.03860) [stat.ML]. Retrieved from <https://arxiv.org/abs/2002.03860>
- [49] Felix Neutatz, Binger Chen, Yazan Alkhatib, Jingwen Ye, and Ziawasch Abedjan. 2022. Data cleaning and AutoML: Would an optimizer choose to clean? *Datenb.-Spektr.* 22, 2 (01 Jul 2022), 121–130. <https://doi.org/10.1007/s13222-022-00413-2>
- [50] Shigeyuki Oba, Masa-aki Sato, and Shin Ishii. 2003. Variational bayes method for mixture of principal component analyzers. *Syst. Comput. Jpn.* 34, 11 (2003), 55–66. <https://doi.org/10.1002/scj.10394>
- [51] Tomasz Orczyk and Piotr Porwik. 2013. Influence of missing data imputation method on the classification accuracy of the medical data. *J. Med. Inf. Technol.* 22 (2013).
- [52] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* 32, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.). Curran Associates, Inc., 8024–8035.
- [53] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine learning in Python. *J. Mach. Learn. Res.* 12 (2011), 2825–2830.

- [54] Ricardo Cardoso Pereira, Miriam Seoane Santos, Pedro Pereira Rodrigues, and Pedro Henriques Abreu. 2020. Reviewing autoencoders for missing data imputation: Technical trends, applications and outcomes. *J. Artif. Intell. Res.* 69 (2020), 1255–1285.
- [55] Alexandre Perez-Lebel, Gael Varoquaux, Marine Le Morvan, Julie Josse, and Jean-Baptiste Poline. 2022. Benchmarking missing-values approaches for predictive models on health databases. *GigaScience* 11 (04 2022). <https://doi.org/10.1093/gigascience/giac013>
- [56] Ben Omega Petrazzini, Hugo Naya, Fernando Lopez-Bello, Gustavo Vazquez, and Lucía Spangenberg. 2021. Evaluation of different approaches for missing data imputation on features associated to genomic data. *BioData Mining* 14, 1 (03 Sep. 2021), 44. <https://doi.org/10.1186/s13040-021-00274-7>
- [57] Jason Poulos and Rafael Valle. 2018. Missing data imputation for supervised learning. *Appl. Artif. Intell.* 32, 2 (2018), 186–196. <https://doi.org/10.1080/08839514.2018.1448143> <https://doi.org/10.1080/08839514.2018.1448143>
- [58] Li Qu, Li Li, Yi Zhang, and Jianming Hu. 2009. PPCA-based missing data imputation for traffic flow volume: A systematic approach. *IEEE Trans. Intell. Transport. Syst.* 10, 3 (2009), 512–522. <https://doi.org/10.1109/TITS.2009.2026312>
- [59] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic data repairs with probabilistic inference. *Proc. VLDB Endow.* 10, 11 (Aug. 2017), 1190–1201. <https://doi.org/10.14778/3137628.3137631>
- [60] Adriano Rivolli, Luis P. F. Garcia, Carlos Soares, Joaquin Vanschoren, and André C. P. L. F. de Carvalho. 2022. Meta-features for meta-learning. *Knowl.-Bas. Syst.* 240 (2022), 108101. <https://doi.org/10.1016/j.knosys.2021.108101>
- [61] Breeshey Roskams-Hieter, Jude Wells, and Sara Wade. 2022. Leveraging variational autoencoders for multiple data imputation. arXiv:2209.15321 [stat.ML]. Retrieved from <https://arxiv.org/abs/2209.15321>
- [62] Donald B. Rubin. 1976. Inference and missing data. *Biometrika* 63, 3 (1976), 581–592.
- [63] Seunghyoung Ryu, Minsoo Kim, and Hongseok Kim. 2020. Denoising autoencoder-based missing value imputation for smart meters. *IEEE Access* PP (02 2020), 1–1. <https://doi.org/10.1109/ACCESS.2020.2976500>
- [64] Reza Shahbazian and Irina Trubitsyna. 2022. DEGAIn: Generative-adversarial-network-based missing data imputation. *Information* 13, 12 (2022). <https://doi.org/10.3390/info13120575>
- [65] Wolfram Stacklies, Henning Redestig, Matthias Scholz, Dirk Walther, and Joachim Selbig. 2007. pcaMethods a bioconductor package providing PCA methods for incomplete data. *Bioinformatics* 23, 9 (03 2007), 1164–1167.
- [66] Daniel J. Stekhoven and Peter Bühlmann. 2011. MissForest–non-parametric missing value imputation for mixed-type data. *Bioinformatics* 28, 1 (10 2011), 112–118. <https://doi.org/10.1093/bioinformatics/btr597>
- [67] Jonathan A. C. Sterne, Ian R. White, John B. Carlin, Michael Spratt, Patrick Royston, Michael G. Kenward, Angela M. Wood, and James R. Carpenter. 2009. Multiple imputation for missing data in epidemiological and clinical research: Potential and pitfalls. *Br. Med. J.* 338 (2009). <https://doi.org/10.1136/bmj.b2393>
- [68] Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. 2012. Auto-WEKA: Automated selection and hyper-parameter optimization of classification algorithms. arXiv:1208.3719. Retrieved from <http://arxiv.org/abs/1208.3719>
- [69] Robert Tibshirani. 1996. Regression shrinkage and selection via the lasso. *J. Roy. Stat. Soc.: Ser. B (Methodol.)* 58, 1 (1996), 267–288. <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>
- [70] Michael E. Tipping and Christopher M. Bishop. 1999. Probabilistic principal component analysis. *J. Roy. Stat. Soc. Ser. B (Stat. Methodol.)* 61, 3 (1999), 611–622. <http://www.jstor.org/stable/2680726>
- [71] Olga Troyanskaya, Michael Cantor, Gavin Sherlock, Pat Brown, Trevor Hastie, Robert Tibshirani, David Botstein, and Russ B. Altman. 2001. Missing value estimation methods for DNA microarrays. *Bioinformatics* 17, 6 (06 2001), 520–525. <https://doi.org/10.1093/bioinformatics/17.6.520>
- [72] Ioannis Tsamardinos and Constantin F. Aliferis. 2003. Towards principled feature selection: Relevancy, filters and wrappers. In *Proceedings of the 9th International Workshop on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. R4)*, Christopher M. Bishop and Brendan J. Frey (Eds.). PMLR, 300–307.
- [73] Ioannis Tsamardinos, Paulos Charonyktakis, Georgios Papoutsoglou, Georgios Borboudakis, Kleanthi Lakiotaki, Jean Claude Zenklusen, Hartmut Juhl, Ekaterini Chatzaki, and Vincenzo Lagani. 2022. Just add data: Automated predictive modeling for knowledge discovery and feature selection. *npj Precis. Oncol.* 6, 1 (16 Jun 2022), 38. <https://doi.org/10.1038/s41698-022-00274-8>
- [74] Ioannis Tsamardinos, Elissavet Greasidou, Michalis Tsagris, and Giorgos Borboudakis. 2017. Bootstrapping the out-of-sample predictions for efficient and accurate cross-validation. arXiv:1708.07180. Retrieved from <http://arxiv.org/abs/1708.07180>
- [75] I. Tsamardinos, V. Lagani, and D. Pappas. 2012. Discovering multiple, equivalent biomarker signatures. In *Proceedings of the 7th Conference of the Hellenic Society for Computational Biology and Bioinformatics (HSCBB '12)*. Heraklion.
- [76] S. van Buuren and C. G. M. Groothuis-Oudshoorn. 1999. *Flexible Multivariate Imputation by MICE*. Vol. (PG/VGZ/99.054). TNO Prevention and Health, Leiden.
- [77] Stef van Buuren and Karin Groothuis-Oudshoorn. 2011. mice: Multivariate imputation by chained equations in R. *J. Stat. Softw.* 45, 3 (2011), 1–67. <https://doi.org/10.18637/jss.v045.i03>

- [78] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo. 2013. OpenML : Networked science in machine learning. *SIGKDD Explor.* 15, 2 (2013), 49–60. <https://doi.org/10.1145/2641190.2641198>
- [79] Akbar K. Waljee, Ashin Mukherjee, Amit G. Singal, Yiwei Zhang, Jeffrey Warren, Ulysses Balis, Jorge Marrero, Ji Zhu, and Peter D. R. Higgins. 2013. Comparison of imputation methods for missing laboratory data in medicine. *BMJ Open* 3, 8 (2013). <https://doi.org/10.1136/bmjopen-2013-002847>
- [80] Akbar K. Waljee, Ashin Mukherjee, Amit G. Singal, Yiwei Zhang, Jeffrey Warren, Ulysses Balis, Jorge Marrero, Ji Zhu, and Peter D. R. Higgins. 2013. Comparison of imputation methods for missing laboratory data in medicine. *BMJ Open* 3, 8 (2013). <https://doi.org/10.1136/bmjopen-2013-002847>
- [81] Katarzyna Woźnica and Przemysław Biecek. 2020. Does imputation matter? Benchmark for predictive models. arXiv:2007.02837 [stat.ML]. Retrieved from <https://arxiv.org/abs/2007.02837>
- [82] Richard Wu, Aoqian Zhang, Ihab Ilyas, and Theodoros Rekatsinas. 2020. Attention-based learning for missing data imputation in HoloClean. In *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze (Eds.), Vol. 2. 307–325.
- [83] Jinsung Yoon, James Jordon, and Mihaela van der Schaar. 2018. GAIN: Missing data imputation using generative adversarial nets. arXiv:1806.02920. Retrieved from <https://arxiv.org/abs/1806.02920>
- [84] Shichao Zhang. 2012. Nearest neighbor selection for iteratively kNN imputation. *J. Syst. Softw.* 85, 11 (2012), 2541–2552. <https://doi.org/10.1016/j.jss.2012.05.073>
- [85] Xinmeng Zhang, Chao Yan, Cheng Gao, Bradley A. Malin, and You Chen. 2020. Predicting missing values in medical data via xgboost regression. *J. Healthc. Inf. Res.* 4, 4 (01 Dec. 2020), 383–394. <https://doi.org/10.1007/s41666-020-00077-1>

Received 28 March 2023; revised 26 November 2023; accepted 19 January 2024